

Formal Verification of systems with unbounded agents

A thesis submitted in partial fulfillment of
the requirements for the degree of

Doctor of Philosophy

by

Tephilla Prince
(Roll No. CS18DP001)

Under the guidance of
Ramchandra Phawade
and

S. Sheerazuddin



॥ सा विद्या या विमुक्तये ॥

भा.उ०.स०. धारवाड
भा. प्रौ. सं. धारवाड
IIT DHARWAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

March, 2026

To my mentors

Thesis Approval

The thesis entitled

Formal Verification of systems with unbounded agents

by

Tephilla Prince

(Roll No. CS18DP001)

is approved for the degree of

Doctor of Philosophy

Internal Examiner

None

External Examiner

Affiliation:

Supervisor

Ramchandra Phawade

External supervisor

S. Sheerazuddin

Chairman

Date: _____

Place: _____

RPC Approval for Thesis Submission

This is to certify that the thesis titled **Formal Verification of systems with unbounded agents** by Tephilla Prince (CS18DP001) is approved for final submission by the Research Progress Committee.

Name: Ramchandra Phawade

Affiliation: IIT Dharwad

Role: Supervisor

Name: S. Sheerazuddin

Affiliation: NIT Calicut

Role: External supervisor

Name: A. Baskar

Affiliation: BITS, Goa

Role: RPC Member

Name: Sreejith AV

Affiliation: IIT Palakkad

Role: RPC Member

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Tephilla Prince
Roll No. CS18DP001

Date:

Abstract

The complexity and size of software has increased with time. And with it, the necessity to provide mathematical guarantees to prove the behaviour of the software with respect to its requirements has increased. Empirical studies show that the sooner the errors are identified and rectified in the software development lifecycle, the lesser the error costs. If there are errors in the design of the software, they propagate to errors in the implementation which become tedious to resolve. Being able to verify the specifications of these systems in the design stage of the software development (such as using bounded model checking) is an important activity that can be done using formal techniques. The characteristics of the software play a role in the choice of the formal technique to verify a software. Client server systems are one of the largest programming paradigms. Crypto exchanges with an unbounded number of investors, multiplayer games where the number of players are not known apriori and services with an unbounded customer base are instances of unbounded client server systems.

We focus on client server systems with single server and unboundedly many clients, where the number of clients are not known apriori and there can be unbounded concurrent interactions between the clients and server. The major challenges are to identify the suitable model to abstract the behaviour of the systems, to identify suitable logics to specify the properties and to identify formal techniques for verifying the properties on the model. First, we model unbounded client-server systems as unbounded Petri nets and their temporal properties are expressed in Linear Temporal Logic (LTL) with integer arithmetic. In this system, there is unboundedness in two dimensions, namely, the number of clients as well as in the client server interactions. Hence, we propose two dimensional bounded model checking (2D-BMC) for exploiting the unboundedness in these two dimensions and utilize SMT solvers for verification.

Second, when the unbounded client-server systems have distinguishable clients, unbounded Petri nets are no longer suitable to model them. Hence, we explore suitable concurrent models where the tokens are distinguished such as ν -nets. We also require a suitable logic,

which is richer than LTL, to represent their properties naturally, such as a variant of First Order logic. The 2D-BMC algorithm is used to automatically verify the system with these extensions and implemented as a prototype.

Third, clients have their own internal structure. We model each client as a Petri net which interacts with the server Petri net. The interaction of unbounded client server systems can be represented as client Petri nets nested within the server Petri net. This is a type of higher order Petri nets, called Elementary Object Systems (EOS). We also explore the possibility of losing information in the EOS at various nesting levels and their effects on the reachability and coverability problems and chart their decidability status. We built a tool to verify properties such as safety, deadlock and reachability properties using LTL on the standard PN and EOS, as well as their lossy counterparts.

This gives us a suite of formal verification tools for client server systems with unbounded clients using various concurrent models and various suitable logics to represent their properties.

Contents

Abstract	ix
List of Tables	xv
List of Figures	xvii
Glossary	xix
Abbreviations	xxi
Symbols	xxiii
1 Introduction	1
1.1 Infinite State Systems	3
1.2 Contributions	6
2 Models for Unbounded Concurrency	9
2.1 Petri nets	9
2.1.1 Petri net Semantics	12
2.2 Petri net Representation for an SMT Solver	14
2.2.1 ν -nets	18
2.2.2 Elementary object Systems	21
2.2.3 A case study: Autonomous Parking System	25
3 Logics for Unbounded Concurrency	29
3.1 Introduction to Linear Temporal Logic	29
3.1.1 Specifying properties	30
3.1.2 Syntax	31

3.1.3	Semantics	33
3.1.4	Bounded Semantics of LTL	33
3.1.5	Encoding LTL Semantics	35
3.2	LTL with Linear Integer Arithmetic and Counting	35
3.2.1	Two dimensional Bounded Semantics of $LT L_{LIA}$	38
3.3	First Order Logic with Monodic restriction	40
3.3.1	Syntax of $FOTL_1$	42
3.3.2	Two dimensional Bounded Semantics of $FOTL_1$	42
3.4	SMT Encoding for the Logic $FOTL_1$	44
4	Bounded Model Checking	49
4.1	Verification of UCS: Central questions	49
4.2	Bounded Model Checking Algorithms	52
4.2.1	Discussion on the BMC algorithms	53
4.2.2	Related Work	56
5	LTL Tool over Petri nets	59
5.1	Bounded Model Checking for unbounded PNs	59
5.2	Encoding	60
5.3	DCModelchecker : A tool to verify Unbounded PNs	65
5.3.1	Architecture	65
5.3.2	Workflow	66
5.3.3	Pre-processing Petri nets	67
5.4	Benchmarks and Comparisons	71
5.4.1	Experiment 1: Computing the Tool Confidence and Verification of LTL- Fireability properties - DCModelChecker 2.0 vs the state-of-the-art tool	72
5.4.2	Experiment 2: Verification of FireabilityCardinality properties DC- ModelChecker 1.0 vs DCModelChecker 2.0	73
5.4.3	Experiment 3: Verification of Unbounded Petri nets	79
5.4.4	Experiment 4: Verification of invariants for unbounded client-server systems using DCModelChecker 2.0	80
5.4.5	Related Petri net verification tools	80

6	First Order LTL Tool with Monodic restriction	83
6.1	FO LTL over Petri nets with names	83
6.2	Tool and Implementation	83
6.2.1	Observations	87
6.2.2	Experimental results	89
6.2.3	Limitations of the tool	89
6.3	Bounded Semantics of FOTL_1	90
6.3.1	Bounded Semantics Without Loop	91
6.3.2	Bounded Semantics with Loop	93
6.4	Related Work	96
6.A	Appendix: Unbounded semantics of FOTL_1	98
7	Verification of Elementary Object Systems	101
7.1	Introduction	101
7.2	Preliminaries	103
7.2.1	Binary Relations	103
7.2.2	Multisets	103
7.2.3	Petri Nets	103
7.2.4	Elementary Object Systems	104
7.3	Problem	107
7.3.1	Lossy EOSs	107
7.3.2	Lossy-reachability/coverability	109
7.4	Coincident Problems	109
7.5	Distinct Problems	113
7.5.1	Distinct Problems for Object-lossy EOSs	113
7.5.2	Distinct Problems for Full-lossy EOSs	115
7.6	Undecidability for object- and system-lossy cEOSs	116
7.6.1	Reduction to Reachability	116
7.6.2	From Reachability to $(\leq_o, 0)$ -coverability	119
7.6.3	From Reachability to $(\leq_s, 0)$ -coverability	120
7.7	Undecidability for full-lossy EOSs	121
7.8	Decidability of $(\leq_f, \omega)$ -reachability for EOSs	122

7.9	Tool to verify lossy Petri Nets and lossy EOSs	125
7.9.1	Telingo	125
7.10	PNs in Telingo	126
7.10.1	PNs in Logic Programs	126
7.10.2	PN dynamics	127
7.10.3	PN Verification Problems	127
7.10.4	PN experiments	129
7.11	EOSs in Telingo	129
7.11.1	EOSs in Logic Programs	130
7.11.2	EOS dynamics	130
7.11.3	EOS verification	131
8	Conclusions	133
	References	137
	List of Publications	153

List of Tables

5.1	Experiment 1: Categorization of LTLFireability results.	72
5.2	Results of Experiment 3	79
5.3	Experiment 3: List of properties verified on unbounded nets	79
6.1	Experiments on APS	89
7.1	Decidability status of lossy problems for full-, object-, and system-lossy EOS and cEOS. $\mathbb{N}_0$ denotes $\mathbb{N} \setminus \{0\}$. References are put next to already known results. The labels next to our results indicate the techniques used to obtain them: <i>[comp.]</i> - compatibility; <i>[2CM]</i> - 2CM reachability; <i>[cEOS]</i> - generalization of cEOS results; <i>[G]</i> - lossiness-counter gadget $\mathcal{G}$ merging; <i>[WSTS]</i> - WSTS theory.	124
7.2	Comparative Results with TAPAAL for the Eratosthenes-PT-010 PN from the MCC benchmarks [85].	129
7.3	Experimental Results with the Eratosthenes-PT-010 PN from the MCC benchmarks.	129

List of Figures

1.1	A BA accepting the ω -word $\alpha = abbaabababa \dots$	4
1.2	Accepting run of a BA	5
1.3	Bounded Model Checking	5
2.1	A simple Petri net N_0 with marking $\langle 1, 0, 0, 0 \rangle$	9
2.2	A simple Petri net N_0 with marking $\langle 0, 1, 1, 0 \rangle$	11
2.3	A conservative Petri net N_1	11
2.4	Petri net N_0 with marking $\langle 0, 0, 0, 2 \rangle$	13
2.5	Concurrent Firing of Petri net N_1	14
2.6	A restricted ν -net modeling an unbounded client-server system	19
2.7	Petri net N_2 counting the parity of a and b in a word	23
2.8	EOS counting the parity of a and b in a word	23
2.9	Petri net N_3 depicting client behaviour	23
2.10	Petri net N_3 depicting server behaviour	24
2.11	EOS modeling the single server (unbounded) multiple client system	24
2.12	State diagram of server	26
2.13	State diagram of client	26
3.1	Büchi automaton for Gp_1	31
3.2	Büchi automaton for Fp_1	31
3.3	Büchi automaton for $GFp_1 \rightarrow GFp_2$	32
3.4	(k, l) lasso with $l=2, k=4$	34
3.5	A Petri net depicting an unbounded client-server system with invariants	36
3.6	Bounded loop-free path of length λ	38
3.7	Bounded path with (λ, l) - loop	38

3.8	Snapshot of the running example (<i>APS</i>) depicting live windows	42
4.1	An unbounded net	52
4.2	Sequence of firings via interleaving semantics	53
4.3	Sequence of firings via true concurrent semantics	53
5.1	Unfolding of the encoded formula $[\mathcal{M}, \psi]_{(\lambda, \kappa)}$ with respect to λ (execution steps) and κ (number of tokens)	64
5.2	DCModelChecker 2.0 architecture	65
5.3	Pre-Processing the model and formula using ANTLR	65
5.4	Workflow for $k = 0$	67
5.5	Workflow for $k = 1$	67
5.6	Experiment 2: Verification of FireabilityCardinality Properties with bound 5 and 10	74
6.1	Snapshot at bound = 2	90
6.2	Snapshot at bound = 5	90
7.1	EOS in Example 5 with marking $\{\{\langle \text{drone}, \{\{\text{batt1}, \text{batt1}\}\}\rangle\}\}$. The idle transitions are omitted.	105
7.2	Object net <i>drone2</i> with one fully charged 2-bounded battery and a fully discharged one.	108
7.3	Depiction of proof of Th. 9.	112
7.4	The lossiness-counter gadget $\mathcal{G}$ in Def. 7 (where $d(p_1) = N_1, d(p_2) = N_2, d(\text{count}) = \blacksquare$) with initial marking $\mu_0 = \{\{\langle p_1, \{\{p\}\}\rangle\}\}$	113
7.5	Part of $\mathbf{E}_K$ capturing an (a) increment, (b) decrement, or (c) zero-check instruction $i \in \delta$	117
7.6	The places, transitions, and conditions we add on top of $\mathcal{M}$	121
7.7	The PN with initial marking in Ex. 8.	126

Glossary

Formal verification Formal verification is the act of proving or disproving the correctness of a system with respect to a certain formal specification or property.

Model Checking Model checking is a formal verification technique which allows for desired behavioral properties of a given system to be verified on the basis of a suitable model of the system through systematic inspection of all states of the model.

Symbolic Model Checking Symbolic model checking is a methodology where instead of explicitly checking all the states of the system, symbolic model checking algorithms manipulate sets of states.

Petri Net A Petri net is a mathematical modeling tool used to simulate and analyze systems that involve concurrency, synchronization, and resource sharing.

Abbreviations

BMC Bounded Model Checking.

BA Büchi automata.

PN Petri nets.

LTL Linear Temporal Logic.

MFOTL Monadic First Order Temporal Logic.

NNF Negation Normal Form.

PNML Petri Net Markup Language.

XML Extensible Markup Language.

ANTLR ANother Tool for Language Recognition.

MCC Model Checking Contest.

ASP Answer Set Programming.

LP Logic Program.

List of Symbols

α The temporal property that is being verified.

κ Variable used to denote the number of clients which currently are in a given local state.

λ Variable used to denote the execution step.

Chapter 1

Introduction

In today's software ecosystem, it has become imperative to give mathematical guarantees for the behaviour of a software with respect to its requirements. Formal verification is the procedure to ensure the correctness of a system with respect to a desired property by checking whether a mathematical model of the system satisfies a specification. There are various *formal* techniques to verify a software system. The applicability and usage of a formal technique depends on various factors such as, the type of software, the stage of the software development life cycle where verification is performed, the resources available for verification. For instance, in the design stage, it may be possible to verify properties on the abstraction of a system, such as model checking (cf. p8, [13]). In the implementation stage, a more dynamic monitoring of the system behaviour and verification might be required such as runtime verification [20], where one works directly with the system (as opposed to an abstraction of it) and on select execution traces of the system (as opposed to all execution traces as in model checking), or depending on the resources, one might want to perform a check and provide correctness guarantees before deployment of the software such as static analysis [37]. The characteristics of the software system play a huge role in the selection of the formal technique that is adopted, such as if the interactions occur one after the other (sequential behaviour), if the interactions occur in parallel (concurrent behaviour), or if the interactions occur in parallel and can be made to occur one after the other (serializable), or if the system properties can be parameterized. We take the perspective of analyzing suitable ways to formally verify a particular software system.

In this work, we focus on unbounded client server systems where there is unbounded concurrency in the interaction between the clients and the servers as well as unboundedness in the number of clients (the number of clients is not known apriori). A first challenge is to identify

the suitable representation of unbounded client server systems, we explore well known models in literature by varying their expressivity such as Petri nets, ν -nets (Petri nets with names), and Elementary Object Systems (Petri nets with nesting). Another challenge is to identify the suitable way to specify the properties of the unbounded client server systems, we work with various temporal logics such as Linear Temporal Logic, Linear Temporal Logic with integer arithmetic, a variant of First Order Logic with monodic restriction. Now that the system can be represented using a suitable formal model and the property can be specified using an appropriate logic, the consequent challenge is to identify the formal technique by which to verify the system against its specifications. According to studies by NASA and the Systems Sciences Institute at IBM [50, 67], the error costs associated with identifying mismatches between requirements and the implemented software at a later stage of the software development lifecycle grow exponentially as the software goes through its development lifecycle. Therefore, we would like to use model checking to verify the unbounded client server systems at the design stage, in order to give formal guarantees early on, and avoid the high error cost. There are also other formal techniques in the design stage, such as equivalence checking, where one verifies whether two different mathematical models of a design represent the same behaviour. However, this is not interesting in our application.

Model Checking is a formal verification technique which allows for desired behavioral properties of a given system to be verified on the basis of a suitable model of the system through systematic inspection of all states of the model. This is a technique that can be used in the design stage of the software development lifecycle. Model checking is a verification technique that has the following features. The software (to be verified) is abstracted into a suitable mathematical *model* and the various behaviours of the *model* are identified as *states*. Those *states* that are possible from the initial setup of the model are called *reachable states*. As stated earlier, the requirements to be verified are formulated using *temporal logic*, it is temporal, to account for the time based properties. The reachable states of the system are traversed to verify the temporal logic properties. If the property fails, we obtain a sequence of states, known as the counterexample. Hence, it is automatic. Thus, model checking can be summarized as an algorithmic technique for checking temporal properties of systems. The model checking problem is also described as: Given a system P and a specification α on the runs of the system, decide whether system P satisfies specification α . This consists of checking that all runs of P constitute models for α . It suffices to show that no run of P is a model for $\neg\alpha$, which is the same

as checking that the intersection of the language accepted by P and the language defined by $\neg\alpha$ is empty. For complex systems with millions of states, model checking can quickly run into the state space explosion problem [43]. If there was a system with n components and each component contained m states, the composition of those components, would contain m^n states, thereby exponentially incrementing the number of states. Techniques [41, 44] to address this problem are described here. Symbolic Model Checking is a methodology where instead of explicitly checking all the states of the system, symbolic model checking algorithms manipulate sets of states [24]. Another approach is verifying representatives of the system or abstractions of the system wherein parts of the system are abstracted so that only the essential properties being verified are preserved in the abstraction [102]. In compositional verification, each component is verified separately and the overall correctness of the system is inferred [78]. While performing on-the-fly verification, instead of explicitly precomputing all the states, the state space is generated while the verification procedure is underway [66, 68]. There are also a set of partial order reduction techniques. For instance, in asynchronous systems where many events are independent of each other, they can be executed in arbitrary order without affecting the outcome of the computation and some redundancy can be avoided, thereby tackling the state space explosion [64]. There is a notion of unfoldings in partial orders that reduces state space explosion [55]. In counter example guided abstraction refinement, while verifying properties, the counterexamples that are obtained are used to refine the initial abstraction [42].

Among these various techniques, we focus on model checking on *bounded* (finite) runs of the system, and restricting the *state space*. This technique is called *bounded* model checking, which is illustrated in Fig. 1.3. In order to describe the behaviours of infinite state systems, such as unbounded client server systems, it is useful to think of automata that accept languages of infinite words, which we shall explore in the upcoming section.

1.1 Infinite State Systems

In this section, we explore automata that accept languages of infinite words such as Büchi automata (BA) [31, 124]. Later on, we shall describe Petri nets (PN), which can directly represent behaviours of the infinite state systems with concurrency (See Sec. 2.1)¹. The term ω -regular

¹Alternatively, one might make a product automata to accept languages of infinite words where the underlying model allows for concurrency. However, Petri nets are a more elegant representation.

languages, describes the class of languages over infinite words, which is accepted by finite state machines. An infinite word (ω -word) $\alpha \in \Sigma^\omega$, is an infinite sequence of symbols from the alphabet Σ . The infinite word can be represented as a function $\alpha : \mathbb{N}_0 \rightarrow \Sigma$. In order to refer to the letter at the i th position, we use $\alpha(i)$. For a finite sequence $[m \dots n]$, $m, n \in \mathbb{N}_0, m \leq n$, we denote $\{m, \dots, n\}$. Hence, $\alpha[m \dots n]$ denotes the finite word occurring between positions m and n , $\alpha(m)\alpha(m+1) \dots \alpha(n)$. It follows that $[m \dots]$ denotes the infinite word $\alpha(m)\alpha(m+1) \dots$ starting at m . Suppose S is a set and σ is an infinite sequence of symbols over S , $\sigma : \mathbb{N}_0 \rightarrow S$. Then, $\text{inf}(\sigma)$ denotes the set of symbols in S that occur infinitely often in σ . It can be formally defined as: $\text{inf}(\sigma) = \{s \in S \mid \exists^\omega n \in \mathbb{N}_0 : \sigma(n) = s\}$.

Definition 1.1.1. An automaton is defined as a triple $A = (S, \rightarrow, S_{in})$ where S is a finite set of states, $S_{in} \subseteq S$ is the set of initial states, and $\rightarrow \subseteq S \times \Sigma \times S$ is the transition relation. In case of deterministic automaton, $\rightarrow$ is a function from $S \times \Sigma \rightarrow S$.

Definition 1.1.2. A Büchi automata (BA) is denoted by (A, G) , containing an automata $A = (S, \rightarrow, S_{in})$ and its good states $G \subseteq S$. We describe this with Example 1.1.

Definition 1.1.3. A run of a Büchi automata (BA), is a legal sequence of states that an automaton can pass through while reading the input α . Let $A = (S, \rightarrow, S_{in})$ be an automaton and $\alpha : \mathbb{N}_0 \rightarrow \Sigma$ be an input word. A run of A on α is an infinite sequence $\rho : \mathbb{N}_0 \rightarrow S$ such that $\rho(0) \in S_{in}$ and $\forall i \in \mathbb{N}_0, \rho(i) \xrightarrow{\alpha(i)} \rho(i+1)$. A run of A on the finite word $w = a_0a_1 \dots a_m$ is a sequence of states $s_0s_1 \dots s_{m+1}$ such that $s_0 \in S_{in}$ and $\forall i \in [0 \dots m], s_i \xrightarrow{a_i} s_{i+1}$.

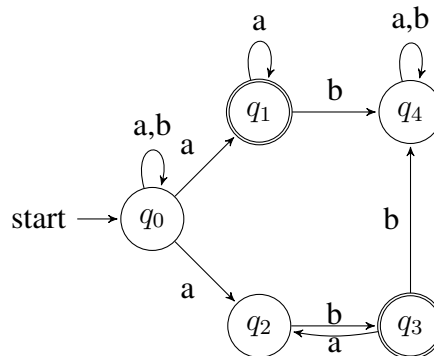


Figure 1.1: A BA accepting the ω -word $\alpha = abbaabababa \dots$

Example 1.1.4. Consider the above figure with $F = \{q_1, q_3\}$ and the ω -word $\alpha = abbaabababa \dots$, some of the possible runs on α are given below:

	a	b	b	a	a	b	a	b	a	b...
q_0	q_0	q_0	q_0	q_0	q_0	q_0	q_2	q_3	q_2	$q_3 \dots$
q_0	q_0	q_0	q_0	q_1	q_1	q_4	q_4	q_4	q_4	$q_4 \dots$
q_0	q_0	q_0	q_0	q_0	q_2	q_3	q_2	q_3	q_2	$q_3 \dots$

Acceptance Condition

Büchi automaton accepts an input if there is a run along which some subset of G occurs infinitely often [87]. Since G is a finite set, it is easy to see that there must actually be a state $g \in G$ that occurs infinitely often along σ . In other words, if we regard the state space of a Büchi automaton as a graph, an accepting run traces an infinite path that starts at some state $s \in S_{in}$, reaches a good state $g \in G$ and loops back to g infinitely often. In the infinite accepting run shown in Fig. 1.2, the move from state s to state g is the non-looping part, that we call *stem*, which ends in the *loop* at state g .

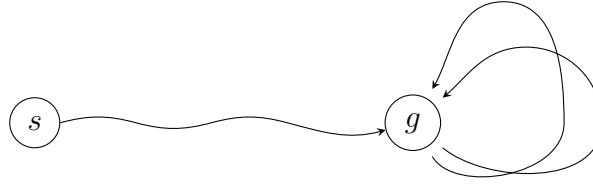


Figure 1.2: Accepting run of a BA

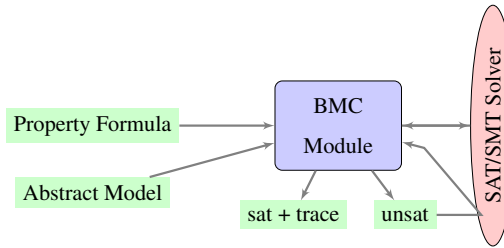


Figure 1.3: Bounded Model Checking

Example 1.1.5. We illustrate bounded model checking in Figure. 1.1, where we would like to check the temporal property that the letter a occurs sometime in the future and reoccurs infinitely often. Now, this can occur, in a non-looping stem part of the run, or in the infinite loop of the run (cf. Fig. 1.2). Notice that the initial state q_0 does not accept the letter a . In order to know, if there is some state, where a is accepted in the future, one needs to look at the next states, apart from the initial state. Using the principle of bounded model checking, we check if $\neg a$ is

accepted in a stem or in loop of all the runs possible in the model. The presence of an accepting run, acts as a counterexample to the property being checked. In this case, the sequence of states $q_0, q_2, q_3, q_2, q_3, \dots$ is a counterexample trace. We can also view this as reaching a state, that accepts the said property.

In Chapter. 3, we describe how to formally specify the properties in a formal manner using a logical language. One drawback is that we cannot explicitly represent concurrency in this model, hence we look at more suitable models to represent concurrency in unbounded client server systems via Petri nets, ν -nets (Petri nets with names), and Elementary Object Systems (Petri nets with nesting) in Chapter. 2. We revisit this notion of automatically performing bounded model checking, while verifying various models and properties in Sec. 5.2 and Sec. 6.A using a set of specialized tools called SAT/SMT solvers. Given a set of constraints, such as in bounded model checking, SAT/SMT solvers can efficiently check if there exists a *satisfying assignment* of values to those constraints. SAT solvers work with Boolean values. SMT solvers offer various underlying theories such as bit vectors and integer arithmetic [17, 51].

1.2 Contributions

In this thesis, we focus on verification of unbounded client server systems and representing them by various suitable formal models as well as verifying the properties of the systems specified using various logics. In this system, there are unbounded interactions between the single server and unboundedly many clients that are not known apriori. The interactions are restricted to requests by the clients that are responded to, by the server. We have not worked with systems where there are interactions between the clients, nor have we worked with the multiple server multiple client systems. These extensions can be interesting future work. The major contributions of this work are the following:

1. **Formulated the 2D-BMC algorithm.** We extend the standard Bounded Model Checking (BMC) algorithm to two dimensional bounded model checking, i.e., 2D-BMC that exploits the two dimensional unboundedness in the Petri nets representing the unbounded client server systems, i.e., unboundedness in the number of clients as well as unbounded concurrency. This is particularly helpful for verifying systems with true concurrency. (See Chapter. 4)

2. **Verification of a Linear Temporal Logic with integer arithmetic properties on Petri nets using 2D-BMC:** Built a tool to verify Linear Temporal Logic with integer arithmetic properties on Petri nets. The tool employs SMT solvers to perform the verification queries using an extension of BMC, called the 2D-BMC algorithm. This tool competes with state of the art Petri Net verification tools and is the first of its kind to use various semantics of Petri nets. (See Chapter. 5)
3. **Verification of a variant of First Order Logic on Petri nets using 2D-BMC:** Implemented a tool to verify properties of First Order Logic with monodic restriction on Petri nets with identifiers, called ν -nets and which employs SMT solvers to perform the verification queries using 2D-BMC algorithm. (See Chapter. 6)
4. **Charted the decidability status of Elementary Object Systems:** The notion of coverability is to ask, given a model and its configuration, is there a second reachable configuration that is smaller than the original configuration. These are important questions in Petri nets and also by extension, in Elementary Object Systems (EOS), which are essentially nets with nesting. In the interactions in the EOSs, imperfect steps are possible, which we call lossiness. Lossiness can occur at various levels of nesting in the EOSs. We charted the decidability status of reachability and coverability problems in EOSs in the case of lossiness at various nesting levels. (See Sec. 7.8)
5. **Verification of reachability of Elementary Object Systems:** The notion of reachability is to ask if a particular configuration can be reached. We implemented an open source tool to verify reachability properties on a class of higher order Petri nets with nesting, called Elementary Object Systems. (See Sec. 7.9)

Chapter 2

Models for Unbounded Concurrency

While formally verifying infinite state systems with concurrency, it is imperative to capture their behaviour in suitable abstract models. In this chapter, we discuss the various abstractions for modeling concurrent systems that are used in the rest of the thesis.

2.1 Petri nets

Petri nets are a commonly used formalism for modeling the dynamic behaviour of systems [98, 103]. Here, we recollect the standard notions associated with nets.

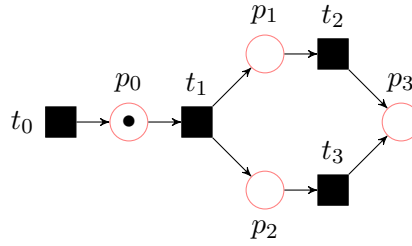


Figure 2.1: A simple Petri net N_0 with marking $\langle 1, 0, 0, 0 \rangle$

Definition 2.1.1. A PN is a tuple $N = (P, T, F)$ where P is a finite set of places, T is a finite set of transitions, $F : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$ is the flow function. Additionally, if the arcs are allowed to have weights, they are called weighted Petri nets. Hence, we also have the weight function $W : F \rightarrow \mathbb{N}_0$, where $\mathbb{N}_0$ is the set of non-negative integers. In Fig. 2.1, $P = \{p_0, p_1, p_2, p_3\}$, $T = \{t_0, t_1, t_2, t_3\}$ and F is represented graphically by the directed arcs.

Definition 2.1.2. The set $\bullet t = \{p \in P \mid F(p, t) > 0\}$ is called the set of pre-places of $t \in T$. The set $t\bullet = \{p \in P \mid F(t, p) > 0\}$ is called the set of post-places of $t \in T$.

For instance, in Fig. 2.1, $\bullet t_0 = \emptyset$, $\bullet t_1 = \{p_0\}$, $\bullet t_2 = \{p_1\}$, $\bullet t_3 = \{p_2\}$. And $t_0^\bullet = \{p_0\}$, $t_1^\bullet = \{p_1, p_2\}$, $t_2^\bullet = t_3^\bullet = \{p_3\}$. The flow function F is also described by the pre and post condition functions.

Definition 2.1.3. Given a PN $N = (P, T, F)$, the pre-condition function pre_N and the post-condition function post_N are defined as follows:

$$\begin{aligned}\text{pre}_N : T &\rightarrow (P \rightarrow \mathbb{N}) & \text{pre}_N(t)(p) &= F(p, t) \\ \text{post}_N : T &\rightarrow (P \rightarrow \mathbb{N}) & \text{post}_N(t)(p) &= F(t, p)\end{aligned}$$

The states of the PN are the distributions of tokens in the places, described by *markings*.

Definition 2.1.4. A marking M of a PN $N = (P, T, F)$ is a function $M : P \rightarrow \mathbb{N}_0$, where $\mathbb{N}_0$ is the set of non-negative integers. A marked PN is a pair $PN = (N, M_0)$ where N is a PN and M_0 is a marking, called initial marking.

In Fig. 2.1 the marking $\langle 1, 0, 0, 0 \rangle$, denotes the distribution of tokens in the places.

Definition 2.1.5 (Enabledness Rule). A transition t is enabled at marking M when $\forall p \in P, M(p) \geq \text{pre}_N(t, p)$.

The *enabledness rule* is a prerequisite for firing a transition t . On the firing of t , the successor of the current marking is obtained according to its pre and post conditions, i.e., by removing $F(p, t)$ tokens from each $p \in \bullet t$ and adding $F(t, p')$ tokens to each $p' \in t^\bullet$, leaving tokens in the remaining places as they are.

Definition 2.1.6 (Firing Rule). Given a marking M in a Petri net, on firing an enabled transition t , we get the successor marking M' , denoted by $M \xrightarrow{t} M'$, such that:

$$\forall p \in P : M'(p) = M(p) - F(p, t) + F(t, p)$$

If there are several enabled transitions at a given marking, exactly one of them is non-deterministically chosen to be fired in the step.

Given the marking in Fig. 2.1 and on firing transition t_1 , the marking in Fig. 2.2 is obtained.

Definition 2.1.7. The set of markings reachable from marking M in a given net N , denoted by $\text{reach}_N(M)$, is the smallest set of markings such that:

- $M \in \text{reach}_N(M)$ and

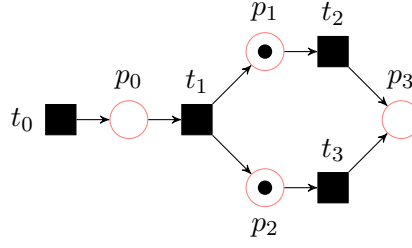


Figure 2.2: A simple Petri net N_0 with marking $\langle 0, 1, 1, 0 \rangle$

- If $M' \xrightarrow{t} M''$ for some $t \in T$ and $M' \in \text{reach}_N(M)$, then $M'' \in \text{reach}_N(M)$.

The reachability graph of a PN N is the directed graph $(\mathcal{N}, E)$ where $\mathcal{N}$ is the set of markings of N , $E \subseteq \mathcal{N} \times \mathcal{N}$, and $(M, M') \in E$ iff there is some $t \in T$ such that $M \xrightarrow{t} M'$. The set of reachable markings of a marked PN (N, M_0) , is $\text{reach}_N(M_0)$.

Definition 2.1.8 (Unbounded Petri Net). An unbounded Petri Net is a Petri Net $N = (P, T, F)$ where P is a finite set of places, T is a finite set of transitions, $F : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$ is the flow function where the $\text{reach}_N(M_0)$ is infinite.

Alternatively, it means that there is no finite integer k such that every place $p \in P$ has fewer than k tokens ($M(p) \leq k$) for all reachable markings M .

Definition 2.1.9 (Conservative Petri net). A PN $N = (P, T, F)$ is conservative iff

$$\forall p \in P \forall t \in T : F(p, t) = F(t, p)$$

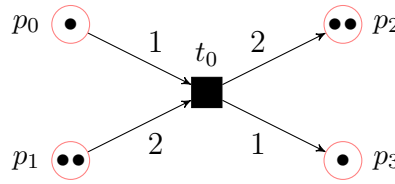


Figure 2.3: A conservative Petri net N_1

A Petri net N is said to be *conservative*, if all transitions fire token-preservingly, i.e., all transitions add exactly as many tokens to their post-places as they subtract from their pre-places. This is illustrated in Fig. 2.3.

Markings are naturally ordered by the coverability relation $\leq$, defined as follows.

Definition 2.1.10. Given two markings M_1 and M_2 , we say that a marking M_2 covers M_1 , denoted by $M_1 \leq M_2$, iff $\forall p \in P, M_1(p) \leq M_2(p)$.

In Petri net N_1 from Fig. 2.1 suppose the marking $M_0 = \langle 0, 0, 0, 0 \rangle$ and on firing t_0 we obtain $M_1 = \langle 1, 0, 0, 0 \rangle$, we say that M_1 covers M_0 .

The *reachability problem* is as follows: given a marked PN (N, M_0) and a marking M , whether $M \in reach_N(M_0)$. For instance, in Fig. 2.3, given the initial marking, M_0 is $\langle 1, 2, 2, 1 \rangle$. We know that the marking $M_1 = \langle 0, 0, 4, 2 \rangle = reach_{N_1}(M_1)$. Hence, $M_1 \in reach_{N_1}(M_1)$.

It is known that the reachability problem for PN is *decidable* but in non-primitive recursive time [47]. The *coverability problem* is as follows: given a marked PN (N, M_0) and a marking M , whether there is a reachable marking $M' \in reach_N(M_0)$ such that $M' \leq M$. It is known that also the coverability problem is decidable and in EXPSpace [113]. Another problem on PN is *deadlock-freeness*.

Definition 2.1.11 (Dead Marking). *A marking M for net N is dead if transition t is not enabled at M for all $t \in T$. The marked PN (N, M_0) is deadlock-free, if there is no dead marking in $reach_N(M_0)$.*

For instance, in Fig. 2.1, if the net is modified such that the transition t_0 is not present, it would result in a deadlock after obtaining the marking $\langle 0, 0, 0, 2 \rangle$. The *deadlock-freeness* problem is as follows: given a marked PN N , whether N is deadlock-free. It is well-known that deadlock-freeness and reachability are recursively equivalent and, thus, deadlock-freeness is decidable but in non-primitive recursive time complexity [39, 65].

2.1.1 Petri net Semantics

In this section, we look at the formal semantics of Petri nets, in particular the interleaving and concurrent semantics. The key difference in the two approaches, is in the firing condition. We recall the firing rule:

Given an enabled transition t at marking M , on firing t , we get the successor marking M' , denoted by $M \xrightarrow{t} M'$, such that:

$$\forall p \in P : M'(p) = M(p) - F(p, t) + F(t, p)$$

In interleaving semantics, there is atmost one transition that is fired in an instance or step of the occurrence sequence. Given an initial marking $M_0 = \langle 0, 0, 0, 0 \rangle$, on firing t_0 twice and t_1 once, we obtain this sequence $\langle 0, 0, 0, 0 \rangle \xrightarrow{t_0} \langle 1, 0, 0, 0 \rangle \xrightarrow{t_0} \langle 2, 0, 0, 0 \rangle \xrightarrow{t_1} \langle 0, 1, 1, 0 \rangle$ as depicted in Fig. 2.2.

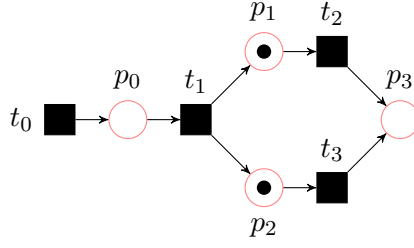


Figure 2.2: Recall the simple Petri net N_0 with marking $\langle 0, 1, 1, 0 \rangle$

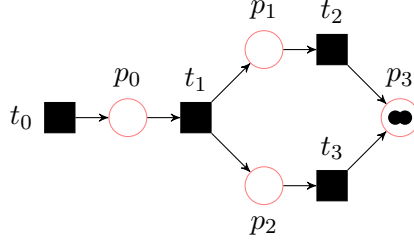


Figure 2.4: Petri net N_0 with marking $\langle 0, 0, 0, 2 \rangle$

Now, both transitions t_2 and t_3 are enabled. In interleaving semantics, one may fire either of the two transitions, one after the other. However, in concurrent semantics, the following sequence is also additionally possible in a single step, resulting in Fig. 2.4.

$$\langle 0, 1, 1, 0 \rangle \xrightarrow{t_2, t_3} \langle 0, 0, 0, 2 \rangle$$

Definition 2.1.12 (Concurrent Firing Rule). *Given a Petri net N and a marking M having a set of enabled transitions $\tau \in T$, we obtain the successor marking M' on concurrent firing of a subset of transitions $\tau' \subseteq \tau$ using the following rule:*

$$\forall p \in \bullet\tau' : M'(p) = M(p) - \sum_{t \in \tau'} F(p, t) + \sum_{t \in \tau'} F(t, p) \text{ if } \sum_{t \in \tau'} F(p, t) - M(p) \geq 0.$$

The condition $\sum_{t \in \tau'} F(p, t) - M(p) \geq 0$ takes care of the sufficiency of tokens at each of the pre-places of the transitions in τ' such that the transitions can be fired. This is a particularly elegant rule in the context of representing concurrent firing of transitions in a Petri net in a SMT solver. This automatically addresses scenarios where the firing of one transition might disable another, the resultant set of transitions that are subsequently fired are all valid transitions that are enabled and whose concurrent firing is allowed by the Petri net semantics.

In Fig. 2.5, $P = \{p_0, p_1, p_2, p_3\}$ and $T = \{t_0, t_1, t_2\}$. Given an initial marking $M_0 = \langle 1, 2, 0, 0 \rangle$, the set of enabled transitions w.r.t M_0 , is $\tau = \{t_0, t_1, t_2\}$. Suppose $\tau' = \{t_1, t_2\}$, then $\bullet\tau' = \{p_1\}$ and the step $\langle 1, 2, 0, 0 \rangle \xrightarrow{t_1 \& t_2} \langle 0, 0, 1, 1 \rangle$ is allowed by the concurrent firing rule

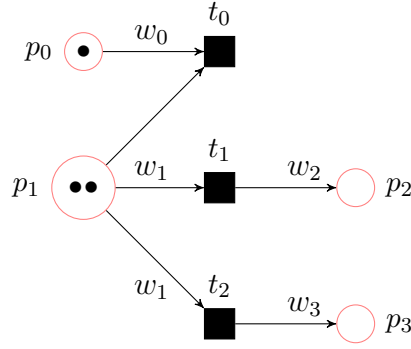


Figure 2.5: Concurrent Firing of Petri net N_1

stated above. However, suppose $M_1 = \langle 1, 1, 0, 0 \rangle$, $\tau' = \{t_1, t_2\}$ and $\bullet\tau' = \{p_1\}$, then the step $\langle 1, 1, 0, 0 \rangle \xrightarrow{t_1 \& t_2}$ is not allowed by the same rule.

Remark: Given a PN where transitions $t_1, t_2, \dots, t_n$ are enabled and there are sufficient tokens in the pre places of $t_1, t_2, \dots, t_n$ such that the transitions may be fired, in the context of true concurrent semantics, we wish to fire maximal number of transitions, wherever possible. Consequently, this means that if there are intermediate markings, which are reachable by an interleaved firing of a subset of the transitions $t_1, t_2, \dots, t_n$, then those markings become unreachable when executing in a truly concurrent manner.

2.2 Petri net Representation for an SMT Solver

The bigger picture is to verify properties of the Petri net using an SMT solver [19]. Satisfiability of formulas with respect to a particular theory such as arrays, integer arithmetic, bit vectors is called as Satisfiability Modulo Theory. The tools employing the decision procedures and programs to solve these satisfiability constraints are known as SMT solvers. Well known SMT solvers include Z3, CVC4 [18, 51]. In this section, we discuss the representation of nets such that they can be added as constraints in an SMT solver, namely Z3. The Petri net representation with interleaving semantics and the true concurrent semantics differ. While verifying infinite state systems, [5] employed backwards reachability for proving safety properties, and in case of concurrent programs, [6] proved liveness and termination via backwards reachability. In the unbounded Petri net setting, [3] is the last known work where unfoldings (in the sense of Ken McMillan [94]) are discussed from a verification point of view. In state of the art Petri net verification tools, interleaving semantics is adopted [9]. We take an alternate perspective of this and unfold unbounded Petri nets with true concurrency using the help of SMT solvers. In this

thesis, by unfolding, we refer to the process of obtaining the subsequent configurations from the initial marking of the net. This is not to be confused with the notion of unfolding described by Ken McMillan in his seminal work on partial orders [94].

We outline the variables and data structures that are necessary to describe the true concurrent semantics. We have a finite set of transitions, $t_0, \dots, t_m$ described in the net, their names are in the list $tNames$. We have a finite set of places, $p_0, \dots, p_l$ their names are in the list $pNames$. Arcs can be of either of two types: where the source is a transition and the target is a place or the source is a place and the target is a transition. The net formalism does not allow arcs between places and between transitions themselves. If they occur, the net description is erroneous, and we cannot move ahead with the unfolding.

We use the vector $iWt[k]$ to store the expressions with respect to k , to compute the incident weights of transitions. We use the vector $Wt[k]$ to store the expressions with respect to k , to denote the change in weights for places. We use two-dimensional matrices $Wt[m][l]$ and $iWt[m][l]$ to denote the net change in weights and the incidence weights (outgoing from places). The expressions for the same are stored in $WTVars$ and $iWTVars$ respectively.

We construct the two-dimensional weight matrix $Wt[row][col]$ of size $m \times l$ which contains the net weight of the arcs. Initially, all the matrix entries are initialized to zero.

For an arc from place p_i to transition t_j with the weight w , we have the matrix entry $Wt[i][j] = Wt[i][j] - w$. For an arc from transition t_j to place p_i with the weight w , we have the matrix entry $Wt[i][j] = Wt[i][j] + w$. Now, if there are incoming and outgoing arcs of the same weights, then the net weight $Wt[i][j] = 0$. Notice that, by looking only at the Wt matrix, we may not distinguish between the case where there are no arcs to and from an element of the net. Hence, it is necessary to have a separate data structure for the same. We have a two-dimensional matrix $iWt[row][col]$ of size $m \times l$ which contains the incident weights to the transitions. For an arc from place p_i to transition t_j with the weight w , we have the matrix entry $iWt[i][j] = Wt[i][j] - w$. The marking of the net consists of the set tokens at each place and represents the state of the net at any instance. The initial marking of the net can be obtained from the net description and contains the number of tokens in each of the places $p_0, \dots, p_l$ in the net. The subsequent markings may be constructed from the matrix Wt and using the transition function of the net. The transition function TF describes the behaviour of the net. The initial marking of the net is stored in *initial*. We introduce a method *printTFTruthTable* that can aid to visualise the transition function TF which is an expression describing the function. Most

utility methods and the 2 – *DBMC* algorithm are similar to that of the interleaving semantics. For experimentation, we have three different versions for constructing the Transition Function which are equivalent to each other (as verified by truth tables, experiments with 352 properties) and are a simplification of the expression using $\wedge$, $\vee$ instead of $\implies$ and so on. Experiments suggest that one version is slightly faster than the others. The variable $T[ti]$ denotes the i th transition being fired. Hence the expression $\neg T[ti]$ denotes that the i -th transition is not fired. For every pair of transitions and places, if there are no outgoing arc from the transition t_i then the expression *preCond* containing the precondition for firing of transitions is constructed as follows:

$$if(emptyOutTi)\{ \tag{2.1}$$

$$preCond = (tmp == iWt[pi][ti] \vee tmp == 0) \tag{2.2}$$

$$\wedge ((T[ti] \wedge tmp == iWt[pi][ti]) \tag{2.3}$$

$$\vee (\neg T[ti] \wedge tmp == 0)) \tag{2.4}$$

$$cumulativeIncidentW = tmp \tag{2.5}$$

$$emptyOutTi = \neg emptyOutTi\} \tag{2.6}$$

$$else\{ \tag{2.7}$$

$$preCond = preCond \wedge (tmp == iWt[pi][ti] \vee tmp == 0) \tag{2.8}$$

$$\wedge ((T[ti] \wedge tmp == iWt[pi][ti]) \tag{2.9}$$

$$\vee (\neg T[ti] \wedge tmp == 0)) \tag{2.10}$$

$$cumulativeIncidentW = cumulativeIncidentW + tmp\} \tag{2.11}$$

The sum of incident weights at a transition t_i is stored in *cumulativeIncidentW* as an expression of the $iWt[pi][ti]$ if there is an arc from the transition t_i to place p_i or it is zero. If the transition is not fired, then there are no weights to be considered.

If the expression is non-empty, the previously constructed *preCond* are **anded** with the newly constructed expression and the cumulative incident weights are updated in the same manner.

The post condition is constructed if the weight $Wt[pi][ti] \neq 0$.

$$\begin{aligned}
postCond &= ((tmp == Wt[pi][ti]) \vee (tmp == 0)) \\
&\wedge ((T[ti] \wedge tmp == Wt[pi][ti]) \\
&\vee (!T[ti] \wedge tmp == 0)) \\
cumulativeWChange &= tmp \\
emptyChangePi &= !emptyChangePi
\end{aligned}$$

Based on the above expressions, we construct the transition function TF

```

if (emptyTF) {
    if (!emptyOutTi) {
        TF = preCond & (Px[pi] + cumulativeIncidentW >= 0)
        emptyTF = false}
    if (!emptyChangePi) {
        if (!emptyOutTi)
            TF = TF & postCond
            & (Py[pi] == Px[pi] + cumulativeWChange)
        else
            TF = postCond & (Py[pi] == Px[pi]
                + cumulativeWChange)
        emptyTF = false
    }
}
else{
    if (!emptyOutTi)
        TF = TF & preCond
        & ((Px[pi] + cumulativeIncidentW) >= 0)
    if (!emptyChangePi)
        TF = TF & postCond
        & (Py[pi] == (Px[pi] + cumulativeWChange))
}

```

The transition function is a conjunction of the preconditions, postconditions and the change in the markings. In case of interleaving semantics, there is an additional conjunction to the

transition function, a disjunction of each transition t_i , to ensure that exactly one transition is fired in a step.

2.2.1 ν -nets

In the rest of this thesis, we consider the specific setting of single server multiple client systems, with distinguishable clients. These distinguishable clients are represented using distinguishable tokens of the Petri net, i.e., tokens appended with identifiers [114]. There are an unbounded number of clients and a fresh client identifier is issued whenever a new client enters the system. When clients exit, the identifiers need to be purged. While the case study is discussed in detail in Chapter. 4, here, we outline the requirements for the formal model and arrive at a suitable representation.

As seen in the Sec. 2.1, Petri nets are suitable to model the *concurrent* behaviour of the clients and are also suitable to capture an unbounded number of clients. The places correspond to the local states of the client and server. We have a disjoint set of server places and client places. The combined interactions of the server and client processes are represented by transitions. The tokens correspond to the processes (server process, client process). Unbounded Petri nets with indistinguishable tokens are not sufficient to differentiate between processes (server process, client process). Hence we look for another model. At first glance, a candidate model is the colored Petri net (CPN) [77] which satisfies the above requirements. CPNs allow arbitrary expressions over user-defined syntax labelling the arcs, and the underlying modeling language (such as CPN Modeling Language in CPN Tools [76]) is highly expressive. However, we do not prefer the CPN, for the following engineering reasons.

First, there is a dearth of tools to automatically *unfold* unbounded colored Petri nets. Recall that the big picture is the automatic verification of unbounded client-server systems. Alternatively, suppose we represent the formal model as a CPN such as using CPN Tools, there are no existing tools that can automatically *unfold* an *unbounded* CPN created using CPN Tools. Existing tools can only unfold *bounded* CPNs [25, 48]. The second candidate model is a type of ν -net [115], which is a CPN defined over a system of component nets, which use a labelling function λ , to handle synchronization between multiple component nets of a larger net system.

We restrict the ν -nets to a single component, providing a simplified definition while doing away with the labelling function used in ν -nets. The client behaviour can be represented as a state machine. Similarly the server behaviour can also be represented as a state machine. In this

representation, we describe the behaviour of the single server multiple client system as a single component.

We begin with some definitions that are necessary for describing the restricted ν -net. Given an arbitrary set A , we denote by $\mathcal{MS}(A)$, the set of finite multisets of A , given by the set of mappings $m : A \rightarrow \mathbb{N}$. We denote by $S(m)$ the support of m , defined as follows: $S(m) = \{a \in A \mid m(a) > 0\}$. Distinguishable tokens (identifiers) are taken from an arbitrary infinite set Id . To handle this, we add matching variables labeling the arcs, taken from a set Var . To handle the movement of tokens inside the ν -net, we employ a finite set of variables, Var using which we label the arcs of the net. There is a special variable $\nu \in Var$, which introduces tokens into the place, which is described later.

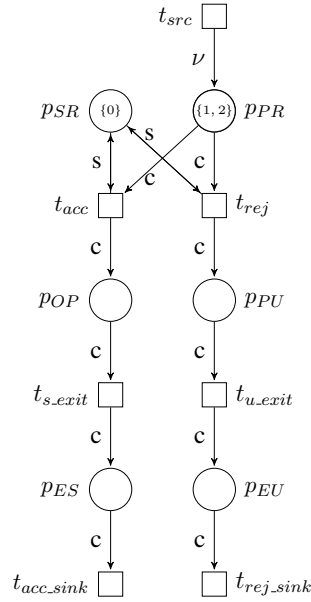


Figure 2.6: A restricted ν -net modeling an unbounded client-server system

Definition 2.2.1. A ν -net is a coloured Petri net $N = (P, T, F)$, where

- P and T are finite disjoint sets of places and transitions, respectively,
- $F : (P \times T) \cup (T \times P) \rightarrow \mathcal{MS}(Var)$ defines the set of arcs of the net, satisfying $\nu \notin pre(t)$ for every $t \in T$.

For a transition t of the net, we define, $post(t) = \bigcup_{p \in P} S(F(t, p))$, $pre(t) = \bigcup_{p \in P} S(F(p, t))$ and $Var(t) = pre(t) \cup post(t)$.

For instance, in Fig. 2.6, $Var = \{\nu, s, c\}$, $T = \{t_{acc}, t_{rej}, t_{s_exit}, t_{u_exit}, t_{acc_sink}, t_{rej_sink}\}$ and $P = \{p_{PR}, p_{SR}, p_{OP}, p_{PU}, p_{ES}, p_{EU}\}$. $pre(t_{acc}) = \{s, c\}$ and $post(t_{acc}) = \{s, c\}$, hence $Var(t_{acc}) = \{s, c\}$.

Definition 2.2.2 (Marking). *A marking of a restricted ν -net $N = (P, T, F)$ is a function $M : P \rightarrow (\mathcal{MS}(Id))$.*

In Fig. 2.6, the initial marking $M_0 = \langle \{0\}, \{1, 2\}, \emptyset, \emptyset, \emptyset, \emptyset \rangle$.

Definition 2.2.3 (Mode). *We denote by $S(M)$ the set of identifiers in M . i.e., $S(M) = \bigcup_{p \in P} S(M(p))$. A mode of a transition t is a mapping $\sigma : Var(t) \rightarrow Id$, instantiating every variable in the adjacent arcs of t to some identifier.*

Let N be a restricted ν -net and M a marking of N according to Defn. 2.2.1.

Definition 2.2.4 (Enabling Rule). *We say that M **enables** the transition t with mode σ whenever:*

- *If $\nu \in Var(t)$ then $\sigma(\nu) \notin S(M)$ and*
- *$\sigma(F(p, t)) \subseteq M(p)$ for all $p \in P$.*

Notice that if $\sigma(\nu) \notin S(M)$ for the enabling of transition, that causes the creation of fresh (equal) identifiers in all the places reached by arcs labelled by the special variable $\nu \in Var$ that appears only in post-condition arcs.

Definition 2.2.5 (Firing Rule). *The reached marking of net N after firing of t with mode σ is denoted by $M \xrightarrow{t(\sigma)} M'$, where $\forall p \in P : M'(p) = M(p) - \sigma(F(p, t)) + \sigma(F(t, p))$.*

In Fig. 2.6, given initial marking $M = \langle \{0\}, \{1, 2\}, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ and $M \xrightarrow{t_{acc}} M'$, $M' = \langle \{0\}, \{2\}, \{1\}, \emptyset, \emptyset, \emptyset \rangle$, i.e., the token $\{1\}$ has moved to the place P_{OP} . The mode is represented in the figure.

In Fig. 2.6, the transitions t_{acc} , t_{rej} , t_{s_exit} , t_{u_exit} , t_{acc_sink} , t_{rej_sink} , represent the accept, reject, exit successfully, exit unsuccessfully and the two sink transitions respectively. A transition is **identifier-preserving** if $post(t) \setminus \{\nu\} \subseteq pre(t)$. Here, all of them are *identifier-preserving* transitions, which ensures that the system with identified clients is represented correctly modeled. The firing of transition t_{src} acts as the source. The arc labelled ν ensures that a new client

identifier is generated in place p_{PR} . The place p_{PR} contains a set of clients requesting for parking. In the unsuccessful scenario, the transition t_{rej} is fired when the server rejects the request, which brings the vehicle to *parking_unavailable* state represented by place p_{PU} . On firing of transition t_{u_exit} , the vehicle goes to *exited_unsuccessfully* state represented by place p_{EU} . The firing of transition t_{rej_sink} is the sink transition for the rejected parking requests. This ensures that the rejected vehicle identifier exits the system and is never reused. If the client arrives after it has exited, it is always issued a fresh identifier. Notice that there are arcs labelled s to indicate the server which has identifier 0, which is necessary for the acceptance or rejection of a parking request. The token with identifier 0 is permanently present in each marking exactly at server place p_{SR} . The ν arc ensures that new identifiers are generated, essentially giving an unbounded number of agents in the ν -net. The arcs labelled c carry the client identifiers from one client place to another. The net behaves as a standard ν -net component with autonomous transitions as described in [115]. We explored simple Petri nets, with identifiable tokens. Next, we shall explore an extension of Petri nets, suitable for modeling real world software systems.

2.2.2 Elementary object Systems

In [126, 127], they introduce the nets-within-nets paradigm, wherein, the tokens of the Petri net can be nets themselves. They are used to model the dynamic nature of token behaviour. This is synonymous with the object-oriented modeling approach introduced by Booch [116], which most software systems, including client-server systems follow.

In nets-within-nets, one can restrict the nesting of nets, to a nesting depth of two, to obtain Elementary Object Systems (EOSs). Here, the level 0 net is called the system net and the (nested) level 1 net is called the object net. In our running example, of the single server multiple clients, we represent server behaviour by the system net and the client behaviour by the object nets that are nested within the system net. In EOSs, we have events, that enable firing of transitions in the system net or object net or at both levels, i.e., system autonomous, object autonomous and synchronized events. Their formal definition is given below:

Definition 2.2.6 (EOS). An EOS $\mathfrak{E}$ is a tuple $\mathfrak{E} = \langle \hat{N}, \mathcal{N}, d, \Theta \rangle$ where:

1. $\hat{N} = \langle \hat{P}, \hat{T}, \hat{F} \rangle$ is a PN called system net; $\hat{T}$ contains a special set $ID_{\hat{P}} = \{id_p \mid p \in \hat{P}\} \subseteq \hat{T}$ of idle transitions such that, for each distinct $p, q \in \hat{P}$, we have $\hat{F}(p, id_p) = \hat{F}(id_p, p) = 1$ and $\hat{F}(q, id_p) = \hat{F}(id_p, q) = 0$.

2. $\mathcal{N}$ is a finite set of PNs, called object PNs, such that $\blacksquare \in \mathcal{N}$ and if $(P_1, T_1, F_1), (P_2, T_2, F_2) \in \mathcal{N} \cup \hat{N}$,¹ then $P_1 \cap P_2 = \emptyset$ and $T_1 \cap T_2 = \emptyset$.
3. $d : \hat{P} \rightarrow \mathcal{N}$ is called the typing function.
4. Θ is a finite set of events where each event is a pair $(\hat{\tau}, \theta)$, where $\hat{\tau} \in \hat{T}$ and $\theta : \mathcal{N} \rightarrow \bigcup_{(P,T,F) \in \mathcal{N}} T^\oplus$, such that $\theta((P, T, F)) \in T^\oplus$ for each $(P, T, F) \in \mathcal{N}$ and, if $\hat{\tau} = id_p$, then $\theta(d(p)) \neq \emptyset$.

Definition 2.2.7 (Nested Markings). Let $\mathfrak{E} = \langle \hat{N}, \mathcal{N}, d, \Theta \rangle$ be an EOS. The set of nested tokens $\mathcal{T}(\mathfrak{E})$ of $\mathfrak{E}$ is the set $\bigcup_{(P,T,F) \in \mathcal{N}} (d^{-1}(P, T, F) \times P^\oplus)$. The set of nested markings $\mathcal{M}(\mathfrak{E})$ of $\mathfrak{E}$ is $\mathcal{T}(\mathfrak{E})^\oplus$. Given $\lambda, \rho \in \mathcal{M}(\mathfrak{E})$, we say that λ is a sub-marking of μ if $\lambda \sqsubseteq \mu$.

Note that λ is a sub-marking of μ iff there is some nested marking μ' such that $\mu = \lambda + \mu'$. EOSs inherit the graphical representation of PNs with the provision that we represent nested tokens via a dashed line from the system net place to an instance of the object net where the internal marking is represented in the standard PN way. However, if the nested token is $\langle p, \varepsilon \rangle$ for a system net place p of type $\blacksquare$, we represent it with a black-token $\blacksquare$ on p . If a place p hosts $n > 2$ black-tokens, then we represent them by writing n on p . Each event $\langle \hat{\tau}, \theta \rangle$ is depicted by labeling $\hat{\tau}$ by $\langle \theta \rangle$ (possibly omitting double curly brackets). If there are several events involving $\hat{\tau}$, then $\hat{\tau}$ has several labels.

Example 1. Fig. 2.7 depicts the Petri net which (1) counts the parity of the a 's and b 's in the input word (firing sequence) (2) on seeing an a , the parity of the alphabet changes and the token moves to the place with a_O (3) on seeing an b , the parity of the alphabet changes and the token moves to the place with b_O . The marking $M_0 = \langle 1, 0, 0, 0 \rangle$ depicts the empty word, with exactly a token in place $a_E b_E$. For instance, given the firing sequences $\sigma = abba$, $M_0 \xrightarrow{\sigma} M_0$. Fig. 2.8 depicts the equivalent EOS which counts the parity of the a 's and b 's in the input word.

We adopt the technique in Example. 1, to the running example, single server multiple client systems.

Example 2. Fig. 2.9 depicts the client behaviour as a Petri net, where the places CR , OP , EU and ES represent *generate client request*, *occupy parking lot*, *exit unsuccessfully* and *exit successfully* respectively. Fig. 2.10 depicts the server behaviour as a Petri net where the places SR , ACR , RR represent *server ready*, *accept client request*, *reject request* respectively. The client behaviour can be modeled as an object net and the system places can model the server

¹This way, the system net and the object nets are pairwise distinct.

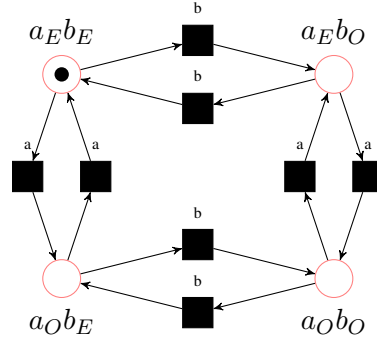


Figure 2.7: Petri net N_2 counting the parity of a and b in a word

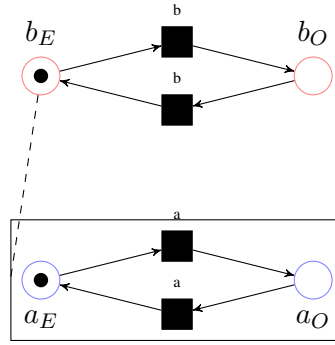


Figure 2.8: EOS counting the parity of a and b in a word

places, and we can obtain an EOS to model the combined behaviour of the single server multiple client system, where the clients are not identified, but an unbounded number of them may be generated. This is shown in Fig. 2.11. Notice that on firing the source transition, there are an unbounded number of object nets that can be generated in the EOS. The server places are all of the same type, which enable to movement of the object tokens until exit.

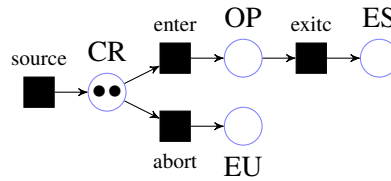


Figure 2.9: Petri net N_3 depicting client behaviour

Notice that the types of the system places are *minimal*, as there is exactly one type of object net, to depict the client type. Technically, $\mathcal{N} = \{\text{client}, \blacksquare\}$ (even if $\blacksquare$ is unused), $d(\text{SR}) = d(\text{ACR}) = d(\text{RR}) = \text{client}$, and Θ synchronizes accept, reject and exit in $\hat{N}$ with enter, abort and exit in client. Formally, $\Theta = \{\langle \text{accept}, \{\{\text{enter}\}\} \rangle, \langle \text{reject}, \{\{\text{abort}\}\} \rangle, \langle \text{exit}, \{\{\text{exitc}\}\} \rangle\}$.

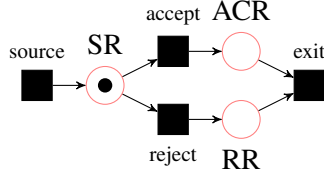


Figure 2.10: Petri net N_3 depicting server behaviour

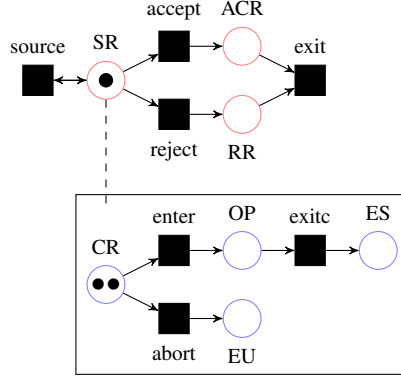


Figure 2.11: EOS modeling the single server (unbounded) multiple client system

The marking $\mu = \langle \text{client}, \{\{\text{client1}, \text{client2}\}\} \rangle$ represents a server instance at SR, with two client requests, waiting to be serviced.

When firing an event $\langle \tau, \theta \rangle$, nested tokens in the system net are consumed according to the preconditions of τ in the standard PN way. At the same time, for each object net N , the inner tokens are merged so as to obtain a PN marking $\mu(N)$ for N (possibly empty). Then, transitions in $\theta(N)$ are fired in the standard PN way obtaining markings $\mu'(N)$. Next, nested markings with empty inner markings are produced in the system net according to the postconditions of τ . Finally, the markings $\mu'(N)$ are non-deterministically distributed among the empty nested tokens, according to the typing function. To be fired, the event must be enabled at both the system and at the object net level. This is captured by the enabledness condition, which makes use of projection operators at the system (Π^1) and at the object net level (Π_N^2 for each $N \in \mathcal{N}$).

Definition 2.2.8 (Projection Operators). *Let $\mathfrak{E}$ be an EOS $\langle \hat{N}, \mathcal{N}, d, \Theta \rangle$. The projection operators Π^1 maps each nested marking $\mu = \sum_{i \in I} \langle \hat{p}_i, M_i \rangle$ for $\mathbf{E}$ to the PN marking $\sum_{i \in I} \hat{p}_i$ for $\hat{N}$. Given an object net $N \in \mathcal{N}$, the projection operators Π_N^2 maps each nested marking $\mu = \sum_{i \in I} \langle \hat{p}_i, M_i \rangle$ for $\mathbf{E}$ to the PN marking $\sum_{j \in J} M_j$ for N where $J = \{i \in I \mid d(\hat{p}_i) = N\}$.*

To define the enabledness condition, we need the following notation. We set $\text{pre}_N(\theta(N)) = \sum_{i \in I} \text{pre}_N(t_i)$ where $(t_i)_{i \in I}$ is an enumeration of $\theta(N)$ counting multiplicities.

We analogously set $\text{post}_N(\theta(N)) = \sum_{i \in I} \text{post}_N(t_i)$.

Definition 2.2.9 (Enabledness Condition). *Let $\mathfrak{E}$ be an EOS $\langle \hat{N}, \mathcal{N}, d, \Theta \rangle$. Given an event $e = \langle \hat{\tau}, \theta \rangle \in \Theta$ and two markings $\lambda, \rho \in \mathcal{M}(\mathfrak{E})$, the enabledness condition $\Phi(\langle \hat{\tau}, \theta \rangle, \lambda, \rho)$ holds iff*

$$\begin{aligned} \Pi^1(\lambda) = \text{pre}_{\hat{N}}(\hat{\tau}) \wedge \Pi^1(\rho) = \text{post}_{\hat{N}}(\hat{\tau}) \wedge \forall N \in \mathcal{N}, \Pi_N^2(\lambda) \geq \text{pre}_N(\theta(N)) \wedge \\ \forall N \in \mathcal{N}, \Pi_N^2(\rho) = \Pi_N^2(\lambda) - \text{pre}_N(\theta(N)) + \text{post}_N(\theta(N)) \end{aligned}$$

The event e is enabled with mode (λ, ρ) on a marking μ iff $\Phi(e, \lambda, \rho)$ holds and $\lambda \sqsubseteq \mu$. Its firing results in the step $\mu \xrightarrow{(e, \lambda, \rho)} \mu - \lambda + \rho$.

Notice that in Fig. 2.11, the transitions `accept` and `reject` are both enabled due to two client object nets in place `SR` and the object net places `enter`, `abort` are enabled as well. Depending on the fired transition in the system net, the corresponding synchronizing move is taken in the object net as well, in the synchronizing event.

Definition 2.2.10 (Reachability in EOS). *The reachability problem for EOSs is defined in the usual way, i.e., whether there is a run (sequence of event firings) from an initial marking μ_0 to a target marking μ_f . $\mu_1 = \langle \text{client}, \{\{\text{client1}, \text{client2}\}\} \rangle$ is a reachable marking.*

Definition 2.2.11 (Coverability in EOS). *The coverability definition is standard, but with respect to the order $\leq$ (in [80]), the component wise ordering relation. For instance, $\mu_1 = \langle \text{client}, \{\{\text{client1}, \text{client2}\}\} \rangle$ and $\mu_2 = \langle \text{client}, \{\{\text{client1}, \text{client2}, \text{client3}\}\} \rangle$. Then, $\mu_1 \leq \mu_2$.*

2.2.3 A case study: Autonomous Parking System

We consider as the running example, the Autonomous Parking System (APS) that manages parking lots and solves the search for parking lots by vehicles in crowded spaces through communication between the system (server) and the vehicle (client). This system has been successfully implemented by the industry [33, 132]. This is a type of single server multiple client system, where the clients are unbounded. The state diagrams for the server and client are given in Fig. 2.12 and Fig. 2.13 respectively. The combined interactions between clients and the server are modelled as a Petri net in Fig. 2.6. In this Petri net there are two types of places—client places and server places that model the client states and server states respectively. There

is at most one token in the server place, and there can be an unbounded number of tokens in the client places. The number of tokens in the client places exactly corresponds to the number of clients in that state at any point in time.

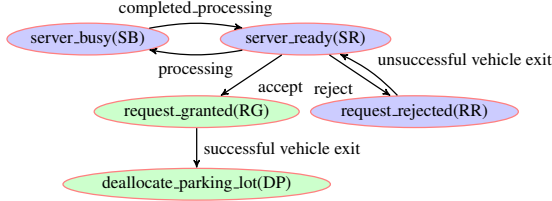


Figure 2.12: State diagram of server

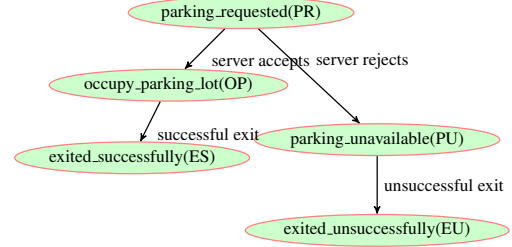


Figure 2.13: State diagram of client

Initially, the system is in the state *server_ready* (*SR*), which is ready to service the requests. To keep the system sufficiently occupied, we assume a steady inflow of requests for parking. When a client inquires about parking space, the client is in the *parking_requested* (*PR*) state. The server may non-deterministically choose to either grant or reject the parking request based on local information such as space availability, the priority of incoming requests, etc. We assume two disjoint workflows for each scenario. First, if the server accepts the request, the server is in *request_granted* (*RG*) state and simultaneously, the client goes to *occupy_parking_lot* (*OP*) state. At some point, the client gives up its allocated parking space, is in *exit_parking_lot_successfully* and simultaneously the server is in *deallocate_parking_lot* (*DP*) state. This marks the successful exit of the client from the system. Second, if the server rejects the request, the client is in *parking_unavailable* (*PU*) state and the server is in *request_rejected* (*RR*) state. The only option is for the client to exit. At any point, the server can either accept or reject the request. After granting the request, the server can go to *server_busy* (*SB*) state. Theoretically, this description allows for an unbounded number of client requests to be processed by the server, albeit there may be limitations on the availability of parking space. We assume that the autonomous parking system can reasonably guide the vehicle manoeuvres within the parking lot. It is not difficult to observe that the combined interactions between the server and clients described above can be interleaved and modelled as a single ν -net as in Fig. 2.6.

In this ν -net, the places p_{PR} , p_{OP} , p_{PU} , p_{ES} , p_{EU} corresponds to the client states *parking_requested*, *occupy_parking_lot*, *parking_unavailable*, *exit_successfully*, *exit_unsuccessfully* and the place p_{SR} corresponds to the server state *server_ready* respectively. The transitions in the net correspond to the transitions in the state diagrams of the server and client. The tran-

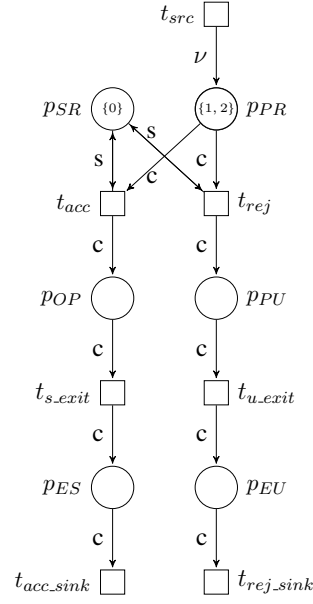


Figure 2.6: Recall the restricted ν -net modeling an unbounded client-server system

sitions t_{acc} , t_{exit} , $t_{acc.sink}$, represent the accept, exit and the sink transition respectively. The firing of transition t_{src} acts as the source, where an unbounded number of vehicle requests can be spawned. In the unsuccessful scenario, the transition t_{rej} is fired when the server rejects the request, which brings the vehicle to *parking_unavailable* state. This is a type of single server multiple client system, where the clients are unbounded. The state diagrams for the server and client are given in Fig. 2.12 and Fig. 2.13, respectively. The combined interactions between clients and the server are modeled as a Petri net in Fig. 2.6.

Conclusion

We make use of the above formal models for concurrency in the rest of this thesis. In order to specify their properties for verifying using bounded model checking, we need to explore how to represent the properties in a manner that is correct as well as suitably written into formulas that can be fed to SAT/SMT solvers for verification. We also introduced the running example APS, which is used in the rest of this thesis.

Chapter 3

Logics for Unbounded Concurrency

Reactive systems are a family of systems where there is continuous interaction between the system and its environment. Autonomous vehicles that steer according to traffic and obstacles, operating systems that handle scheduling of processes based on resource utilization, robots that navigate their terrain using various inputs are examples of reactive systems. A reactive program such as an autonomous navigation system can be viewed as an abstract function from an input domain to an output domain whose behaviour consists of a transformation from initial states to final states. Typically, they do not terminate. In the context of verification of infinite state reactive programs, in order to specify the properties of programs, the properties need to be expressed in the form of a logic formula. Owing to their non-terminating nature, we need a mechanism for talking about the way the system evolves along potentially infinite computations. In this chapter, we discuss the various logics used in the rest of the thesis.

3.1 Introduction to Linear Temporal Logic

Temporal logic has become a well-established formalism for writing specifications. Many varieties of temporal logic have been defined in the past, we focus on Linear Temporal Logic (LTL) [108, 129]. There is a close connection between models of LTL formulas and languages of infinite words—the models of an LTL formula constitute an ω -regular language over an appropriate alphabet. As a result, the satisfiability problem for LTL reduces to checking for emptiness of ω -regular languages [130]. Later, in [129], the connection between LTL and ω -regular languages has been extended to model checking. Unlike the satisfiability problem, which asks if a given formula α has a model, the model-checking problem is one of verification: the task

is to verify whether a given finite-state program P satisfies a specification α . This consists of checking that all runs of P constitute models for α . Since finite-state reactive programs can be represented quite naturally as Büchi automata, model checking also reduces to a problem in automata theory. It suffices to show that no run of P is a model for $\neg\alpha$, which is the same as checking that the intersection of the language accepted by P and the language defined by $\neg\alpha$ is empty. We shall explore the model checking algorithm in detail in Chapter. 4. First, we shall see how to express the properties in LTL.

3.1.1 Specifying properties

Linear temporal logic is very useful in specifying system properties. We consider atomic properties $p_1, p_2, \dots$ of the system (or program) to be verified. Suppose p_1 and p_1 are atomic propositions in the context of a reactive system. We can formulate specifications of this (generic) system in LTL as follows:

Guarantee:	Sometime p_1 becomes true
Safety:	Always p_1 is true
Periodicity :	Initially, p_1 is true and p_1 is true precisely at every third moment
Obligation :	Sometime p_1 is true but p_2 is never true.
Recurrence :	Again and again, p_1 is true
Request-Response:	Always when p_1 is true, p_2 will be true sometime later
Until :	Always when p_1 is true, sometime later p_1 will be true again and in the meantime p_2 is always true
Fairness :	If p_1 is true again and again, then so is p_2 .

We reformulate these specifications by using the temporal operators. Their syntax and semantics are explained shortly afterwards.

- Gp for *always (from now onwards) p is true*
- Fp for *eventually (sometime, including present) p is true*
- Xp for *p is true next time*
- p_1Up_2 for *p_1 is true until eventually p_2 is true*

Now, the above specifications are rewritten using LTL as follows:

Fp_1	Sometime p_1 becomes true
Gp_1	Always p_1 is true. (See Figure 3.1).
$(p_1 \wedge X\neg p_1 \wedge XXX\neg p_1 \wedge G(p_1 \iff XXXp_1))$	Initially, p_1 is true and p_1 is true precisely at every third moment.
$Fp_1 \wedge \neg Fp_2$	Sometime p_1 is true but p_2 is never true. (See Figure 3.2).
GFp_1	Again and again, p_1 is true
$G(Fp_1 \rightarrow XFp_2)$	Always when p_1 is true, p_2 will be true sometime later.
$G(p_1 \rightarrow X(p_2Up_2))$	Always when p_1 is true, sometime later p_1 will be true again and in the meantime p_2 is always true.
$GFp_1 \rightarrow GFp_2$	If p_1 is true again and again, then so is p_2 . (See Figure 3.3).

The LTL formulas can be translated into Büchi automata.

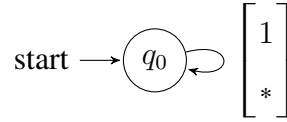


Figure 3.1: Büchi automaton for Gp_1

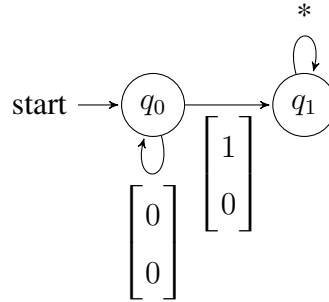


Figure 3.2: Büchi automaton for Fp_1

Next, we discuss the syntax of LTL.

3.1.2 Syntax

The LTL formulas over atomic boolean propositions $V = \{p_1, \dots, p_n\}$ are inductively defined as follows:

- p_i is a LTL formula.

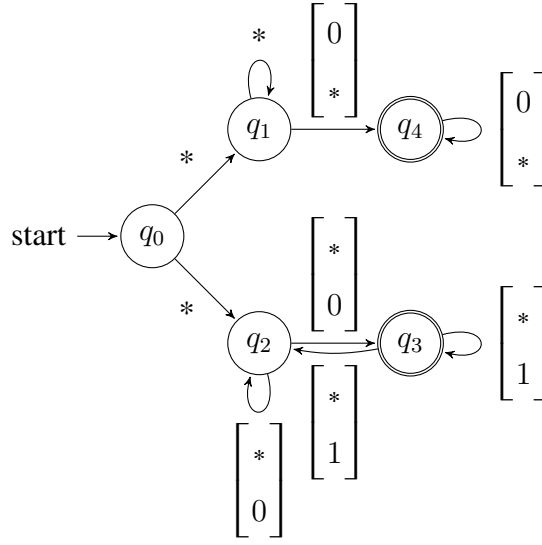


Figure 3.3: Büchi automaton for $GFp_1 \rightarrow GFp_2$

- If ϕ, ψ are LTL formulas, then so are $\neg\phi, \phi \vee \psi, \phi \wedge \psi, \phi \rightarrow \psi$
- If ϕ, ψ are LTL formulas, $X\psi, F\phi, G\phi, \phi U \psi$ where X is the next operator, F denotes sometime, and G denotes globally.

The system contains a set of states S , a set of initial states $I \subseteq S$ and transition relation $T \subseteq S \times S$. The interpretation of propositional variables may change over time but is determined by the current state of the system. This is denoted by the labelling function $L : S \rightarrow P(V)$, where S is the set of states. A propositional variable p is true in a system state s iff $p \in L(s)$.

Definition 3.1.1. A Kripke structure M is given by a tuple $M = (S, I, T, L)$ where S is the set of states, $I \subseteq S$ is the set of initial states, $T \subseteq S \times S$ is the transition relation and $L : S \mapsto P(A)$ is the labeling function, where A is the set of atomic propositions, and $P(A)$ denotes the powerset over A . We use labels over states to describe the atomic propositions that hold in that state. For instance, for a state $s \in S$ the set $L(s)$ contains the atomic propositions that hold in s .

We fix one Kripke structure $K = (S, I, T, L)$ over the variables V . Given the state s is a vector containing all variables V in the current state and s' is a vector of their primed copies in the successor state, the transition relation is written as $T(s, s')$ which is interpreted as $T(s, s')$ holds iff there is a transition from $s \rightarrow s'$.

3.1.3 Semantics

The semantics of LTL are defined along paths of the model. A path π is an infinite sequence of states $\pi = (s_0, s_1, s_2, \dots)$ where $s_i \rightarrow s_{i+1}$. A path π is initialized if its first state $\pi(0) = s_0$ is an initial state. An LTL formula f holds along a path π written as $\pi \models f$ iff

- $\pi \models p$ iff $p \in L(\pi(0))$
- Similarly $\pi \models \neg p$ iff $p \notin L(\pi(0))$
- $\pi \models g \vee h$ iff $\pi \models g$ or $\pi \models h$
- $\pi \models g \wedge h$ iff $\pi \models g$ and $\pi \models h$
- $\pi \models Fg$ iff $\exists j \in \mathbb{N} : \pi^j \models g$
- $\pi \models Gg$ iff $\forall j \in \mathbb{N} : \pi^j \models g$
- $\pi \models Xg$ iff $\pi^1 \models g$

In this context, the model checking problem is to determine whether Kripke structure $K \models f$ holds. A formula f has a witness in K iff there is an initialized path π where $\pi \models f$. Hence, $K \models f$ iff $\neg f$ doesn't have a witness in K . Therefore, we can reduce the model checking problem to the search for witnesses using negation and translation into Negation Normal Form, where the negations are pushed to the variables as follows:

- $\neg(g \wedge h) \equiv (\neg g) \vee (\neg h)$
- $\neg Fg \equiv G\neg g$
- $\neg Gg \equiv F\neg g$
- $\neg X \equiv X\neg g$

3.1.4 Bounded Semantics of LTL

We recall the bounded semantics of LTL given in [23, 24]. It is observed that some infinite paths can be represented by a finite prefix with a loop. An infinite path π is a (k, l) lasso, iff $\pi(k+1+j) = \pi(l+j)$, $\forall j \in \mathbb{N}$. This is represented in Figure 3.4. The path can be represented

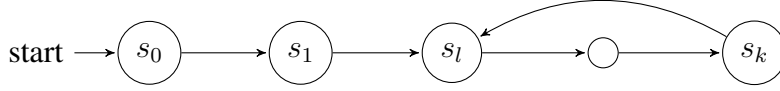


Figure 3.4: (k, l) lasso with $l=2, k=4$

as $\pi = \pi_{stem} \cdot \pi_{loop}^\omega$. In case of the finite system K , the search for witness can be restricted to lassos.

We can rewrite the semantics given above as follows:

- $\pi^i \models p$ iff $p \in L(\pi(i))$
- Similarly $\pi^i \models \neg p$ iff $p \notin L(\pi(i))$
- $\pi^i \models g \vee h$ iff $\pi^i \models g$ or $\pi^i \models h$
- $\pi^i \models g \wedge h$ iff $\pi^i \models g$ and $\pi^i \models h$
- $\pi^i \models Fg$ iff $\exists j \in \mathbb{N} : \pi^{i+j} \models g$
- $\pi^i \models Gg$ iff $\forall j \in \mathbb{N} : \pi^{i+j} \models g$
- $\pi^i \models Xg$ iff $\pi^{i+1} \models g$

In order to obtain bounded semantics, we need to observe only the first $k + 1$ states, and let $i \in 0 \dots k$. If π is (k, l) lasso, then $\pi(k + 1 + j) = \pi(l + j), \forall j \in \mathbb{N}$. Hence, we have the following:

- $\pi^i \models Fg$ iff $\exists j \in \min(i, l), \dots, k : \pi^j \models g$
- $\pi^i \models Gg$ iff $\forall j \in \min(i, l), \dots, k : \pi^{i+j} \models g$
- $\pi^i \models Xg$ iff $\pi^{i+1} \models g$ if $i < k$
- $\pi^i \models Xg$ iff $\pi^l \models g$ if $i = k$

Suppose our property to be tested is a safety property Gp , we need a witness for $F\neg p$. If p doesn't hold in some initial state s in system K , $k = 0$ is sufficient.

If π is not a (k, l) lasso, for any l , and we do not want to examine suffix beyond bound k , then we cannot draw conclusions about $\pi^k \models Xg$ nor about $\pi^k \models Gg$ for $i < k$.

3.1.5 Encoding LTL Semantics

The bounded semantics of LTL are used to encode the LTL formulas which are fed to SMT solvers [23, 24]. The encoding can be implemented as a recursive procedure that takes an LTL formula f , a fixed bound k , the loop start l , and the position i as parameters, where $l, i \in [0, \dots, k]$. The output of this procedure is ${}_l[f]_k^i$. In case of a (k, l) loop in the model.

$$\begin{aligned}
{}_l[p]_k^i &\equiv p_i & {}_l[\neg p]_k^i &\equiv \neg p_i \\
{}_l[g \vee h]_k^i &\equiv {}_l[g]_k^i \vee {}_l[h]_k^i & {}_l[g \wedge h]_k^i &\equiv {}_l[g]_k^i \wedge {}_l[h]_k^i \\
{}_l[Fg]_k^i &\equiv \bigvee_{\min}^k(l, i) \quad {}_l[g]_k^i & {}_l[Gg]_k^i &\equiv \bigwedge_{\min}^k(l, i) \quad {}_l[g]_k^i \\
{}_l[Xg]_k^i &\equiv {}_l[g]_k^j \text{ with } j = i + 1 \text{ if } i < k \text{ else } j = l
\end{aligned}$$

In the absence of loops, we use the following encoding:

$$[Fg]_k^i \equiv \bigvee_{j=i}^k [g]_k^j \quad [Gg]_k^i \equiv \perp \quad [Xg]_k^i \equiv [g]_k^{i+1} \text{ if } i < k \text{ else } \perp \text{ if } j = k$$

The full encoding is given as: $[f]_k \equiv [f]_k^0 \vee \bigvee_{l=0}^k \wedge_l [f]_k^0$.

Using the encoding, we can find a witness for a particular bound k . In case the formula is satisfiable, we are certain to have found a witness. If the formula is unsatisfiable, we can increment k and look for a witness of greater length. This leads to the natural question, of when we should stop increasing k , when no witness has been found yet. We discuss the algorithm in greater detail in the later chapters.

3.2 LTL with Linear Integer Arithmetic and Counting

LTL is a natural choice to describe the temporal properties of such systems. We introduce the logic LTL_{LIA} as an extension of LTL with a few differences, namely linear integer arithmetic operators and counting at the propositional level, which we describe in this section. We are interested in LTL_{LIA} as it is easy to specify temporal properties as well as invariants in it. With respect to Fig. 3.5, consider the invariant, where there is exactly one token in either place p_{SR} or p_{SB} or p_{RR} . This is easily expressed in LTL_{LIA} as $G(\#p_{SR} + \#p_{SB} + \#p_{RR} = 1)$.

In the case of LTL , atomic formulas are propositional constants which have no further structure. In LTL_{LIA} , there are two types of atomic formulas: (1) P_s - describing basic server properties, which are propositional constants and can be used to describe the transitions of the net (2) $\beta \in \Delta$, the set of client formulas, which is formally given by:

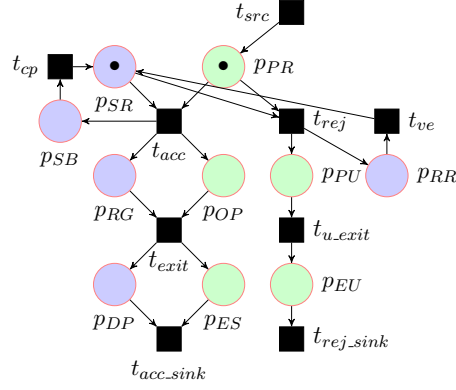


Figure 3.5: A Petri net depicting an unbounded client-server system with invariants

$$\alpha, \hat{\alpha} ::= \#p \mid \mathbf{c} \mid (\mathbf{c} * \#p) \mid (\#p * \mathbf{c}) \mid (\alpha + \hat{\alpha}) \mid (\alpha - \hat{\alpha})$$

$$\beta ::= (\alpha < \hat{\alpha}) \mid (\alpha > \hat{\alpha}) \mid (\alpha \leq \hat{\alpha}) \mid (\alpha \geq \hat{\alpha}) \mid (\alpha = \hat{\alpha})$$

where $p \in P_c$, the set of client propositions and $\mathbf{c} \in \mathbb{Z}^+$.

The set of server formulas Ψ are defined as follows:

$$\psi ::= q \in P_s \mid \beta \in \Delta \mid \neg\psi \mid \psi \vee \psi' \mid \psi \wedge \psi' \mid X\psi \mid F\psi \mid G\psi \mid \psi U \psi'$$

where $\psi, \psi' \in \Psi$. Modalities X , F , G and U are the usual modal operators: Next, Eventually, Globally and Until, respectively.

Unbounded Semantics of LTL_{LIA}

The logic LTL_{LIA} is interpreted over model sequences. Let CN be a countable set of client names that can be assigned to the processes in the system. Each client otherwise known as agent, is introduced into the system dynamically and may eventually exit the system. The clients which are currently in the system and have not yet exited are referred to as live clients (live agents). This notion becomes particularly noticeable while introducing the identifiable clients (agents) (cf. Sec. 3.3). Formally, a model is a sequence $\varrho = m_0, m_1, \dots$, where for all $i \geq 0$ m_i is a triple (ν_i, V_i, ξ_i) such that:

1. $\nu_i \subset_{fin} P_s$, gives the local properties of the server at instant i .
2. $V_i \subset_{fin} CN$ gives the clients alive at instant i . Further, for all $i \geq 0$, $V_{i+1} \subseteq V_i$ or $V_i \subseteq V_{i+1}$ which indicates that the clients can enter or exit the system dynamically.

3. $\xi_i : V_i \rightarrow 2^{P_c}$ gives the properties satisfied by each live agent at the i th instant.

The evaluation of the set of terms α at time instance i is derived from m_i and given by the following function: $\|\alpha\|_i : \alpha \times \mathbb{N} \rightarrow \mathbb{N}$. This is defined inductively as follows:

Base case: The term $\#p$ evaluates to a non-negative integer, $\|\#p\|_i$ denoting the number of clients satisfying property p at instance i where $\|\#p\|_i = \{a \in V_i \mid p \in \xi_i(a)\}$. The evaluation of c at instance i , $\|c\|_i = c$.

Inductive case: The terms $(c\#p)$ (and $(\#pc)$) evaluates to $c\|\#p\|_i$ (and $\|\#p\|_i c$ respectively) the multiplication of an integer denoted by $\#p$ with a constant c . The terms $(\alpha + \hat{\alpha})$ and $(\alpha - \hat{\alpha})$ evaluate to addition and subtraction of integers $\|\alpha\|_i, \|\hat{\alpha}\|_i$.

The truth of a formula at an instant in the model is given by the $\models$ relation defined by induction over the structure of ψ as follows:

1. $\varrho, i \models (\alpha < \hat{\alpha})$ iff $\|\alpha\|_i < \|\hat{\alpha}\|_i$.
2. $\varrho, i \models (\alpha > \hat{\alpha})$ iff $\|\alpha\|_i > \|\hat{\alpha}\|_i$.
3. $\varrho, i \models (\alpha \leq \hat{\alpha})$ iff $\|\alpha\|_i \leq \|\hat{\alpha}\|_i$.
4. $\varrho, i \models (\alpha \geq \hat{\alpha})$ iff $\|\alpha\|_i \geq \|\hat{\alpha}\|_i$.
5. $\varrho, i \models (\alpha = \hat{\alpha})$ iff $\|\alpha\|_i = \|\hat{\alpha}\|_i$.
6. $\varrho, i \models q$ iff $q \in \nu_i$. Note that q 's denote atomic local server propositions. Therefore, a q holds in the model ϱ at instance i if q is in the set ν_i .
7. $\varrho, i \models \neg\psi$ iff $\varrho, i \not\models \psi$.
8. $\varrho, i \models \psi \vee \psi'$ iff $\varrho, i \models \psi$ or $\varrho, i \models \psi'$.
9. $\varrho, i \models \psi \wedge \psi'$ iff $\varrho, i \models \psi$ and $\varrho, i \models \psi'$.
10. $\varrho, i \models X\psi$ iff $\varrho, i + 1 \models \psi$.
11. $\varrho, i \models F\psi$ iff $\exists j \geq i, \varrho, j \models \psi$.
12. $\varrho, i \models G\psi$ iff $\forall j \geq i, \varrho, j \models \psi$.
13. $\varrho, i \models \psi U \psi'$ iff $\exists j \geq i, \varrho, j \models \psi'$ and for all $i \leq j' < j : \varrho, j' \models \psi$.

3.2.1 Two dimensional Bounded Semantics of LTL_{LIA}

Bounded semantics of LTL was described in Sec. 3.1.4. Here we give the bounded semantics of LTL_{LIA} in order to arrive at the SMT encoding in Sec. 5.2 which is necessary for BMC. We use $\models^{\langle \lambda, \kappa \rangle}$ as an restriction of $\models$ over bounded runs of length λ . The bound k is divided into two parts: $k = \kappa + \lambda$, where κ is a bound on the number of clients and λ is a bound on the execution steps of the net. So we call this bounded semantics as two dimensional bounded semantics of LTL_{LIA} . Since the runs are bounded, there are at most κ agents in the system, namely $CN = \{0, \dots, \kappa - 1\}$.

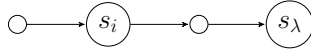


Figure 3.6: Bounded loop-free path of length λ

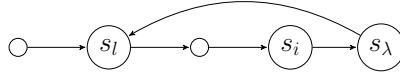


Figure 3.7: Bounded path with (λ, l) - loop

First, we describe the bounded semantics without loop where $0 \leq i \leq \lambda$:

1. $\varrho, i \models^{\langle \lambda, \kappa \rangle} (\alpha < \hat{\alpha})$ iff $\|\alpha\|_i < \|\hat{\alpha}\|_i$.
2. $\varrho, i \models^{\langle \lambda, \kappa \rangle} (\alpha > \hat{\alpha})$ iff $\|\alpha\|_i > \|\hat{\alpha}\|_i$.
3. $\varrho, i \models^{\langle \lambda, \kappa \rangle} (\alpha \leq \hat{\alpha})$ iff $\|\alpha\|_i \leq \|\hat{\alpha}\|_i$.
4. $\varrho, i \models^{\langle \lambda, \kappa \rangle} (\alpha \geq \hat{\alpha})$ iff $\|\alpha\|_i \geq \|\hat{\alpha}\|_i$.
5. $\varrho, i \models^{\langle \lambda, \kappa \rangle} (\alpha = \hat{\alpha})$ iff $\|\alpha\|_i = \|\hat{\alpha}\|_i$.
6. $\varrho, i \models^{\langle \lambda, \kappa \rangle} q$ iff $q \in \nu_i$.
7. $\varrho, i \models^{\langle \lambda, \kappa \rangle} \neg\psi$ iff $\varrho, i \not\models^{\langle \lambda, \kappa \rangle} \psi$.
8. $\varrho, i \models^{\langle \lambda, \kappa \rangle} \psi \vee \psi'$ iff $\varrho, i \models^{\langle \lambda, \kappa \rangle} \psi$ or $\varrho, i \models^{\langle \lambda, \kappa \rangle} \psi'$.
9. $\varrho, i \models^{\langle \lambda, \kappa \rangle} \psi \wedge \psi'$ iff $\varrho, i \models^{\langle \lambda, \kappa \rangle} \psi$ and $\varrho, i \models^{\langle \lambda, \kappa \rangle} \psi'$.
10. $\varrho, i \models^{\langle \lambda, \kappa \rangle} X\psi$ iff $\begin{cases} \varrho, i + 1 \models^{\langle \lambda, \kappa \rangle} \psi & \text{if } (i < \lambda) \\ \varrho, i \models^{\langle \lambda, \kappa \rangle} X\psi & \text{otherwise} \end{cases}$

When instance i is less than the bound λ , the formula ψ is evaluated at instance $i + 1$ else, the formula is unsatisfiable.

$$11. \varrho, i \models^{\langle \lambda, \kappa \rangle} F\psi \text{ iff } \exists j : i \leq j \leq \lambda, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi.$$

This formula is satisfiable if there is some instance j such that $i \leq j \leq \lambda$ at which the property ψ holds.

$$12. \varrho, i \not\models^{\langle \lambda, \kappa \rangle} G\psi.$$

In the absence of a loop in the bounded run, the above formula is always unsatisfiable.

$$13. \varrho, i \models^{\langle \lambda, \kappa \rangle} \psi U \psi' \text{ iff } \exists j : i \leq j \leq \lambda, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi' \text{ and for all } j' : i \leq j' < j : \varrho, j' \models^{\langle \lambda, \kappa \rangle} \psi.$$

This formula is satisfied when formula ψ' is satisfied at some instance j and for all instances less than j , formula ψ is satisfied.

Second, we describe the bounded semantics with (λ, l) -loop [23] where $0 \leq i \leq \lambda$ and $0 \leq l \leq \lambda$ as in Fig. 3.7. Semantics and explanations are given only where they differ from the corresponding case without loop:

$$10. \varrho, i \models^{\langle \lambda, \kappa \rangle} X\psi \text{ iff } \begin{cases} \varrho, i + 1 \models^{\langle \lambda, \kappa \rangle} \psi & \text{if } (i < \lambda) \\ \varrho, l \models^{\langle \lambda, \kappa \rangle} \psi & \text{otherwise} \end{cases}$$

When instance i is less than the bound λ , the formula ψ is evaluated at instance $i + 1$ else, ψ is evaluated at instance l , which is the next instance of λ .

$$11. \varrho, i \models^{\langle \lambda, \kappa \rangle} F\psi \text{ iff } \exists j : \min(l, i) \leq j \leq \lambda, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi.$$

This formula is satisfiable if there is some instance j such that $\min(l, i) \leq j \leq \lambda$, where the formula ψ is satisfied.

$$12. \varrho, i \models^{\langle \lambda, \kappa \rangle} G\psi \text{ iff } \forall j : \min(l, i) \leq j \leq \lambda, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi.$$

This formula is satisfiable if for all instances j such that $\min(l, i) \leq j \leq \lambda$, the formula ψ is satisfied in each instance.

$$13. \varrho, i \models^{\langle \lambda, \kappa \rangle} \psi U \psi' \text{ iff } \left\{ \begin{array}{l} \exists j : i \leq j \leq \lambda, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi' \text{ and} \quad \text{if } (i \leq l) \\ \forall j' : i \leq j' < j : \varrho, j' \models^{\langle \lambda, \kappa \rangle} \psi \\ \exists j : l \leq j \leq \lambda, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi' \text{ and} \quad \text{if } (i > l) \\ \forall j' : l \leq j' < j : \varrho, j' \models^{\langle \lambda, \kappa \rangle} \psi \\ \text{or} \\ \exists j : l \leq j < i, \varrho, j \models^{\langle \lambda, \kappa \rangle} \psi' \text{ and} \\ \forall j' : l \leq j' < j : \varrho, j' \models^{\langle \lambda, \kappa \rangle} \psi \end{array} \right.$$

Consider the two cases:

- If $(i \leq l)$, the current instance i is less than or equal to the loop instance l , this formula is satisfied when formula ψ' is satisfied at some instance j such that $i \leq j \leq \lambda$ and for all instances between i and j , formula ψ is satisfied.
- If $(i > l)$, the formula may be satisfied in either of the two intervals between i to λ or between l to i . Hence, the formula is satisfied if either of the following are satisfied: formula ψ' is satisfied at some instance j such that $i \leq j \leq \lambda$ and for all instances less than j , formula ψ is satisfied or, formula ψ' is satisfied at some instance j such that $l \leq j < i$ and for all instances between l and j , formula ψ is satisfied.

3.3 First Order Logic with Monodic restriction

The monodic logic FOTL_1 is a syntactic subclass of Monadic First Order Temporal Logic (MFOTL) [74] with two restrictions : it is monadic and all formulas have quantifier rank 1. We choose this logic FOTL_1 as it is natural for expressing properties of unbounded client-server systems where the number of clients is both unbounded and dynamic. A *monodic* formula is a well-formed formula with at most one free variable in the scope of a temporal modality. In this section, we describe the syntax and semantics of FOTL_1 with suitable examples. To give a flavour of FOTL_1 and its expressibility, we enumerate some properties of APS that are not easily expressible in Linear Temporal Logic (LTL). Let P_s be the set of atomic propositions of the server and P_c be the set of client predicates. In the APS running example, they are defined

as follows:

$$\begin{aligned}
P_c &= \{parking_requested(PR), occupy_parking_lot(OP), \\
&\quad parking_unavailable(PU), exit_successfully(ES), \\
&\quad exited_unsuccessfully(EU)\} \\
P_s &= \{server_ready(SR)\}
\end{aligned}$$

1. When a vehicle requests a parking space, it is always the case that for every vehicle, it eventually exits the system, either successfully after being granted a parking space, or unsuccessfully, when its request is denied.

$$\begin{aligned}
\psi_1 &= \mathbf{G}_s(\forall x) \left(parking_requested(x) \Rightarrow \right. \\
&\quad \left. \mathbf{F}_c(exit_successfully(x) \vee exit_unsuccessfully(x)) \right)
\end{aligned}$$

2. It is always the case that if the client occupies a parking lot, it will eventually exit the parking lot.

$$\psi_2 = \mathbf{G}_s(\forall x) \left(occupy_parking_lot(x) \Rightarrow \mathbf{F}_c(exit_successfully(x)) \right)$$

3. There may be clients whose requests are rejected.

$$\psi_3 = \mathbf{G}_s(\exists x) \left(parking_requested(x) \wedge \mathbf{F}_c(exit_unsuccessfully(x)) \right)$$

4. There may be clients who have requested for parking and who wait in the parking unavailable state until they are able to exit the system.

$$\begin{aligned}
\psi_4 &= \mathbf{G}_s(\exists x) \left(parking_requested(x) \wedge \right. \\
&\quad \left. \mathbf{F}_c(parking_unavailable(x) \mathbf{U}_c exit_unsuccessfully(x)) \right)
\end{aligned}$$

It can be observed that there are no free variables in the scope of $\mathbf{G}_s$ and exactly one free variable in the scope of the client modalities. It is also possible to construct FOTL₁ specifications with propositions from P_s and server transitions. The ease of expressibility of the client and server behaviour is the key motivation behind the logic FOTL₁ which is formally described in the subsequent section.

3.3.1 Syntax of FOTL₁

We describe the syntax of logic FOTL₁ below. The set of *client formulae*, Δ , is the *boolean and temporal modal closure* of atomic client formulae P_c :

$$\alpha, \beta \in \Delta ::= p(x), p \in P_c \mid \neg\alpha \mid \alpha \vee \beta \mid \alpha \wedge \beta \mid \mathbf{X}_c\alpha \mid \mathbf{F}_c\alpha \mid \mathbf{G}_c\alpha \mid \alpha \mathbf{U}_c\beta$$

The set of *server formulae*, Ψ , is the *boolean and temporal modal closure* of $\Phi = \{(\exists x)\alpha, (\forall x)\alpha \mid \alpha \in \Delta\}$ and atomic server formulae P_s :

$$\Psi ::= q \in P_s \mid \neg\psi \mid \phi \in \Phi \mid \psi_1 \vee \psi_2 \mid \psi_1 \wedge \psi_2 \mid \mathbf{X}_s\psi \mid \mathbf{F}_s\psi \mid \mathbf{G}_s\psi \mid \psi_1 \mathbf{U}_s\psi_2$$

where $\psi, \psi_1, \psi_2 \in \Psi$. It can be observed that in FOTL₁, the quantifier depth is at most one and quantifier alternation is not allowed. The syntax allows us to specify only monodic formulas. Every variable in the server formulas is bounded; FOTL₁ allows for only client formulas to be quantified and we do not have quantifiers over server formulae.

3.3.2 Two dimensional Bounded Semantics of FOTL₁

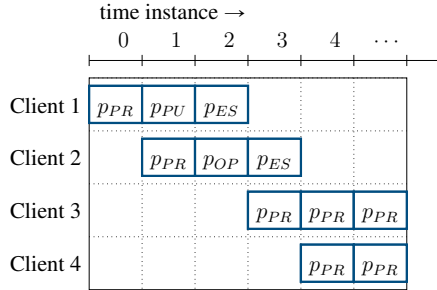


Figure 3.8: Snapshot of the running example (*APS*) depicting live windows

We consider the *unbounded client-server systems* where all clients are of the same type and they may enter and exit the system dynamically. At any point in time, the number of clients is *bounded*, but their cardinality is *unknown* and *dynamic*. We refer to the clients that are present in the system at any point in time as *live agents* (clients). The *live window* of a particular client begins when it enters the system and ends when the client exits the system. Hence, if there are several *live agents*, their *live windows* would overlap each other. This is interesting as it allows us to reason about the *live clients* which satisfy particular properties simultaneously. We illustrate these concepts with respect to the running example *APS* before formally defining them.

Example 3.3.1. Fig. 3.8 depicts the snapshot of the system with 4 distinguishable clients, with overlapping live windows, with the bound 5. The x axis denotes the time instance. The clients are along the y axis. While the system is unbounded, there are a finite number of clients at an instant, as shown in this figure. Each row shows the local state of that client. For each instance, the local state of the client is in the cell i.e., client 1 is at state p_{PR} at instance 0. For client 1, the left boundary, when it enters the system is at instance 0 and its right boundary is at instance 2, when it exits the system. This corresponds to the client requesting parking and getting rejected. There may be multiple clients in the same local state (client 3 and client 4 are in state p_{PR} at instance 4). There may be clients which are live at the bound 5 and have not exited the system, such as clients 3 and 4. This is an interesting case, where the bound (in the snapshot) is equal to the current right boundary for the client.

At the outset, we define the following objects:

- Let CN be a countable set of client names. The client enters the system and gets a unique identifier (name) from CN . When the client exits the system, the identifier gets discarded and is never reused.
- Let $CS = (Q, \Sigma, \delta, I, F)$ be a finite state machine describing the behaviour of a client. Let $L : Q \rightarrow 2^{P_c}$ be the definition (labelling function) of each client state $q \in Q$ in terms of a subset of properties P_c true in that state. The states in F are sink states with no outgoing arcs.

Example: In the running example (APS), the clients have exactly one initial state $I = \text{parking_requested} (PR)$ and two final states namely,

$$F = \{\text{exit_successfully} (ES), \text{exit_unsuccessfully} (EU)\}.$$

- Let $\mathfrak{J} : CN \times \mathbb{N}_0 \rightarrow Q$ be a partial mapping describing the local state of each client $a \in CN$ at an instance $i \in \mathbb{N}_0$. For instance, $\mathfrak{J}(a, i) \in q$, means that the local state of each client a at instance i is state q , where $q \in Q$.

Formally, a model is a triple $M = (\nu, V, \xi)$ where

1. ν gives the local behaviour of the server as follows:

$$\nu = \nu_0 \nu_1 \nu_2 \dots, \text{ where for all } 0 \leq i, \nu_i \subseteq P_s,$$

2. V gives the set of live agents (clients) at each instance.

$V = V_0V_1V_2\dots$, where for all $0 \leq i$, V_i is a finite subset of CN , gives the set of live agents at the i th instance.

For every $0 \leq i$, V_i and V_{i+1} satisfy the following properties:

- (a) if $V_i \subseteq V_{i+1}$ then for every $a \in V_{i+1} - V_i$ such that $\mathfrak{Z}(a, i+1) \in I$.
- (b) if $V_{i+1} \subseteq V_i$ then for every $a \in V_i - V_{i+1}$ such that $\mathfrak{Z}(a, i) \in F$.

V may satisfy the following interesting property. For any $a \in CN$ and $i \in \mathbb{N}_0$ if $a \in V_i$ and if there exists $j > i$ such that $a \notin V_j$ then we may define the left and right boundaries of the live window for a , denoted by $left(a)$ and $right(a)$ where $left(a) \leq i \leq right(a)$. If no such j exists then there is no right boundary for the live window of that client.

3. $\xi = \xi_0\xi_1\xi_2\dots$, where for all $0 \leq i$, $\xi_i : V_i \rightarrow 2^{P_c}$ gives the properties satisfied by each live agent at i th instance, in other words, the corresponding states of live agents. In terms of $\mathfrak{Z}$, $L(\mathfrak{Z}(a, i)) = \xi_i(a)$. Alternatively, ξ_i can be given as $\xi_i : V_i \times P_c \rightarrow \{\top, \perp\}$.

3.4 SMT Encoding for the Logic FOTL₁

In this section, we describe the SMT encoding for FOTL₁ which will enable us to implement a bounded model checker tool for restricted ν -nets using FOTL₁ specifications. Let $[\mathcal{M}]_{\langle \lambda, \kappa \rangle}$ be the SMT encoding of λ -bounded runs of the net $\mathcal{M}$ containing at most κ agents. The SMT encoding $[\mathcal{M}]_{\langle \lambda, \kappa \rangle}$ can be given based on the definition of the restricted ν -net. Let ψ be a property written in the logic language FOTL₁ which is being model-checked in bounded runs of $\mathcal{M}$. For a given $\langle \lambda, \kappa \rangle$ and i (ψ is asserted at i , $0 \leq i \leq \lambda$), we define two encoding functions $[\psi]_{\langle \lambda, \kappa \rangle}^i$ and ${}_l[\psi]_{\langle \lambda, \kappa \rangle}^i$. The formula $[\psi]_{\langle \lambda, \kappa \rangle}^i$ denotes the SMT encoding of ψ , where the bounded run of length λ and at most κ agents in the system does not contain a loop. The formula ${}_l[\psi]_{\langle \lambda, \kappa \rangle}^i$ denotes the SMT encoding of ψ , where the bounded run of length λ and at most κ agents contains a loop which is asserted at i .

We add the following formulas about the dead agents (agents that are not alive) to the system specification:

- If a client exits the parking lot, then it becomes dead in the next state.

$$\delta_1 = \bigwedge_{0 \leq i \leq \lambda-1} \left(\bigwedge_{0 \leq j \leq \kappa-1} \text{exit_successfully}(j, i) \Rightarrow (\text{dead}(j, i+1)) \right)$$

$$\delta_2 = \bigwedge_{0 \leq i \leq \lambda-1} \left(\bigwedge_{0 \leq j \leq \kappa-1} \text{exit_unsuccessfully}(j, i) \Rightarrow (\text{dead}(j, i+1)) \right)$$

- If a client is dead then it remains dead.

$$\delta_3 = \bigwedge_{0 \leq i \leq \lambda-1} \left(\bigwedge_{0 \leq j \leq \kappa-1} \text{dead}(j) \Rightarrow (\text{dead}(j, i+1)) \right)$$

In case of a different case study, where we consider any other net, instead of APS, we can suitably replace $\text{exit_successfully}(j)$, $\text{exit_unsuccessfully}(j)$ by the relevant termination condition(s).

The SMT encoding is derived from the bounded semantics in section 3.3. The following encodings are extensions of similar mappings defined in [24]. First, we inductively define $[\psi]_{\langle \lambda, \kappa \rangle}^i$ as follows:

1. $[q]_{\langle \lambda, \kappa \rangle}^i \equiv q_i$
2. $[\neg q]_{\langle \lambda, \kappa \rangle}^i \equiv \neg q_i$
3. $[(\exists x)\alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{0 \leq j \leq \kappa-1} [\alpha[j/x]]_{\langle \lambda, \kappa \rangle}^i$
4. $[(\forall x)\alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \bigwedge_{0 \leq j \leq \kappa-1} [\alpha[j/x]]_{\langle \lambda, \kappa \rangle}^i$
5. $[\psi_1 \vee \psi_2]_{\langle \lambda, \kappa \rangle}^i \equiv [\psi_1]_{\langle \lambda, \kappa \rangle}^i \vee [\psi_2]_{\langle \lambda, \kappa \rangle}^i$
6. $[\psi_1 \wedge \psi_2]_{\langle \lambda, \kappa \rangle}^i \equiv [\psi_1]_{\langle \lambda, \kappa \rangle}^i \wedge [\psi_2]_{\langle \lambda, \kappa \rangle}^i$
7. $[\mathbf{X}_s \psi]_{\langle \lambda, \kappa \rangle}^i \equiv \begin{cases} [\psi]_{\langle \lambda, \kappa \rangle}^{i+1} & \text{if } (i < \lambda) \\ \text{False} & \text{otherwise} \end{cases}$
8. $[\mathbf{F}_s \psi]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{i \leq j \leq \lambda} [\psi]_{\langle \lambda, \kappa \rangle}^j$
9. $[\mathbf{G}_s \psi]_{\langle \lambda, \kappa \rangle}^i \equiv \text{False}$
10. $[\psi_1 \mathbf{U}_s \psi_2]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{i \leq j \leq \lambda} ([\psi_2]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{i \leq j' < j} [\psi_1]_{\langle \lambda, \kappa \rangle}^{j'})$
11. $[p(j)]_{\langle \lambda, \kappa \rangle}^i \equiv p(j, i)$
12. $[\neg \alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \neg [\alpha]_{\langle \lambda, \kappa \rangle}^i$
13. $[\alpha \vee \beta]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i \vee [\beta]_{\langle \lambda, \kappa \rangle}^i$
14. $[\alpha \wedge \beta]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i \wedge [\beta]_{\langle \lambda, \kappa \rangle}^i$

$$15. [\mathbf{X}_c \alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \begin{cases} (dead(j, i+1) \Rightarrow False) \wedge (\neg dead(j, i+1) \Rightarrow [\alpha]_{\langle \lambda, \kappa \rangle}^{i+1}) & \text{if } (i < \lambda) \\ False & \text{if } (i = \lambda) \end{cases}$$

If the client is dead at the instance $i+1$, the original formula evaluates to false. If the client is live, the standard bounded semantics apply. Similar semantics are given in equations 16 – 18 based on the client liveness at instance j .

$$16. [\mathbf{F}_c \alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{i \leq j \leq \lambda} \left((dead(j, j) \Rightarrow False) \wedge (\neg dead(j, j) \Rightarrow [\alpha]_{\langle \lambda, \kappa \rangle}^j) \right)$$

$$17. [\mathbf{G}_c \alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \bigwedge_{i \leq j \leq \lambda} \left((dead(j, j) \Rightarrow False) \wedge (\neg dead(j, j) \Rightarrow [\alpha]_{\langle \lambda, \kappa \rangle}^j) \right)$$

$$18. [\alpha \mathbf{U}_c \beta]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{i \leq j \leq \lambda} \left((dead(j, j) \Rightarrow False) \wedge (\neg dead(j, j) \Rightarrow prop_encode_{\alpha \mathbf{U}_c \beta}) \right)$$

$$\text{where } prop_encode_{\alpha \mathbf{U}_c \beta} = \bigwedge_{i \leq j \leq \lambda} [\beta]_{\langle \lambda, \kappa \rangle}^j \wedge \bigvee_{i \leq j' < j} [\alpha]_{\langle \lambda, \kappa \rangle}^{j'}$$

Second, we inductively define ${}_l[\psi]_{\langle \lambda, \kappa \rangle}^i$, for the loop case as follows:

$$1. {}_l[q]_{\langle \lambda, \kappa \rangle}^i \equiv q_i$$

$$2. {}_l[\neg q]_{\langle \lambda, \kappa \rangle}^i \equiv \neg q_i$$

$$3. {}_l[(\exists x) \alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{0 \leq j \leq \kappa-1} {}_l[\alpha[j/x]]_{\langle \lambda, \kappa \rangle}^i$$

$$4. {}_l[(\forall x) \alpha]_{\langle \lambda, \kappa \rangle}^i \equiv \bigwedge_{0 \leq j \leq \kappa-1} {}_l[\alpha[j/x]]_{\langle \lambda, \kappa \rangle}^i$$

$$5. {}_l[\psi_1 \vee \psi_2]_{\langle \lambda, \kappa \rangle}^i \equiv {}_l[\psi_1]_{\langle \lambda, \kappa \rangle}^i \vee {}_l[\psi_2]_{\langle \lambda, \kappa \rangle}^i$$

$$6. {}_l[\psi_1 \wedge \psi_2]_{\langle \lambda, \kappa \rangle}^i \equiv {}_l[\psi_1]_{\langle \lambda, \kappa \rangle}^i \wedge {}_l[\psi_2]_{\langle \lambda, \kappa \rangle}^i$$

$$7. {}_l[\mathbf{X}_s \psi]_{\langle \lambda, \kappa \rangle}^i \equiv \begin{cases} {}_l[\psi]_{\langle \lambda, \kappa \rangle}^{i+1} & \text{if } (i < \lambda) \\ {}_l[\psi]_{\langle \lambda, \kappa \rangle}^l & \text{if } (i = \lambda) \end{cases}$$

$$8. {}_l[\mathbf{F}_s \psi]_{\langle \lambda, \kappa \rangle}^i \equiv \bigvee_{\min(l, i) \leq j \leq \lambda} {}_l[\psi]_{\langle \lambda, \kappa \rangle}^j$$

$$9. {}_l[\mathbf{G}_s \psi]_{\langle \lambda, \kappa \rangle}^i \equiv \bigwedge_{\min(l, i) \leq j \leq \lambda} {}_l[\psi]_{\langle \lambda, \kappa \rangle}^j$$

$$\begin{aligned}
10. \quad {}_l[\psi_1 \mathbf{U}_s \psi_2]_{\langle \lambda, \kappa \rangle}^i &\equiv \begin{cases} \bigvee_{i \leq j \leq \lambda} ({}_l[\psi_2]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{i \leq j' < j} {}_l[\psi_1]_{\langle \lambda, \kappa \rangle}^{j'}) & \text{if } (i \leq l) \\ \left(\left(\bigvee_{i \leq j \leq \lambda} ({}_l[\psi_2]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{i \leq j' < j} {}_l[\psi_1]_{\langle \lambda, \kappa \rangle}^{j'}) \right) \right. \\ \quad \text{or} & \text{if } (i > l) \\ \left. \left(\bigvee_{l \leq j < i} ({}_l[\psi_2]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{l \leq j' < j} {}_l[\psi_1]_{\langle \lambda, \kappa \rangle}^{j'}) \right) \right) \end{cases} \\
11. \quad {}_l[p(j)]_{\langle \lambda, \kappa \rangle}^i &\equiv p(j, i) \\
12. \quad {}_l[\neg \alpha]_{\langle \lambda, \kappa \rangle}^i &\equiv \neg {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^i \\
13. \quad {}_l[\alpha \vee \beta]_{\langle \lambda, \kappa \rangle}^i &\equiv {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^i \vee {}_l[\beta]_{\langle \lambda, \kappa \rangle}^i \\
14. \quad {}_l[\alpha \wedge \beta]_{\langle \lambda, \kappa \rangle}^i &\equiv {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^i \wedge {}_l[\beta]_{\langle \lambda, \kappa \rangle}^i \\
15. \quad {}_l[\mathbf{X}_c \alpha]_{\langle \lambda, \kappa \rangle}^i &\equiv \begin{cases} (dead(j, i+1) \Rightarrow False) \wedge (\neg dead(j, i+1) \Rightarrow {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^{i+1}) & \text{if } (i < \lambda) \\ (dead(j, l) \Rightarrow False) \wedge (\neg dead(j, l) \Rightarrow {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^l) & \text{if } (i = \lambda) \end{cases}
\end{aligned}$$

If the client is dead at the next instance ($i + 1$ or l , respectively), the original formula evaluates to false. If the client is live, the standard bounded semantics apply. Similar semantics are given in equations 16 – 18 based on the client liveness at instance j .

$$\begin{aligned}
16. \quad {}_l[\mathbf{F}_c \alpha]_{\langle \lambda, \kappa \rangle}^i &\equiv \bigvee_{\min(l, i) \leq j \leq \lambda} \left((dead(j, j) \Rightarrow False) \wedge (\neg dead(j, j) \Rightarrow {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^j) \right) \\
17. \quad {}_l[\mathbf{G}_c \alpha]_{\langle \lambda, \kappa \rangle}^i &\equiv \bigwedge_{\min(l, i) \leq j \leq \lambda} \left((dead(j, j) \Rightarrow False) \wedge (\neg dead(j, j) \Rightarrow {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^j) \right) \\
18. \quad {}_l[\alpha \mathbf{U}_c \beta]_{\langle \lambda, \kappa \rangle}^i &\equiv \bigvee_{\min(l, i) \leq j \leq \lambda} \left((dead(j, j) \Rightarrow False) \wedge (\neg dead(j, j) \Rightarrow loop_prop_encode_{\alpha \mathbf{U}_c \beta}) \right) \\
loop_prop_encode_{\alpha \mathbf{U}_c \beta} &= \begin{cases} \bigvee_{i \leq j \leq \lambda} ({}_l[\beta]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{i \leq j' < j} {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^{j'}) & \text{if } (i \leq \lambda) \\ \left(\left(\bigvee_{i \leq j \leq \lambda} ({}_l[\beta]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{i \leq j' < j} {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^{j'}) \right) \right. \\ \quad \text{or} & \text{if } (i > \lambda) \\ \left. \left(\bigvee_{l \leq j < i} ({}_l[\beta]_{\langle \lambda, \kappa \rangle}^j \wedge \bigwedge_{l \leq j' < j} {}_l[\alpha]_{\langle \lambda, \kappa \rangle}^{j'}) \right) \right) \end{cases}
\end{aligned}$$

Conclusion

The above encoding is utilized in the verification tool that utilizes SMT solvers, which is described in Sec. 6.A. Our larger goal is to verify unbounded client server systems. We shall explore the challenges associated with it, the existing literature on verification of these systems and the gap that we aim to fill with our work in the upcoming chapter.

Chapter 4

Bounded Model Checking

Client server systems are a computing paradigm where work loads are distributed by the service providers called server, to the service requesters called clients, or alternatively, the clients request resources from a server. There are variations to this model, where there are single servers and multiple clients, multiple servers and clients and communication among the various entities, passive servers and communicating clients [40]. For instance, stock markets and cryptocurrency exchanges with an unbounded number of investors, multiplayer games where the players are not known apriori can be viewed as client server systems with unbounded agents which we refer to as unbounded client server systems (UCS). The larger goal is to formally verify properties on unbounded client-server systems with concurrency. In the subsequent section, we list the research problems about UCS.

4.1 Verification of UCS: Central questions

The central questions that we ask and address in this work are the following:

1. How do we formally verify UCS?
2. How do we formally model UCS?
3. How do we express properties of client-server systems naturally?
 - (a) Do existing logics suffice?
 - (b) What additional properties are interesting in the context of unbounded client-server systems?

4. Are there formal verification tools for UCS? Can we add to the repertoire?

(a) How are existing tools different, unique? Is another tool necessary?

How do we formally verify UCS? Let us address the first question- formal verification of unbounded client-server systems. In order to formally verify a system, we need to first abstract the system into a formal model and then, describe its properties as logic specifications. UCS can be viewed as a distributed system which permits concurrent servicing of requests. In literature, Petri nets are a well-studied formal model for representing concurrent and distributed systems. The big picture is to employ Petri nets and their extensions for modeling unbounded client-server systems where there is true concurrency in the interaction between the clients and the servers as well as unboundedness in the number of clients. In Sec. 2, we discussed the various suitable formal models - simple PNs, ν -nets and EOSs. To represent the properties of the systems, we employ temporal logics (See Sec. 3).

In order to verify the properties of the unbounded client-server systems (modeled as PNs), we discuss a formal verification technique called model checking. Traditional symbolic model checking employs an exhaustive method to analyse all possible executions of a formal model, but suffers from state space explosion. Bounded Model Checking (BMC) with satisfiability solving, is an alternate approach to address the state space explosion problem. In BMC, a symbolic execution of a transition system and the negation of a property to be verified (translated into a logic formula) is fed to a satisfiability solver. If the formula is satisfiable, a counterexample is found and the property does not hold. If the formula is unsatisfiable up to a pre-defined bound, say k , it is concluded that the property holds upto bound k . It is easy to see that this approach is not complete, however, it is widely adopted in practice for verification of safety properties (where bounds are computable). We explore the applicability of BMC to verify UCS and discuss the algorithm consequently in Sec. 4.2.

How do we express properties of client-server systems naturally? What additional properties are interesting in the context of unbounded client-server systems? Typically, properties such as safety (a client request is never stuck indefinitely without being accepted or rejected), reachability (a client request is eventually serviced) and deadlock-freeness (a server or client action are always taken at each instance) are relevant for study. And these properties are expressible in a temporal logic such as LTL as in Sec. 3.1.

Second, we could additionally make use of interger arithmetic to specify invariants about the system in a counting logic where we have propositions that can count at the atomic level. Assuming arbitrary processes a, b , an interesting invariant would be: The total number of clients undergoing process a and number of clients undergoing process b does not exceed n . This essentially gives an upper bound and can be described in the counting logic $LT L_{LIA}$ described in Sec. 3.2.

Third, in the context of *unbounded* client-server systems, where clients dynamically enter the system and exit them, it is interesting to reason about the set of clients that are live (i.e., currently interacting with the server) at that instant. If we are unable to model live clients, we would need additional mechanisms to keep track of the clients that have exit the system, which can quickly grow in number, since it is an unbounded system. Since these live clients are distinguishable (via client identifiers) we could reason easily about the live clients and rephrase the following properties over sets of live clients: safety (is it always the case that any set of live clients is never stuck indefinitely without being accepted or rejected) reachability (is it always the case that all live client requests are eventually serviced) and deadlock-freeness (is it always the case that a server or live client action are always taken at each instance)? These properties are expressible naturally, in a fragment of first order logic $FOTL_1$ described in Sec. 3.3.

Are there formal verification tools for UCS? Although unbounded client-server systems are abundant in the software ecosystem, to the best of our knowledge, there are no specialized tools to verify unbounded client-server systems. However, [120] are recent works in the context of verifying unbounded systems. In [120], they employ SMT solvers to verify unbounded concurrent data structures where there is unboundedness in the list size as well as the number of threads that may access the list. They abstract the system to a bounded model and perform standard bounded model checking on it. We reckon that by employing a technique such as two-dimensional bounded model checking, proposed in Sec. 4.2, it may be possible to verify the system, while allowing concurrent behaviours (which we discuss later in this section). This may result in the solver converging to a satisfying assignment quicker (which we demonstrate in our experiments).

With regards to verification of unbounded nets, [3, 4] studied the model checking of unbounded PNs with backwards reachability, where a set of safety properties were verified. While modeling unbounded client-server systems as unbounded PNs (cf. Sec. 2.1), we identify the gap

in modeling the behaviour of this system with true concurrency and explore ways to verify properties beyond safety. There is no known work on verifying properties of unbounded PNs over truly concurrent semantics. The existing PN (and unbounded PN) verification tools employ interleaving semantics [3, 4, 6, 9]. We fill this gap while focussing on unbounded client-server systems. It is to be noted that our approach is in contrast to parametric model checking on various software systems [125]. We do not parameterize the dimensions, rather allow them to *unfold* simultaneously.

The novelty of our work is twofold; first, to give a succinct representation of unbounded client-server systems, second, to study true concurrent behaviour and consequently to formally verify unbounded client-server systems in practice over various suitable logics and providing tools for them, while taking advantage of SMT solvers.

4.2 Bounded Model Checking Algorithms

In the subsequent section, we describe BMC algorithms with respect to verification of LTL properties on Petri nets. Notice that in Algorithm. 1 and Algorithm. 2, the temporal property α can be replaced with properties in any logic from Sec. 3 and the model can be replaced by any formal model. Where the unfolded configuration of the Petri net is mentioned (cf. Line. 11 in Algorithm. 1 and cf. Line. 14 in Algorithm. 2), the Petri net may be unfolded using either interleaving or true concurrent semantics.

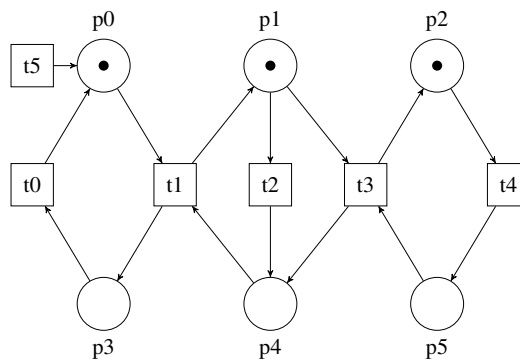


Figure 4.1: An unbounded net

Example 3. Given an unbounded PN Fig. 4.1, one possible sequence of transition firings with interleaving semantics, where exactly one enabled transition is fired at instant is shown in Fig. 4.2. The below are the other possible firing sequences:

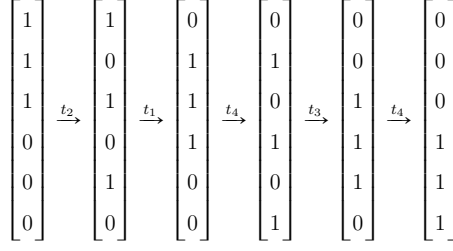


Figure 4.2: Sequence of firings via interleaving semantics

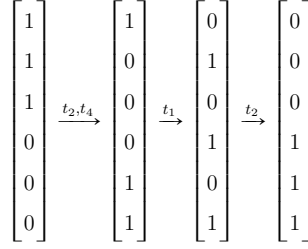


Figure 4.3: Sequence of firings via true concurrent semantics

$$t_2 \rightarrow t_1 \rightarrow t_4 \rightarrow t_3 \rightarrow t_4$$

$$t_4 \rightarrow t_2 \rightarrow t_1 \rightarrow t_2$$

However, with respect to Fig. 4.1 and true concurrent semantics, we obtain the much shorter firing sequence Fig. 4.3.

In the verification tools discussed in Sec. 5 and Sec. 6, we default to using true concurrent semantics, while also allowing interleaving semantics.

4.2.1 Discussion on the Bounded Model Checking (BMC) algorithms

First we disambiguate the notion of bounds. In the BMC approach, the number of steps upto which the system is verified, say k , indirectly places a bound on the number of tokens which are present in the net, thereby restricting the number of clients that are represented in the unbounded client-server system. For instance, when unrolling the system for k steps in the BMC approach, it indirectly places a cap that there can be atmost k newly generated tokens at a place (taking into consideration the weight of the arc and the initial marking). It is to be noted that the bound in BMC is not fixed apriori, which differentiates it from the parametric verification approach where the parameter (the number of clients) is fixed apriori.

The standard BMC approach for Petri nets is given in Algorithm. 1. The inputs for Algorithm. 1 are the marked PN M with initial marking M_0 , temporal property α in NNF and

Algorithm 1 The standard Bounded Model Checking algorithm for Petri nets

Input: Marked PN M , temporal property α in Negation Normal Form (NNF) and bound.**Output:** SAT with counterexample trace or UNSAT**Require:** $bound \geq 0$

```
1:  $index \leftarrow 0$ 
2: InitializeSolver( $M_0, \alpha$ )
3: if SOLVER.CHECK()==SAT then
4:   Display trace and exit
5: end if
6: if SOLVER.CHECK()==UNSAT then
7:    $index \leftarrow 1$ 
8:   while  $index \leq bound$  do
9:      $M_{index} \leftarrow ConstructUnfoldedNet(M, index)$ 
10:    InitializeSolver( $M_{index}, \alpha$ )
11:    if SOLVER.CHECK()==SAT then
12:      Display trace and exit
13:    else if SOLVER.CHECK()==UNSAT then
14:       $index \leftarrow index + 1$ 
15:    end if
16:  end while
17: end if
```

the BMC bound. The variable $index$ keeps track of the steps for which the system is being unfolded. Line 1-4 initializes the solver and checks whether the input formula is satisfied. If it is satisfied, the counterexample trace is displayed and the procedure terminates. In the case that the formula is unsatisfiable then we unfold the net for one more step (i.e., M_{index}) and initialize the solver to check if the formula is satisfied or not. In the case that the formula is not satisfiable, we repeat the process of unfolding the net until we either hit the BMC bound or we obtain a counterexample trace.

Recall that we described the two dimensional bounded semantics of LTL_{LIA} in Sec. 3.2.1 where κ is a bound on the number of clients (agents) and λ is a bound on the execution steps of the net. In Algorithm. 1 the unfolding of the constructed net used only the $index$. In contrast, in Algorithm. 2, the unfolding of the constructed net uses variations of λ and κ .

Algorithm 2 The 2D-BMC algorithm for Petri nets

Input: Marked PN M , temporal property α in Negation Normal Form (NNF) and bound.

Output: SAT with counterexample trace or UNSAT

Require: $bound \geq 0$

```
1:  $index \leftarrow 0$ 
2: InitializeSolver( $M_0, \alpha$ )
3: if SOLVER.CHECK()==SAT then
4:   Display trace and exit
5: end if
6: if SOLVER.CHECK()==UNSAT then
7:    $index \leftarrow 1$ 
8:   while  $index \leq bound$  do
9:     for  $\lambda \leftarrow 1$  to  $bound$  do
10:      for  $\kappa \leftarrow 1$  to  $bound$  do
11:         $M_{index} \leftarrow ConstructUnfoldedNet(M, index, \lambda, \kappa)$ 
12:        InitializeSolver( $M_{index}, \alpha$ )
13:        if SOLVER.CHECK()==SAT then
14:          Display trace and exit
15:        else if SOLVER.CHECK()==UNSAT then
16:           $index \leftarrow index + 1$ 
17:        end if
18:      end for
19:    end for
20:  end while
21: end if
```

We illustrate the usage of the algorithms via an example.

Example 4. Consider the Fig. 4.1 with the following temporal properties.

1. First consider the *LTL* reachability property. $F(p_0 = 0 \& p_1 = 0 \& p_2 = 0 \& p_3 = 1 \& p_4 = 1 \& p_5 = 1)$. It is beneficial to use Algorithm. 1 since it is a reachability property and with 5 steps with interleaved unfolding (or 3 steps in case of concurrent unfolding), the marking can be reached. It is not beneficial to use Algorithm. 2 here, since it will not

converge to a solution faster.

2. Second, consider the $LT L_{LIA}$ property, where

$F((\#x)p_3(x) > p_0(x) \wedge p_4(x) > p_1(x) \wedge p_5(x) > p_2(x))$. This property requires searching by varying both the number of tokens as well as the time instance, hence Algorithm. 2 is suitable here.

4.2.2 Related Work

In symbolic model checking [95] the state transition system is not represented explicitly but as propositional formulas. This prevents what is known as “state-space explosion” problem in program verification, as it slashes the size of representation by one degree of exponent. Originally binary decision diagrams (BDDs) were used to symbolically represent systems, and operations on system states corresponded to BDD operations. In practice, BDDs can handle systems defined using hundreds of variables, but often blow up in space. The technique called Bounded Model Checking (BMC) was an attempt to replace BDDs with SAT (and now SMT) in symbolic model checking. However SAT lacks the possibility to eliminate variables, which is a key operation in BDD-based model checking. In BMC, the solution is to focus on falsification and, at least in a first approximation, drop completeness. It has been found that SAT-based model checking, at least for falsification, scales much better [121].

Autonomous Parking System (APS), which we have considered in our paper, is a kind of single server multiple client system, where one server and an unbounded number of clients collaborate. APS is an instance of an infinite (parameterized) family of finite state systems. Such systems are parameterized because the number is known only at run time. Such a family can usually be seen as one single infinite-state system. Checking reachability in parameterized systems is, in general, undecidable [11].

The verification problem for a family of similar state-transition systems is easy to formulate: Given a family $M = \{M_i\}_{i=1}^{\infty}$ of systems M_i and a temporal formula α , verify that for every i , M_i satisfies α . This is known as the Parameterized Model-checking Problem (PMCP). As mentioned above, PMCP is undecidable, in general. However, for *specific* families the problem may be solvable.

In one of the earliest works on PMCP, Browne et al. [30] consider the problem of verifying a family of token rings, that is, the family of rings of size 2, size 3, size 4 and so on. In order to

verify the entire family, they establish a *bisimulation* relation between a two-process ring and an n -process token ring for any $n \geq 2$. It follows that the two-process ring and the n -process ring satisfy exactly the same temporal formulae.

In [109], Pnueli *et al.* introduce the $(0, 1, \infty)$ -counter abstraction method by which a parameterized system of unbounded size is abstracted into a finite-state system. As each process in the parameterized system is finite-state, the abstract variables are limited counters which count, for each local state s of a process, $\kappa(s)$ —the number of processes which currently are in local state s . The counters are saturated at 2, which means that $\kappa(s) = 2$ whenever 2 or more processes are at state s . The emphasis is on the derivation of an *adequate and sound* set of fairness requirements that enable proofs of liveness properties of the abstract system, from which we can safely conclude a corresponding liveness property of the original parameterized system.

Recently, BMC has been successfully used for verifying safety properties of a special kind of parameterized systems, namely, fault-tolerant distributed algorithms (FTDA). In [83], Konnov *et al.*, demonstrate that reachability properties of FTDAs can be verified by BMC based techniques. In order to ensure completeness, an upper bound on the distance between states is required. So, they show that the diameters of accelerated counter systems of FTDAs have a quadratic upper bound in the number of local transitions.

Note that APS can not be modelled as FTDAs, which are bit more specialized. In FTDAs the firing of a transition depends not only on the current states of a fixed number of processes, but also on the total number of processes [14]. This is quite unlike Petri nets which we have used to model APS.

One of the earliest work on BMC for Petri nets was [69]. In this paper, the author applies BMC to 1-safe Petri nets by translating the bounded reachability problem for 1-safe Petri nets into constraint Boolean circuit satisfiability. The BCSAT constraint Boolean satisfiability checker is used to check whether the generated constraint circuit is satisfiable, thereby solving the bounded reachability problem of 1-safe Petri nets. BMC technique was later extended to verify properties of Timed Automata [12] and Timed Petri nets [96].

BMC using SMT Solvers has been an active area of research [110]. In particular, while applying BMC in the Petri net setting, techniques such as net reductions, and structural reductions [123] are applied. Recently, in [8, 9] the generalized reachability of Petri net is encoded and solved using SMT solvers.

However, BMC for Petri nets while expressing properties using LTL and its extensions while leveraging the power of SMT Solvers remains unexplored. Our work attempts to bridge this gap while preserving the original Petri net structure. In Section 5.2 we provide the SMT encoding of the counting logic LTL_{LIA} which distinguishes our work. In [117] a similar counting linear temporal logic for specifying properties of multi-robot applications is proposed. In contrast, LTL_{LIA} uniquely supports specifying properties of client-server systems.

A recent related work on SMT-based verification of safety properties of parameterized multi-agent system (PMAS) using infinite-state model checking was done in [58]. A similar work is [21], where the authors describe parameterised verification of an unbounded number of data-aware multi-agent systems whose properties are specified using a branching temporal logic. In [86] parameterised verification of multi-agent systems is explored and a model checker MCMAS-P implementing parameterised model checking is also described. In these papers, they perform parameterised model checking, whereas in our case, we do not know the number of agents beforehand. In [91] they consider dynamic multi-agent systems (HD-MAS) where the number of agents evolve which is the closest formal model to ours. Our logic can easily be extended to express properties of multi-agent systems which is part of future work.

Conclusion

Now that we have charted out the boundary of what has been verified with respect to these unbounded client server systems, we shall utilize the logics from Chapter. 3 to specify properties of these systems on Petri nets and build a verification tool in the upcoming chapter.

Chapter 5

LTL Tool over Petri nets

In this chapter, we elaborate on the first contribution of the thesis - bounded model checking of unbounded Petri nets with concurrent semantics. First, we discuss bounded model checking for various properties on Petri nets. Second, we provide the SMT encoding for unbounded Petri nets and their concurrent unfolding. Third, we describe the model checking tool for verifying temporal properties (expressed in LTL with linear integer arithmetic) for unbounded Petri nets and discuss the experimental results.

5.1 Bounded Model Checking for unbounded PNs

The intuition behind BMC and its extension, $2D$ -BMC is to represent a counterexample-trace of bounded length symbolically and check the resulting formula with a SAT/SMT solver. If the formula is satisfiable and thus the path feasible, a satisfying assignment returned by the solver can be translated into a concrete counterexample trace that shows that the property is violated. Otherwise, the bound is increased and the process repeated. Complete extensions to this approach allow to stop this process at one point, with the conclusion that the property cannot be violated, hopefully before the available resources are exhausted. The details of $2D$ -BMC are discussed in detail in Sec. 4.2. In this section, we focus on performing $2D$ -BMC on unbounded Petri nets, taking advantage of true concurrency in the nets and representing the properties of the nets in a temporal logic LTL_{LIA} . Linear Temporal Logic (LTL) [108, 129] is a natural choice to describe temporal properties of unbounded systems. While LTL is useful to us, we cannot express any counting properties, hence, we propose using the counting logic language LTL_{LIA} an extension of LTL with a few differences (cf. Sec. 3.2). In the case of

LTL , atomic formulas are propositional constants which have no further structure. The syntax of the logic is explained with respect to client-server systems, but is also applicable in other infinite state systems such as Fig. 2.6. In LTL_{LIA} , there are three types of atomic formulas: (1) describing basic server properties, P_s which are propositional constants (2) **counting** sentences of the kinds $(\#x > c)\alpha$ and $(\#x \leq c)\alpha$ over client properties and c is a non-negative integer, denoting the number of clients in α and (3) **comparing** sentences of the kinds $(\#x)\alpha \leq \beta$ and $(\#x)\alpha > \beta$.

Formally, the set of client formulas Δ is given by:

$$\begin{aligned} \alpha, \beta \in \Delta ::= & (\#x > c)p(x) \mid (\#x \leq c)p(x) \mid (\#x)p(x) \leq q(x) \mid (\#x)p(x) > q(x) \\ & \mid \alpha \vee \beta \mid \alpha \wedge \beta \end{aligned}$$

where $p, q \in P_c$ and c is a non-negative integer, as noted above. While Δ does not contain an explicit $=$ operator, it is natural to express equality of $p(x)$ and $q(x)$. One way to express this is as follows: $\neg((\#x)p(x) > q(x)) \vee ((\#x)q(x) > p(x))$. Another way would be $(\#x)p(x) \leq q(x) \wedge \neg((\#x)q(x) > p(x))$.

The server formulas are defined as follows:

$$\psi \in \Psi ::= q \in P_s \mid \varphi \in \Delta \mid \neg\psi \mid \psi \vee \psi \mid \psi \wedge \psi \mid X\psi \mid F\psi \mid G\psi \mid \psi_1 U \psi_2$$

Modalities X , F , G and U are the usual modal operators: Next, Eventually, Globally and Until respectively. For the semantics and encoding of this logic we refer the reader to Sec. 3.2.

5.2 Encoding

Now that we have discussed the logic, in the subsequent section, we encode the bounded model checking problem for nets on LTL_{LIA} using various semantics.

Encoding 2D-BMC problems for nets on LTL_{LIA}

First, we describe the notation that we have used. Let $[\mathcal{M}]$ be the SMT encoding of the Petri net model of the system $\mathcal{M}$. Let ϕ be the LTL_{LIA} property that we want to verify. As usual we negate the property ϕ and let $\psi = \neg\phi$. We also assume that ψ is used in its negation normal

form. The 2D-BMC encoding of the system $\mathcal{M}$ against ψ for the bound $k = \lambda + \kappa$ (where $k \geq 0$) is denoted by $[\mathcal{M}, \psi]_{\langle \lambda, \kappa \rangle}$ and defined as follows:

$$[\mathcal{M}, \psi]_{\langle \lambda, \kappa \rangle} = [\mathcal{M}]_{\langle \lambda, \kappa \rangle} \wedge \left(\left(\neg L_{\langle \lambda, \kappa \rangle} \wedge [\psi]_{\langle \lambda, \kappa \rangle}^0 \right) \vee \bigvee_{l=0}^{\kappa} ({}_l L_{\langle \lambda, \kappa \rangle} \wedge {}_l [\psi]_{\langle \lambda, \kappa \rangle}^0) \right)$$

The bound k has two parts λ and κ : λ gives the bound for time instances and κ gives the bound for the number of clients. The propositional formula $[\mathcal{M}]_{\langle \lambda, \kappa \rangle}$ encodes the runs of $\mathcal{M}$ of bound $k = \lambda + \kappa$. The formulae $[\mathcal{M}]_{\langle \lambda, \kappa \rangle}$, for $0 \leq \lambda, \kappa \leq k$ are defined later in this section.

The formulas ${}_l L_{\langle \lambda, \kappa \rangle}$ ($0 \leq l \leq \lambda$) and $L_{\langle \lambda, \kappa \rangle}$ are loop conditions that are mentioned in the encoding above. For any $0 \leq l \leq \lambda$, ${}_l L_{\langle \lambda, \kappa \rangle} = \mathcal{T}(s_\lambda, s_l)$ where s_l is the l th state in the run of $\mathcal{M}$ and s_λ is the λ th state. Here, the state is defined in terms of value of the marking vector M . So, for any instance i , s_i corresponds to the state of vector M at i . Clearly, when ${}_l L_{\langle \lambda, \kappa \rangle}$ holds it means there is a transition from s_λ to s_l which denotes a back loop to the l th state. The other loop condition is defined as follows: $L_{\langle \lambda, \kappa \rangle} = \bigvee_{0 \leq l \leq \lambda} {}_l L_{\langle \lambda, \kappa \rangle}$. When $L_{\langle \lambda, \kappa \rangle}$ holds, it means there is a back loop to some state in the bounded run of $\mathcal{M}$.

Formula $[\psi]_{\langle \lambda, \kappa \rangle}^0$ is the SMT encoding of ψ for the bound $k = \lambda + \kappa$ when ψ is asserted at initial instance $i = 0$ and there is no loop in the run of $\mathcal{M}$. On the other hand, ${}_l [\psi]_{\langle \lambda, \kappa \rangle}^0$ is the SMT encoding of ψ for the bound $k = \lambda + \kappa$ when ψ is asserted at initial instance $i = 0$ and there is a back loop to the l th state in the run of $\mathcal{M}$. The formula $[\psi]_{\langle \lambda, \kappa \rangle}^i$ denotes the SMT encoding of ψ where the bounded run does not have a loop. The formula ${}_l [\psi]_{\langle \lambda, \kappa \rangle}^i$ denotes the SMT encoding of ψ where the bounded run contains a loop for any i . These encodings are extensions of similar mappings defined in [24] and are given in Section 3.2.

The bound $k = \lambda + \kappa$ starts from 0 and is incremented by 1 in each (macro-)step. For a fixed k , λ may start from 0, incremented by 1 in each (micro-)step till k . Simultaneously, κ may move from k to 0 and decrement by 1 in each (micro-)step. We look at each (micro-)step. Let the variables being used in the Boolean encoding of Petri net be $t_0, t_1, \dots, t_{n_t}$ and $p_0, p_1, \dots, p_{n_p}$ to represent transitions and places respectively. We use the i -th copy of the transition variable for instance i as follows $t_{0i}, \dots, t_{n_t i}$, place variables $p_{0j}, \dots, p_{n_p i}$, where $0 \leq i \leq \lambda$, in $[\mathcal{M}]_{\langle \lambda, \kappa \rangle}$. For any $\kappa \geq 0$, we define

$$[\mathcal{M}]_{\langle 0, \kappa \rangle} = I(s_0) \wedge \left(\bigwedge_{0 \leq j \leq n_p} p_{j0} \leq \kappa \right).$$

Inductively, for any $\lambda > 0$,

$$[\mathcal{M}]_{\langle \lambda, \kappa \rangle} = [\mathcal{M}]_{\langle \lambda-1, \kappa \rangle} \wedge \left(T(s_{\lambda-1}, s_\lambda) \wedge \left(\bigwedge_{0 \leq j \leq n_p} p_{j\lambda} \leq \kappa \right) \right).$$

Now that we have formally described the notations for encoding of the system $\mathcal{M}$, we derive the SMT encoding of $LT L_{LIA}$ for $\mathcal{M}$ from the bounded semantics described earlier.

In this section, we describe the SMT encoding, which is necessary for implementing a bounded model checker tool for Petri nets using $LT L_{LIA}$ specifications. We need to introduce counter variables for each place p in the input Petri net in order to define the SMT encodings of the property ψ . These extra variables are as follows: $\{c_p^i \mid i \geq 0, \text{ and } p \text{ is a place in the Petri net}\}$. Now we describe the SMT encodings of ψ .

We define $[\psi]_{\langle \lambda, \kappa \rangle}^i$ inductively as follows:

1. $[\#p]_{\langle \lambda, \kappa \rangle}^i \equiv c_p^i$
2. $[\mathbf{c}]_{\langle \lambda, \kappa \rangle}^i \equiv \mathbf{c}$
3. $[\mathbf{c} * \#p]_{\langle \lambda, \kappa \rangle}^i \equiv \mathbf{c} \times c_p^i$
4. $[\#p * \mathbf{c}]_{\langle \lambda, \kappa \rangle}^i \equiv c_p^i \times \mathbf{c}$
5. $[\alpha + \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i + [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
6. $[\alpha - \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i - [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
7. $[\alpha < \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i < [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
8. $[\alpha > \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i > [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
9. $[\alpha \leq \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i \leq [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
10. $[\alpha \geq \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i \geq [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
11. $[\alpha = \hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i \equiv [\alpha]_{\langle \lambda, \kappa \rangle}^i = [\hat{\alpha}]_{\langle \lambda, \kappa \rangle}^i$
12. $[q]_{\langle \lambda, \kappa \rangle}^i \equiv q_i$
13. $[\neg q]_{\langle \lambda, \kappa \rangle}^i \equiv \neg q_i$
14. $[\psi \vee \psi']_{\langle \lambda, \kappa \rangle}^i \equiv [\psi]_{\langle \lambda, \kappa \rangle}^i \vee [\psi']_{\langle \lambda, \kappa \rangle}^i$
15. $[\psi \wedge \psi']_{\langle \lambda, \kappa \rangle}^i \equiv [\psi]_{\langle \lambda, \kappa \rangle}^i \wedge [\psi']_{\langle \lambda, \kappa \rangle}^i$

16. $[X\psi]_{\langle\lambda,\kappa\rangle}^i \equiv \begin{cases} [\psi]_{\langle\lambda,\kappa\rangle}^{i+1} & \text{if } i < \lambda \\ \text{False} & \text{otherwise} \end{cases}$
17. $[F\psi]_{\langle\lambda,\kappa\rangle}^i \equiv \bigvee_{i \leq j \leq \lambda} [\psi]_{\langle\lambda,\kappa\rangle}^j$
18. $[G\psi]_{\langle\lambda,\kappa\rangle}^i \equiv \text{False}$
19. $[\psi U \psi']_{\langle\lambda,\kappa\rangle}^i \equiv \bigvee_{i \leq j \leq \lambda} ([\psi']_{\langle\lambda,\kappa\rangle}^j \wedge \bigwedge_{i \leq j' < j} [\psi]_{\langle\lambda,\kappa\rangle}^{j'})$

We define ${}_l[\psi]_{\langle\lambda,\kappa\rangle}^i$ inductively. To conserve space, encoding is given only where they differ from the corresponding case without loop.

16. ${}_l[X\psi]_{\langle\lambda,\kappa\rangle}^i \equiv \begin{cases} {}_l[\psi]_{\langle\lambda,\kappa\rangle}^{i+1} & \text{if } (i < \lambda) \\ {}_l[\psi]_{\langle\lambda,\kappa\rangle}^l & \text{if } (i = \lambda) \end{cases}$
17. ${}_l[F\psi]_{\langle\lambda,\kappa\rangle}^i \equiv \bigvee_{\min(l,i) \leq j \leq \lambda} {}_l[\psi]_{\langle\lambda,\kappa\rangle}^j$
18. ${}_l[G\psi]_{\langle\lambda,\kappa\rangle}^i \equiv \bigwedge_{\min(l,i) \leq j \leq \lambda} {}_l[\psi]_{\langle\lambda,\kappa\rangle}^j$
19. ${}_l[\psi U \psi']_{\langle\lambda,\kappa\rangle}^i \equiv \begin{cases} \bigvee_{i \leq j \leq \lambda} ({}_l[\psi']_{\langle\lambda,\kappa\rangle}^j \wedge \bigwedge_{i \leq j' < j} {}_l[\psi]_{\langle\lambda,\kappa\rangle}^{j'}) & \text{if } (i \leq l) \\ \left(\left(\bigvee_{i \leq j \leq \lambda} ({}_l[\psi']_{\langle\lambda,\kappa\rangle}^j \wedge \bigwedge_{i \leq j' < j} {}_l[\psi]_{\langle\lambda,\kappa\rangle}^{j'}) \right) \right. \\ \quad \text{or} & \text{if } (i > l) \\ \left. \left(\bigvee_{l \leq j < i} ({}_l[\psi']_{\langle\lambda,\kappa\rangle}^j \wedge \bigwedge_{l \leq j' < j} {}_l[\psi]_{\langle\lambda,\kappa\rangle}^{j'}) \right) \right) & \end{cases}$

In order to determine the encoding of $[M]_{\langle 0, \kappa \rangle}$, we need to perform **and** operation on the initial configuration and the place constraints. The place constraints are due to the κ variable, for each of the places.

$$[M]_{\langle 0, \kappa \rangle} = I(s_0) \wedge \left(\bigwedge_{0 \leq j \leq n_p} p_{j0} \leq \kappa \right).$$

The variable with respect to time instances is denoted as λ . Hence, it is also a positive integer. In order to determine $[M]_{\langle \lambda, \kappa \rangle}$, we can make use of the previously encoded $[M]_{\langle \lambda-1, \kappa \rangle}$ and unfold the transition relation by one length, and **and** it with the place constraints as done previously (cf. Fig.5.1). Hence we get the resultant relation:

$$[M]_{\langle \lambda, \kappa \rangle} = [M]_{\langle \lambda-1, \kappa \rangle} \wedge (T(s_{\lambda-1}, s_\lambda) \wedge \left(\bigwedge_{0 \leq j \leq n_p} p_{j\lambda} \leq \kappa \right)).$$

The encoding of the $\neg$, $\wedge$, $\vee$ operators are straightforward. We also define the encoding using the temporal operators.

Notice that $[(\#x > k)p(x)]_{\langle \lambda, \kappa \rangle}^i$ is encoded using the counter c_p at the place p , and this is translated as $c_p > \kappa$. Similarly, we have

$[(\#x > k)p(x)]_{\langle \lambda, \kappa \rangle}^i \equiv c_p > \kappa$. This encoding is similar for the loop-free formula as well.

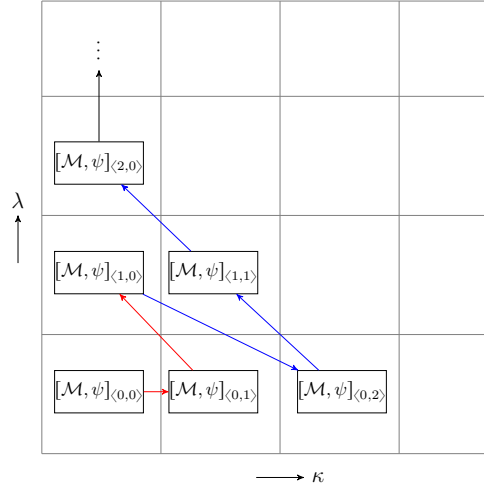


Figure 5.1: Unfolding of the encoded formula $[\mathcal{M}, \psi]_{(\lambda, \kappa)}$ with respect to λ (execution steps) and κ (number of tokens)

Unfolding the encoded formula

The unfolding of the formula $[\mathcal{M}, \psi]_{\langle \lambda, \kappa \rangle}$ for each k with respect to λ (execution steps) and κ (number of tokens), upto $k = 2$ is depicted pictorially in Fig. 5.1. Initially, when $k = \lambda + \kappa = 0$, $k = 0$ (bound), $\lambda = 0$ (time instance), $\kappa = 0$ (number of clients), the formula $[\mathcal{M}, \psi]_{\langle 0, 0 \rangle}$ is evaluated. If this is found to be satisfiable, then, a witness is obtained, and we are done. If not, in the next macro-step of the unfolding, where $k = \lambda + \kappa = 1$, there are possibly two micro-steps to be explored $\langle \lambda, \kappa \rangle = \langle 0, 1 \rangle$ and $\langle \lambda, \kappa \rangle = \langle 1, 0 \rangle$. First, κ is incremented and the formula $[\mathcal{M}, \psi]_{\langle 0, 1 \rangle}$ is evaluated. If this found to be unsatisfiable, λ is incremented and the formula $[\mathcal{M}, \psi]_{\langle 1, 0 \rangle}$ is evaluated. In each micro-step of the unfolding, either λ or κ are incremented, and the resulting formula is verified. The Fig. 5.1 represents one among many possible ways of exploring the state space in the system.

5.3 DCMModelchecker : A tool to verify Unbounded PNs

DCModelChecker 1.0 and DCModelChecker 2.0 perform 2D-BMC of Petri nets using the logic $LT L_{LIA}$. The key difference between the two tools is in the semantics used for the nets. DCModelChecker 1.0 [107] can verify $LT L_{LIA}$ properties with interleaving semantics. We incrementally extended DCModelChecker 1.0 by adding support for true concurrent semantics to the tool, to obtain DCModelChecker 2.0 [106] which supports both. In the rest of this chapter, when we mention tool, we refer to DCModelChecker 2.0. We give the architecture and the overview of the tool in Section 5.3.1. We describe the workflow in Section 5.3.2, and in Section 5.4, we report the experiments. The experiments are easily reproducible by directly executing the scripts in our artifact [106].

5.3.1 Architecture

The general architecture of the tool is shown in Fig. 5.2. The tool has two primary inputs- system description in standard Petri Net Markup Language (PNML) [72] format and the property to be tested expressed in $LT L_{LIA}$. PNML is based on the Extensible Markup Language (XML), which lends itself to interoperability between tools, while ensuring readability. We employ the ANother Tool for Language Recognition (ANTLR) [101] tool for parsing the two inputs. We obtain industrial and academic benchmarks from Model Checking Contest (MCC) [84]. Additionally, we handcraft some Petri net inputs using a Petri net Editor [22, 133]. We make use of both types of benchmarks, for comparative testing. While several Petri net verification tools perform bounded model checking, ours is unique in the model checking strategy, displaying the counterexample and useful in particular for verifying temporal properties and invariants of unbounded client-server systems.

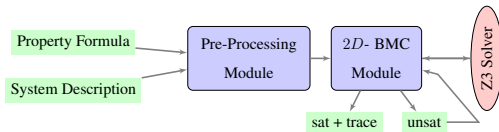


Figure 5.2: DCMModelChecker 2.0 architecture

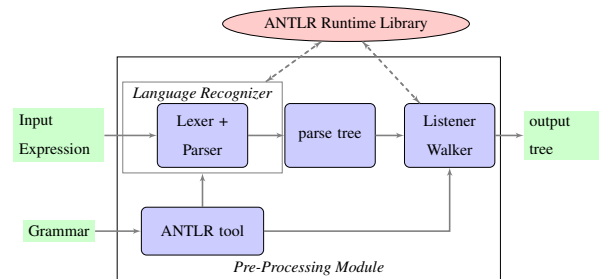


Figure 5.3: Pre-Processing the model and formula using ANTLR

First, the two inputs are fed to the pre-processing module and consequently to the DC-ModelChecker 2.0 tool. The objective of the pre-processing module is to read and validate the two inputs. We make use of ANTLR to achieve this. We give the grammar of the nets and properties to the tool so that it can recognize it against its respective grammar as shown in Fig. 5.3. ANTLR generates a parser for that language that can automatically build parse trees representing how a grammar matches the input. The parse trees can be walked to construct the required data structures. We have hand coded the grammar for the PNML format of both types and the grammar of LT_{LIA} , to be used by ANTLR. This is explained in detail in Sec. 5.3.3.

The tool reads the output of the pre-processing module and checks if the model satisfies the property or not. We make use of the Z3 SAT/SMT Solver [51], to solve the encoded formula and give us a result of unsatisfiable, or satisfiable with a counterexample trace. If unsatisfiable, the tool can increment the bound and look further, until the external termination bound is hit, according to the BMC algorithm. We chose Z3, for its wide industrial applications, developer community support, and ease of use. The detailed workflow is discussed in the subsequent section.

5.3.2 Workflow

The system description M , the property formula ϕ , and external termination bound k are given to us. First, we negate the property, $\psi = \neg\phi$. Note that the negation normal form of ψ is always used in the BMC process. For the bound $k = 0$ and the corresponding micro-step $\langle\lambda, \kappa\rangle = \langle 0, 0\rangle$, we construct the formula $[M, \psi]_{\langle 0, 0\rangle}$ and feed it to the solver. If the above formula is satisfiable, the property ϕ is violated in the initial configuration and we have a **witness** at $k = 0$ and we can stop our search. If the base case is unsatisfiable, we consider the next micro-step $\langle\lambda, \kappa\rangle$ – with the bound $k = \lambda + \kappa = 1$ – and construct the formula $[M, \psi]_{\langle\lambda, \kappa\rangle}$ and feed it to the solver. If this formula is satisfiable, the property ϕ is violated for $k = \lambda + \kappa$ and we have a **witness** for this k and we can stop our search. Otherwise, we may continue with the next micro-step $\langle\lambda', \kappa'\rangle$ and so on. The order in which the micro-steps are considered is illustrated in Fig. 5.1. We have depicted the workflow only up to $k = 1$ in Fig. 5.5. In practice, for the property ϕ , we search until we have considered all micro-steps for the termination bound k . As expected, for the given model and bound 100 (i.e, when parameter k reaches 100) it is observed that a counterexample is not found, hence we terminate. This means that the property holds for all traces of the model up to the length 100.

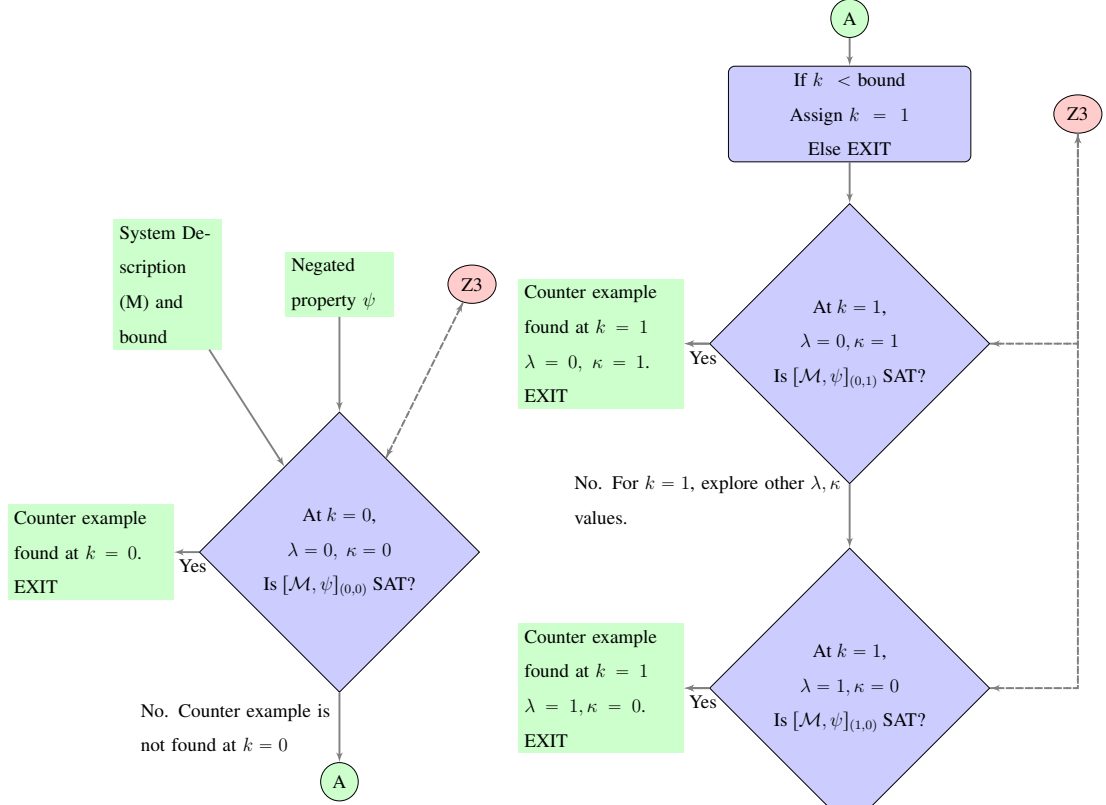


Figure 5.4: Workflow for $k = 0$

Figure 5.5: Workflow for $k = 1$

5.3.3 Pre-processing Petri nets

In order to build a verification tool that uses Petri nets, we require a standard way of representation. While, various Petri net verification tools exist, recently, the most widely accepted standard is the Petri Net Markup Language (PNML). PNML is based on the Extensible Markup Language, or XML for short, which lends itself to interoperability between tools, while ensuring readability. We employ the ANother Tool for Language Recognition (ANTLR) for parsing the PNML input. This tool was chosen, since it will also enable us to pre-process the various extensions to Petri nets (cf. Chapter. 2) and the various logic specifications (cf. Chapter. 3). This module is also available as an open source tool [112].

The tools discussed and built as part of this research work are compatible with any Petri net verification/analysis tools that use the PNML format. Additionally, one may also use existing graphical tools to visualise the Petri nets, limited only by the size of the Petri net [133]. This

decision was made early in the work, to enable reuse of the prototypes and modules built in the tool chain by other research groups and to ensure reproducibility of results.¹ We list down the actual **lexer** and **parser** grammars that were used to pre-process the nets in the following section.

Lexer rules:

First, we name the following set of rules, such that we can refer to in the parsing phase:

```
1 lexer grammar PNMLLexer;
```

Consider the following rule that recognizes the escape sequences for tab spaces, carriage return and new line respectively and skips over them when they occur in the input file (PNML file).

```
1 S : [ \t\r\n]+ -> skip ;
```

The PNML format is a type of markup file which contains tags. We have the following rules to identify the open, close and comment tags; the digits and text:

```
1 COMMENT : '<!--' .*? '-->' -> skip;
2 SPECIAL_OPEN: '<?' ->
    pushMode(INSIDE) ;
3 OPEN : '<' -> pushMode(INSIDE) ;
4 DIGIT : '[0-9]+' ;
```

```
5 TEXT : '~[&]+' ; //16 bit char except
    < and &
6 mode INSIDE;
7 CLOSE : '>' -> popMode ;
8 SPECIAL_CLOSE: '?>' -> popMode ;
9 SLASH_CLOSE : '/>' -> popMode ;
```

We have rules to tokenize the keywords and special characters used in the PNML file:

```
1 SLASH : '/' ;
2 EQUALS : '=' ;
3 QUOTE : '"' ;
4 SQUOTE : '\'' ;
5 PLACE : 'place' ;
6 TRANSITION : 'transition' ;
7 ARC : 'arc' ;
8 INITIAL : 'initialMarking' ;
9 INSCRIPTION : 'inscription' ;
10 TEXTTAG : 'text' ;
11 USCORE : '_' ;
12 SOURCE : 'source' ;
13 TARGET : 'target' ;
14 ID : 'id' ;
15 STRING : '"' ~[<"]* '"'
16         | '\'' ~[<']* '\'' ;
```

¹It is noteworthy, that when this research was submitted to Petri net tool venues, the tool was awarded the reproducibility badge, however as the tool paper itself was not accepted, it is not being displayed alongside.

We have a special set of rules for the name tag. The name consists of an alphabet and may be succeeded by any combination of digits, alphabets and special symbols (hyphen, underscore, dot), as allowed by the Petri net graphical analysis tool that we used [133].

```

1 Name : NameStartChar NameChar* ;
2 NameChar : NameStartChar
3           | '-' | '_' | '.' | DIGIT
4           | '\u00B7'
5           | '\u0300' .. '\u036F'
6           | '\u203F' .. '\u2040' ;
7 NameStartChar : [:a-zA-Z]
8               | '\u2070' .. '\u218F'
9               | '\u2C00' .. '\u2FEF'
10              | '\u3001' .. '\uD7FF'
11              | '\uF900' .. '\uFDCF'
12              | '\uFDF0' .. '\uFFFD' ;

```

Parser rules:

Now that we have the lexer rules that can identify and tokenize the input, in this section, we write the the parser rules which will be used by ANTLR to construct the parse tree from the valid input and throw errors if any.

First, we set the specific vocabulary of the parser as **PNMLLexer** (the set of lexer rules we defined above):

```

1 parser grammar PNMLParser;
2 options {
3     tokenVocab = PNMLLexer;
4 }

```

A valid PNML file contains a header followed by valid elements:

```

1 doc: header element;

```

An element may be either an open tag followed by a close tag or an empty tag, each containing its name and possibly containing a set of attributes. There may be open tags for exactly one of the predefined keywords such as place, transition etc. Notice that it is not possible to define just a close tag using these set of rules. If such an input exists, our tool parses the PNML file and invalidates it by throwing a suitable error.

```

1 header: '<?' Name attribute* '?>';
2 element:
3     '<' Name attribute* '>' (
4         place
5         | transition
6         | arc
7         | element
8         | textTagDigit
9         | textTag
10        | TEXT
11    ) * '<' '/' Name '>'
12    | '<' Name attribute* '/>';

```

The place tag contains a mandatory identifier attribute and may contain details to represent the initial marking or special text for parsing.

```

1 place: '<' 'place' 'id' '=' STRING '>' (initial | element | TEXT) * '<' '/'
      'place' '>';

```

The rule for parsing the initial marking:

```

1 initial:
2     '<' INITIAL '>' (textTagDigit | textTag | element | TEXT) * '<' '/'
      INITIAL '>';

```

The rule for parsing transitions, with a mandatory identifier.

```

1 transition:
2     '<' 'transition' 'id' '=' STRING '>' (element | TEXT) * '<' '/'
      'transition' '>';

```

The rule to recognize arcs from a place/transition to transition/place along with their identifiers.

```

1 arc:
2     '<' 'arc' 'id' '=' STRING source target '>' (
3         inscription
4         | element
5         | TEXT
6     ) * '<' '/' 'arc' '>'
7     | '<' 'arc' 'id' '=' STRING source target '/>';
8 source: 'source' '=' STRING;
9 target: 'target' '=' STRING;

```

Some additional rules to parse the text and specify the type of net for validation:

```
1 inscription:
2   '<' INSCRIPTION '>' (textTagDigit | textTag | element | TEXT)* '<' '/'
   INSCRIPTION '>';
3 textTagDigit: '<' TEXTTAG '>' DIGIT '<' '/' TEXTTAG '>';
4 textTag: '<' TEXTTAG '>' TEXT '<' '/' TEXTTAG '>';
5 attribute: ('id' | Name) '=' STRING;
```

Similarly, the rules are written to validate the logic LTL_{LIA} as well and are available in our artifact. As illustrated, in Fig. 5.2, given a valid system description in a PNML file and a valid specification which are validated by the pre-processing module, the 2D-BMC algorithm verifies the specification using Z3 and produces a counterexample trace or times out. In the next section, we look at the experiments conducted on our bounded model checking tool.

5.4 Benchmarks and Comparisons

In this section, we detail the experimental evaluation of DCMoelChecker 2.0. The goals of our evaluation are to demonstrate: (1) the correctness of results returned by DCMoelChecker 2.0 in comparison with the state-of-the-art ITS-Tools [122] (2) the comparison of DCMoelChecker 2.0 against DCMoelChecker 1.0 on similar FireabilityCardinality properties which are not verifiable by tools in the MCC (3) verification of Unbounded Petri nets in comparison with other tools (4) the strength of DCMoelChecker 2.0 is in additionally verifying invariants specified in LTL_{LIA} which are not verifiable by tools in the MCC nor DCMoelChecker 1.0. To replicate the experiments and plots, a collection of simple shell and python scripts are publicly available in our artifact [106].

Benchmarks: In the first set of experiments described in Section 5.4.1 we consider benchmark models from the MCC [84] and translated the LTLFireability properties into LTL_{LIA} . In our second set of experiments in Section 5.4.2, we make use of the MCC benchmark models and write the properties containing both fireability and cardinality constraints for those models, which are not expressible in the language of MCC. In the third set of experiments, we make use of synthetic unbounded Petri nets [9]. The fourth experiment demonstrates verification of invariants using the unbounded client-server system benchmark (APS) using DCMoelChecker 2.0.

		ITS Tools	
		True	False
DCModelChecker 2.0	SAT Property is False	A Erroneous output by DCModelChecker $ A = 7$	B Both tools agree on the property being false $ B = 87$
	UNSAT Property is True	C $ C = 72$	D $ D = 179$

Table 5.1: Experiment 1: Categorization of LTLFireability results.

5.4.1 Experiment 1: Computing the Tool Confidence and Verification of LTLFireability properties - DCModelChecker 2.0 vs the state-of-the-art tool

To compute the Tool Confidence, 22 benchmarks from the MCC were verified using DCModelChecker 2.0 and compared against the state-of-the-art model checking tool ITS-Tools [122]. ITS-Tools has 100 % tool confidence [2]. We compare against only this tool since (at the time of experimentation) ITS-Tools was the MCC winner in the LTL Fireability category. Since the purpose of this experiment was to evaluate the tool confidence, it would not make any difference to compare against more than one tool. The comparative experiments were performed with a timeout of 3600 seconds.

Evaluation Criteria: ITS-Tools returns True when the property holds true and False when the property does not hold. DCModelChecker 2.0 however, returns a satisfying counterexample to the negation of the property when the property does not hold. When DCModelChecker 2.0 returns UNSAT, it means that the property holds up to the given bound, and that the property may become false at a greater bound. Hence, for comparison, we may categorize the results of both tools as shown in Table 5.1.

The instances in category A are those false positive properties where DCModelChecker 2.0 returned a counterexample, whereas ITS-Tools returned false. The instances in category B are those where both tools agreed that the property was false, this is particularly significant since it gives the correctness of DCModelChecker 2.0. The instances in category C are those where

DCModelChecker 2.0 returns UNSAT and ITS-Tools returns true. The instances in category D are those where DCModelChecker 2.0 returns UNSAT and ITS-Tools returns false. Those instances in categories C and D may become false at a greater bound, hence we do not conclusively take them into account. We consider only the properties that fall in category A and B for computing the confidence ratio. This is similar to the MCC tool confidence computation [1] which excludes those values that are not computed by the tool. The Confidence of a tool is a value $C_{tool} \in [0, 1]$, where 1 denotes that the tool returns the correct result that is agreed upon by atleast 3 participating tools and 0 denotes that the tool never returns the commonly agreed result (otherwise referred to as trusted values). In the LTL Formulas examination category, ITS-Tools has a tool confidence of 100%, $C_{ITS-Tools} = 1$. Hence, it is sufficient to compare DCModelChecker 2.0 against ITS-Tools to check for the correctness of the results. The C_{tool} is computed as follows:

$$C_{tool} = \frac{|V_{tool}|}{|V|} \quad (5.1)$$

where $|V|$ is the number of trusted values, which we obtain from ITS-Tools. $|V_{tool}|$ is the number of results obtained by DCModelChecker 2.0 within the set of trusted values, such that $V \subseteq V_{tool}$. For Experiment 1, comparing LTL Fireability properties of DCModelChecker and ITS-Tools, we obtained the following tool confidence $C_{DCModelChecker} = \frac{|A|}{|A+B|} = 0.90$. The underlying 2D-BMC strategy is not complete.

5.4.2 Experiment 2: Verification of FireabilityCardinality properties DC-ModelChecker 1.0 vs DCModelChecker 2.0

In this set of experiments, DCModelChecker 1.0 and DCModelChecker 2.0 are compared on similar properties. Here, 20 benchmarks from the MCC have been considered and their properties expressed in $\mathcal{L}_C$ [105] for verification by DCModelChecker 1.0 and their equivalent LTL_{LIA} properties for verification by DCModelChecker 2.0 respectively. The logic LTL_{LIA} is strictly more expressive than logic $\mathcal{L}_C$. It is to be noted that the languages $\mathcal{L}_C$ and LTL_{LIA} are not equivalent to the logic language used in the MCC. Hence, we have given 16 synthetic properties called FireabilityCardinality properties, derived from the LTLFireability and LTL-Cardinality properties from the MCC.

We introduced the set of FireabilityCardinality properties as a combination of the LTL-

Fireability and LTLCardinality properties. For instance, $G(F(t_0) \wedge F(\#p_1 > 999))$, which is a conjunction of the temporal property describing the firing of transition t_0 and the counting property with place p_1 .

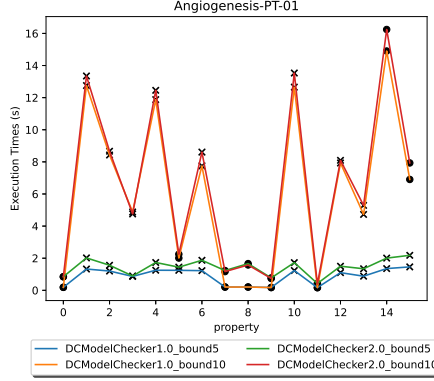
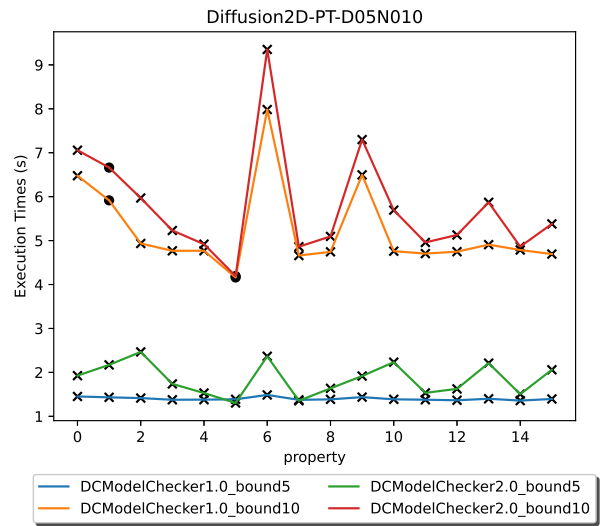
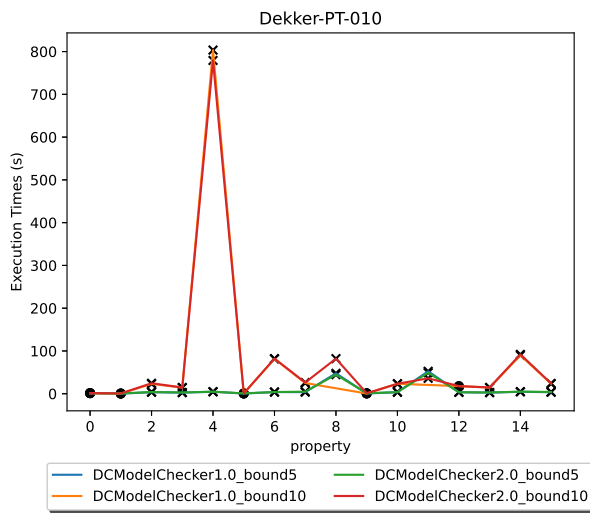
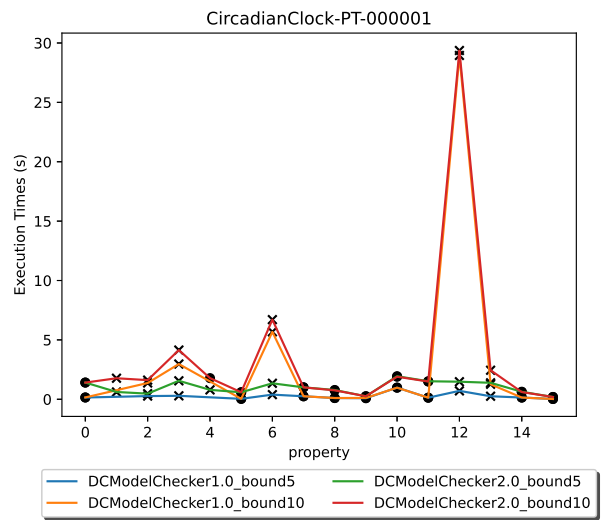
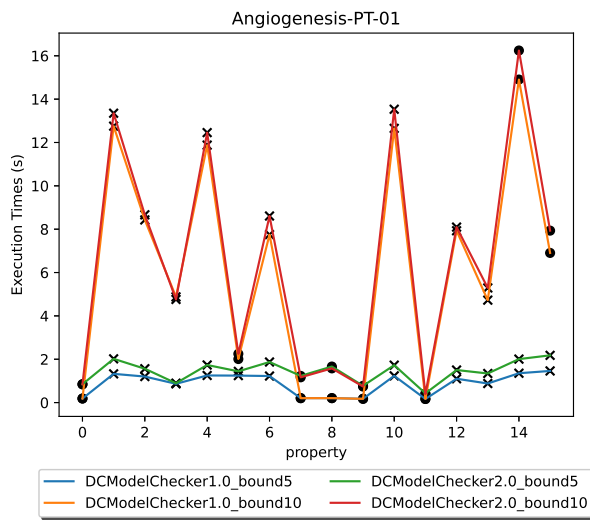
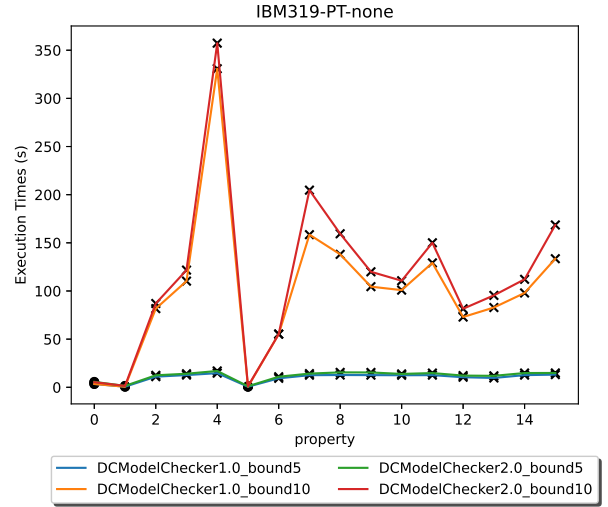
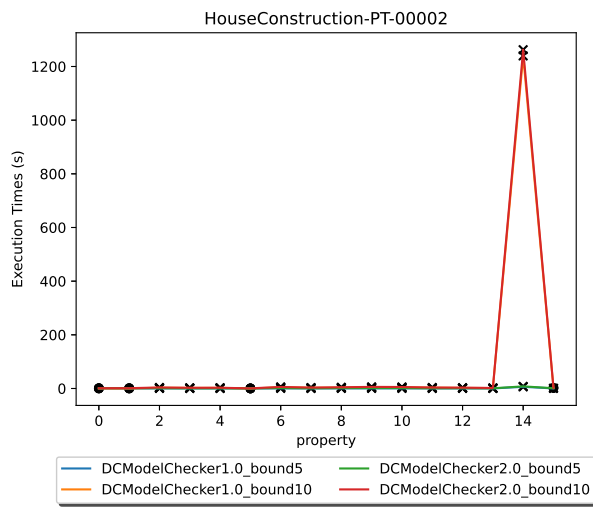
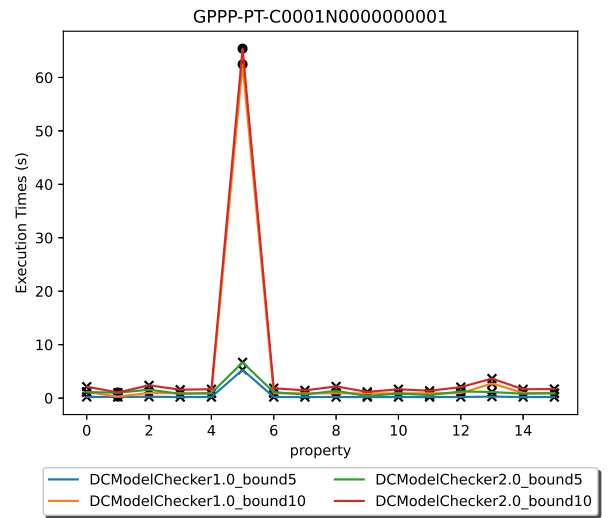
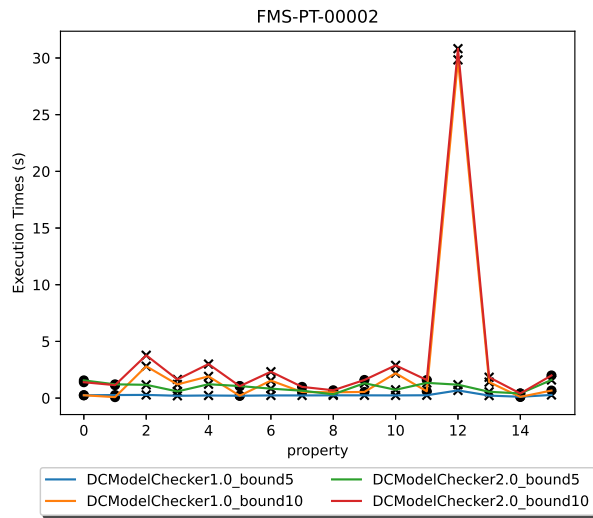
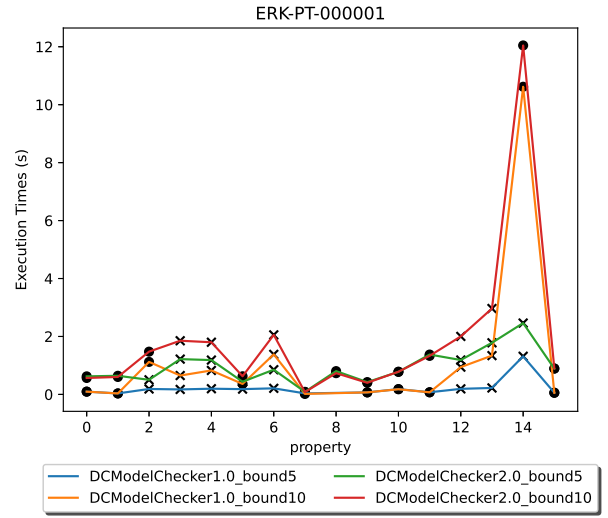
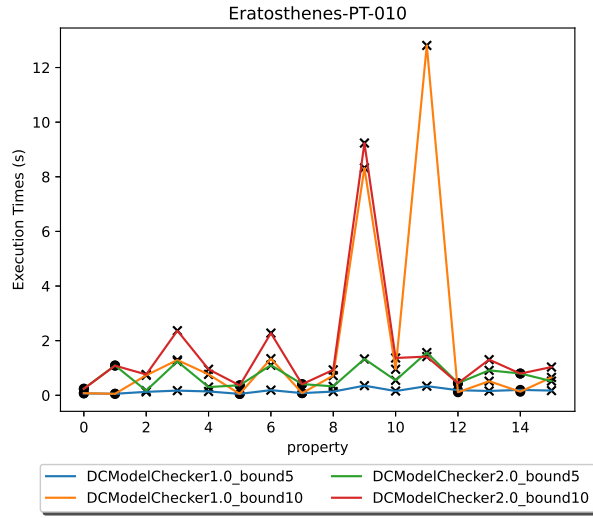
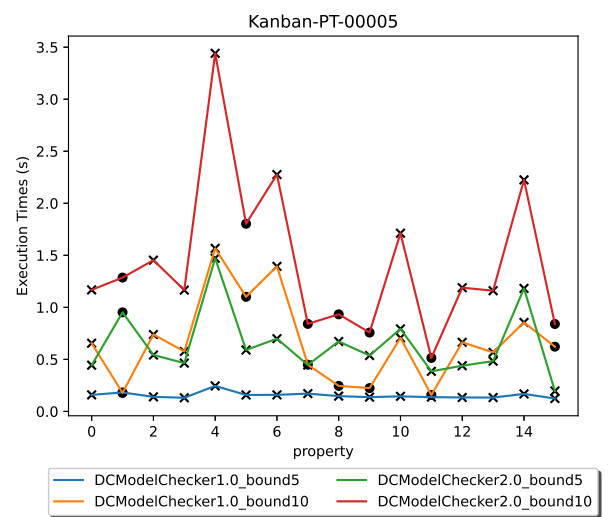
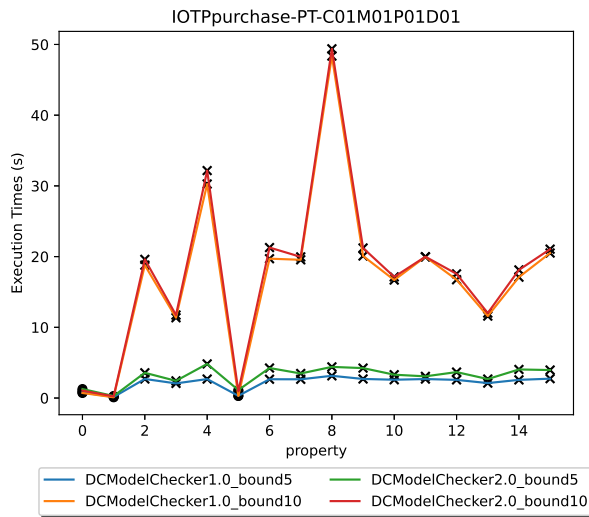
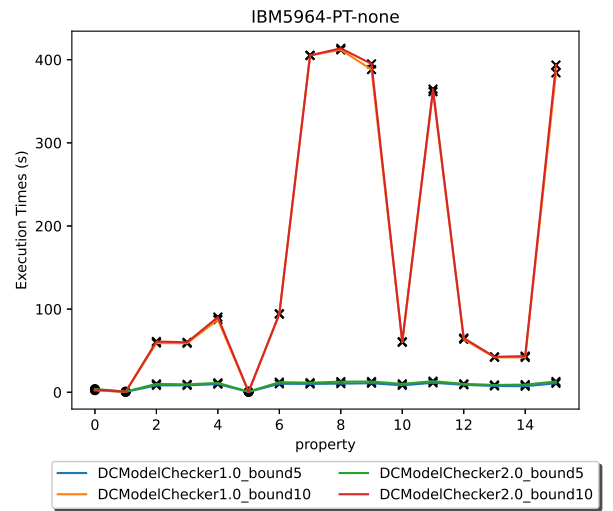
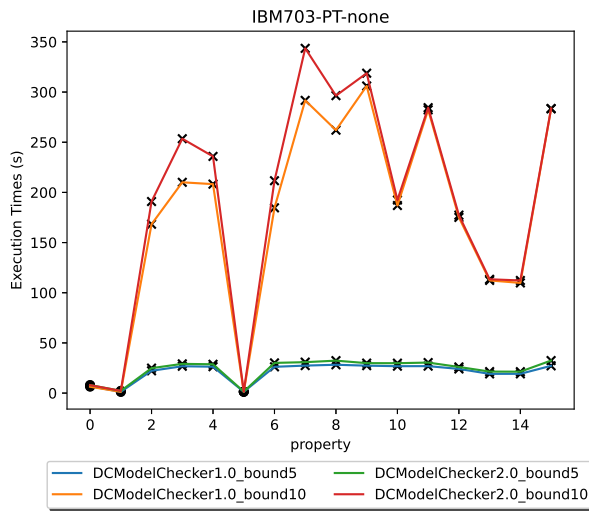


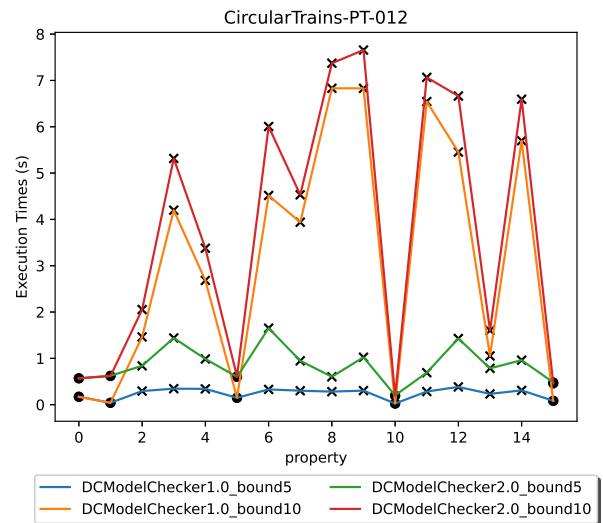
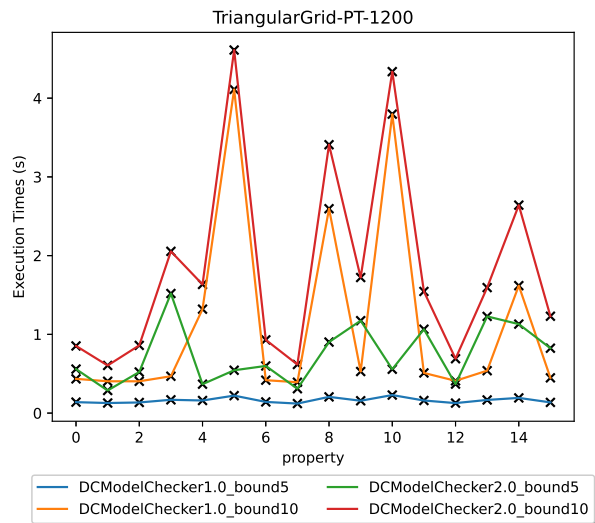
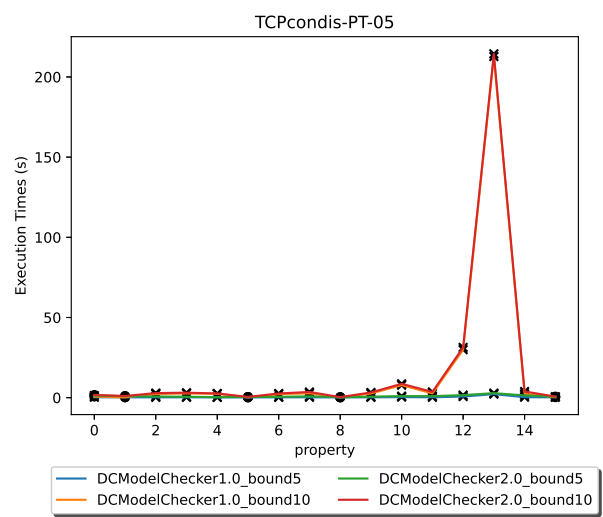
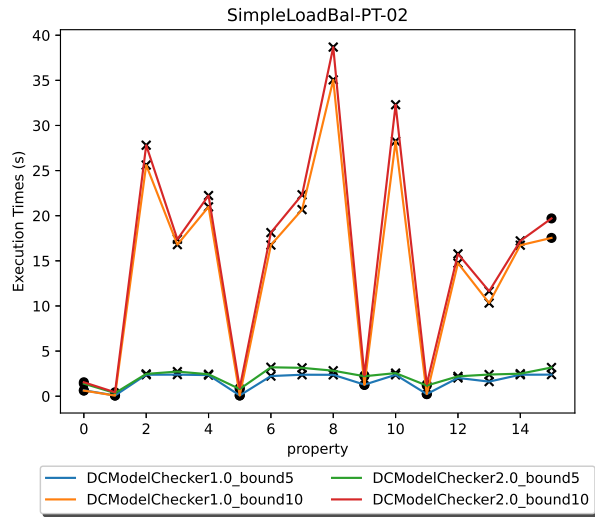
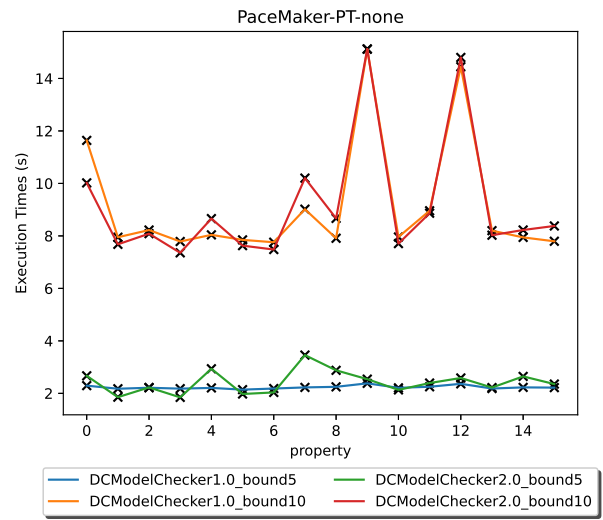
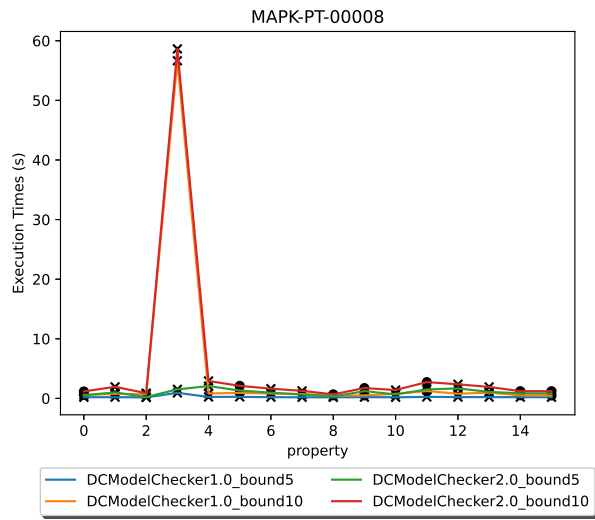
Figure 5.6: Experiment 2: Verification of FireabilityCardinality Properties with bound 5 and 10

For each of these 20 benchmarks, DCMoelChecker 2.0 was pitted against DCMoelChecker 1.0 to verify a set of 16 FireabilityCardinality properties for each model, totalling 320 properties and 1280 runs with a timeout of 3600 seconds each. The execution times (seconds) are plotted as separate line plots for bounds 5 and 10. As a sample, 16 properties were verified for the model Angiogenesis-PT-01 as shown in Fig 5.6. Each instance represents a FireabilityCardinality property. If the property is false and the tool returns a counterexample trace, it is depicted by a $\bullet$ and the symbol $\times$ denotes that the tool returned unsatisfiable (indicating that the property may be false at a greater bound). In Fig 5.6, we can observe that for bound 5 (blue line), out of the 16 properties, 5 properties were false. On increasing the bound to 10, we obtain 3 new counterexample traces (corresponding to properties 5, 14 and 15) as more search space is explored by the tool, which also takes more time. This trend is observed in all models in our experiment, when the bound is increased, new counterexamples may be found by tool. The execution times of DCMoelChecker 2.0 and DCMoelChecker 1.0 are comparable. We observe that DCMoelChecker2.0 is better in the sense of falsification of properties (not in terms of execution times), which can be seen from the plots such as the CircadianClock-PT-000001 model, where in property 4, DCMoelChecker1.0 (at bound 5 and 10) says unsat, whereas, DCMoelChecker2.0 (at bound 10) gives a satisfying counterexample trace. It is important to note that for each property both tools agree on the SAT/ UNSAT answers returned by them. The plots depicting the experiments follow.









5.4.3 Experiment 3: Verification of Unbounded Petri nets

To quote Amat et al, “It is difficult to find benchmarks for unbounded Petri nets” [9]. The strength of DCMoelChecker 2.0 lies in verifying unbounded Petri nets. Since, MCC benchmarks are bounded [9], we make use of the unbounded Petri nets given by [99] and verify LTLFireability properties in comparison with DCMoelChecker 1.0 [107], ITS-Tools [122]. The results of these experiments are in Table 5.2, and [107]. Each property that was verified is false. In each case it was observed that DCMoelChecker 2.0 and DCMoelChecker 1.0 returned that the property is false and additionally gave a counterexample trace, whereas ITS-Tools and Tapaal [49] returned that the property is false. Here, we compared our tools against both ITS-Tools and Tapaal, since Tapaal is faster than ITS-Tools and we give a comparative perspective of our tool’s performance. In Section 5.4.3, we verified LTLFireability properties of unbounded Petri nets which were all found to be false (while specifying the bound as 5). The properties are described in Table 5.3. The detailed output is available in the artifact [107].

Model	Execution Times(ms)			
	DCMoelChecker 1.0 (bound =5)	DCMoelChecker 2.0 (bound =5)	ITS-Tools	Tapaal
Parity	0.010	0.40	0.933	0.02
PGCD	0.019	0.98	1.487	0.07
Process	0.033	0.85	1.201	1e-05
CryptoMiner	0.036	0.64	0.957	7e-06
Murphy	0.031	0.83	1.161	8e-06

Table 5.2: Results of Experiment 3

Model	Property
Parity	$\neg(t_0 U t_1)$
PGCD	$\neg GF(t_0 U t_1)$
Process	$\neg F(t_0 U t_1)$
CryptoMiner	$\neg F(\text{OB } U \text{ GH})$
Murphy	$\neg F(t_1 U t_4)$

Table 5.3: Experiment 3: List of properties verified on unbounded nets

5.4.4 Experiment 4: Verification of invariants for unbounded client-server systems using DCModelChecker 2.0

We have additionally verified client-server properties specified in the $LT L_{LIA}$ language on our synthetic benchmark APS, against $LT L_{LIA}$ specifications. These cannot be verified using DCModelChecker 1.0 [107]. The specifications are available in our artifact [106]. Our experiments demonstrate that DCModelChecker 2.0 has an advantage over DCModelChecker 1.0 in being able to additionally verify invariants. And it can be added to the arsenal of BMC tools for Petri nets.

5.4.5 Related Petri net verification tools

There are several tools such as KREACH [54], Petrinizer [57], QCOVER [26], ICOVER [63] verifying specific classes of properties like reachability and coverability of Petri nets. Recently, in [8, 9] the generalized reachability of Petri net is encoded into a BMC problem and solved using SMT solvers. However, BMC for unbounded Petri nets with true concurrency and LTL properties while leveraging the power of SMT Solvers remained unexplored until now. DCModelChecker bridges this gap.

The MCC has attracted many tools for formal verification of concurrent systems that use a portfolio approach. In particular, we looked at ITS-Tools [122] which implements structural reduction techniques [123] and uses a layer of SMT when solving LTL. Moreover, it does not perform BMC. While ITS-Tools is much faster on bounded nets (MCC benchmarks) it gives only binary answers (sat or unsat). From our experiments, on a subset of unbounded nets, while Tapaal is faster, it also gives binary answers. On the contrary, our tool preserves the original Petri net structure and additionally gives a counterexample trace which is useful in bug finding in systems.

Conclusion

In this chapter, we discussed the first of its kind verification tool for checking properties of unbounded client server systems using 2D-BMC.

In Experiment 1 (Sec. 5.4.1) where we compute the tool confidence as well as verify LTL-Fireability properties, we compare against only ITS-Tools since (at the time of experimentation) ITS-Tools was the MCC winner in the LTL Fireability category. Since the purpose of this exper-

iment was to evaluate the tool confidence, it would not make any difference to compare against more than one tool, apart from the winning tool. In Experiment 3 (Sec. 5.4.3) where we verify unbounded Petri nets which is not a category in the contest, we compared our tools against both ITS-Tools and Tapaal, since Tapaal is faster than ITS-Tools and we give a comparative perspective of our tool's performance.

We would like to have an optimal encoding of the PN with true concurrent semantics, such that we can better understand the advantage of true concurrent semantics. This will be done as part of future work.

Next, we shall look at an extension of the Petri net, called ν -net and a different and richer logic, a variant of First Order logic, to express a different set of specifications of unbounded client server systems in the upcoming chapter.

Chapter 6

First Order LTL Tool with Monodic restriction

6.1 FO LTL over Petri nets with names

Formal verification of Petri nets is well studied. In practice, many communication protocols, services and applications are client-server systems. These systems can be modeled as Petri nets and verified using existing tools. However, these tools are not specifically suited for unbounded client-server systems as they do not allow the user to explicitly specify client and server properties as well as their unboundedness. Traditional logics such as LTL or CTL are not suitable for expressing these properties either. It is necessary to find suitable logics to express properties of unbounded client-server systems where the number of clients is not known a priori and the clients are distinguishable. To specify the properties of such systems, we consider a one variable fragment of Monadic First Order Temporal Logic (MFOTL), called FOTL_1 which is described in detail in Section 3.3. We propose a restriction on the ν -net to model the unbounded client server system. This model has been described earlier in Sec. 2.2.3.

6.2 Tool and Implementation

The verification tool follows the same underlying architecture as DCMoelChecker 2.0, where we employ Z3 to perform SMT queries using Linear Integer Arithmetic theory. The key difference is in the choice of logic and model. We make use of ANTLR to pre-process the ν -net, as previously discussed. In the rest of this chapter, we refer to the FOTL_1 properties of the

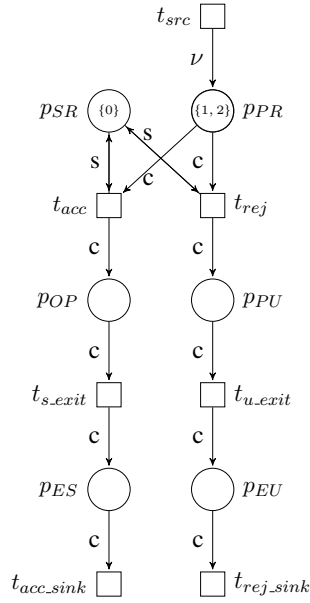


Figure 2.6: Recall the restricted ν -net modeling an unbounded client-server system

APS case study (See. 2.2.3). Since this is a first of its kind, there are no similar tools to compare the results of the model checker. However, we can see from the liveness windows that the counterexample or UNSAT obtained from the tool are justified.

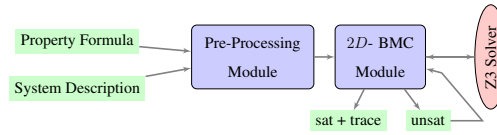


Figure 5.2: DCMModelChecker 3.0 architecture

Recognizing FOTL₁ properties

A valid FOTL₁ formula has the server formula *sformula* which contains the client formula *cformula* as a sub formula.

```

1 grammar mlogic;
2 input : sformula EOF;
3 sformula
4   : sorop | sandop | simpliesop | suntilop | snotop
5     | snextop | sdiamondop | sboxop ;
6 cformula
7   : (corop | candop | cimpliedop | cuntilop | cnotop
8     | cnextop | cdiamondop | cboxop );

```

We define the server temporal modalities:

```
1 sorop
2   : (exists | forall )(place_x | snotop )
3     OR
4     ssubformula;
5 sandop
6   : (exists | forall )(place_x | snotop )
7     AND
8     ssubformula;
9 simpliesop
10  : (exists | forall )(place_x | snotop )
11    IMPLIES
12    ssubformula;
13 suntilop
14  : (exists | forall ) (place_x | snotop | snextop | sdiamondop | sbboxop)
15    SUNTIL
16    ssubformula;
17
18 snotop
19  : NOT (exists | forall ) ssubformula;
20 snextop
21  : SNEXT (exists | forall ) ssubformula;
22 sdiamondop
23  : SDIAMOND (exists | forall ) ssubformula;
24 sbboxop
25  : SBBOX (exists | forall ) ssubformula;
26
27 ssubformula : LPAREN (cformula| place_x | snotop | snextop | sdiamondop
28                  | sbboxop | sandop | sorop | simpliesop | simpliesop)
                RPAREN;
```

Consequently, we define the client temporal modalities:

```
1 corop
2   : (place_x | cnotop | cnextop | cdiamondop | cbboxop)
3     OR
4     csubformula;
5 candop
6   : (place_x | cnotop | cnextop | cdiamondop | cbboxop)
```

```

7      AND
8      csubformula;
9 cimpliesop
10     : (place_x | cnotop | cnextop | cdiamondop | cboxop)
11       IMPLIES
12       csubformula;
13 cuntilop
14     : (place_x | cnotop | cnextop | cdiamondop | cboxop)
15       CUNTIL
16       csubformula;
17 cnotop
18     : NOT csubformula;
19 cnextop
20     : CNEXT csubformula;
21 cdiamondop
22     : CDIAMOND csubformula;
23 cboxop
24     : CBOX csubformula;
25
26 csubformula: LPAREN (place_x | cnotop | cnextop | cdiamondop
27                   | cboxop | candop | corop | cimpliesop | cuntilop) RPAREN;

```

We provide the rules governing quantifiers:

```

1 exists : EXISTS VAR;
2 forall : FORALL VAR;
3 place_x : PLACE DIGIT LPAREN VAR RPAREN;

```

We recognize the valid tokens for the various temporal modalities and operators:

```

1 EXISTS: 'E';
2 FORALL: 'V';
3 PLACE : 'p';
4 NOT   : '~';
5 SNEXT : 'X_s';
6 SDIAMOND : 'F_s';
7 SBOX  : 'G_s';
8 CNEXT : 'X_c';
9 CDIAMOND : 'F_c';
10 CBOX  : 'G_c';
11 OR   : '|';

```

```

12 AND : '&';
13 IMPLIES : '%';
14 SUNTIL : 'U_s';
15 CUNTIL : 'U_c';

```

We recognize the valid symbols such as paranthesis and identifiers and spaces:

```

1 LPAREN : '(';
2 RPAREN : ')';
3 NAME : '\'' ~ '[' + '\'';
4 VAR: [a-z];
5 DIGIT : '-' ? [0-9] +;
6 ENDLINE : ('\r\n' | '\n' | '\r') + -> skip;
7 WHITESPACE : [\t ] + -> skip;

```

Instead of providing separate parser and lexer rules, we provide the grammar for recognizing FOTL₁ in one shot. Using the above rules, we validate the FOTL₁ specifications in the accompanying tool.

6.2.1 Observations

We list down some observations about the logic FOTL₁ which supplement our understanding of the language.

Definition 6.2.1. *Ghost Interval* Given a temporal property of a client, whose right boundary is λ' such that the $\lambda' < \lambda$. There is a ghost interval between λ and λ' where the formula α need not be asserted.

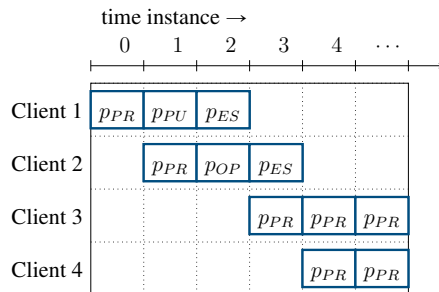


Figure 3.8: Snapshot of the running example (APS) depicting live windows

In Fig. 3.8, there is a *ghost interval* for client 1 between 1 (right boundary) and 5 (the bound λ).

Let $\psi_5 = (\forall x) \left(\mathbf{G}_c(\text{parking_requested}(x) \mathbf{U}_c \text{exit_successfully}(x)) \right)$. In ψ_5 , $\mathbf{G}_c$ holds only till the *live window* ends for the corresponding client i.e, this will be asserted only as long as the client is *live*. Similarly, all other client temporal modalities $\mathbf{F}_c$, $\mathbf{U}_c$, and $\mathbf{X}_c$ need not be asserted beyond the *live window* of that particular client.

At any given time, there are as many client lassos as there are clients, which are then combined to construct the server lasso. For currently *active* clients the right boundary will be the execution bound λ .

We list down some of the properties that cannot be expressed in FOTL_1 :

1. Consider the property, where there is at least one token in either place p_{SR} or p_{SB} or p_{RR} . From the net, $p_{SR} + p_{SB} + p_{RR} = 1$ is an invariant. These places represent the server being available, busy and the server rejecting the request.
2. The following property is not expressible in FOTL_1 : It is always the case that the client request with an identifier (say) 1001 is rejected.
3. It is always the case that if the client request is accepted at time instance x , then the client exits the parking space at time instance z , $z > x$.
4. Formulas with quantifiers before server temporal modality are not permitted.
5. We do not have formulas of type, client i satisfies a formula whereas client j doesn't satisfy the same formula. We do not have equality/inequality to differentiate between the clients. We assume that all clients behave the same way and are of the same type. In the future, this model could be extended to clients of a finite set of types.
6. We cannot compare clients in the same state.

$$\psi_6 = \mathbf{G}_s((\exists x)(\exists y) \text{parking_requested}(x) \wedge \text{parking_requested}(y) \wedge \mathbf{F}_c(\text{exit_successfully}(x) \wedge \mathbf{F}_c(\text{exit_successfully}(y))))$$

7. Here, there are no free variables either in the scope of $\mathbf{G}_s$ or $\mathbf{F}_s$.

$$\text{Let } \psi_7 = (\mathbf{G}_s(\exists x) \text{parking_requested}(x)) \wedge (\mathbf{F}_s(\exists y) \text{parking_requested}(y))$$

The formula ψ_7 states that it is always the case that there is a client who has requested parking and eventually, there exists a client who has also requested parking. However, we cannot compare these two clients in our logic.

The ease of expressibility of the client and server behaviour is the key motivation behind the logic FOTL₁ which is formally described in Sec. 3.3.

6.2.2 Experimental results

We test the properties of the APS case study (See Sec. 2.2.3) written using FOTL₁ using our open source tool [104]. The tool provides the detailed counterexample trace when the property is violated. We assume a bound of 5 for the below experiments in Table. 6.1. The properties are present in Sec. 3.3. To the best of our knowledge, there are no known benchmarks for FOTL₁ and ν -nets, hence we create our own ν -net model using a PN editor called Wolfgang and properties in FOTL₁ for our experiments.

Table 6.1: Experiments on APS

Property ID	Nesting Depth of temporal operators	Number of clauses	time (ms)
ψ_1	2	3	0.32
ψ_2	2	2	0.4
ψ_3	2	2	0.5
ψ_4	2	3	0.38

6.2.3 Limitations of the tool

The current toolchain depends on a PNML Editor, Wolfgang [133], which we use to draw the net and mention the arc labels. However, identifiable tokens are not supported by the Wolfgang editor, as it is primarily a Petri net and Colored Petri net simulation tool. Hence, we have to handcraft the ν -net arcs, which is not sustainable when the number of places and transitions are huge. While there are several tools to model other types of PNs, to the best of our knowledge, there are none that provide a graphical representation of ν -nets. We consider this as a future extension to this work. We would like to perform detailed experiments for various other ν -net models and by varying the nesting depth of the formulas, to identify if it has an impact on the execution times.

While the logic FOTL_1 allows us to reason about distinguishable clients, we cannot describe properties of each individual client. In future, we would like to explore different logics to do the same.

6.3 Bounded Semantics of FOTL_1

We introduce the (unbounded) satisfaction relation $\models_x$ with respect to the free variable x , where x is used to denote the client names. This relation is not necessary for the BMC tool, but it is useful in Lemma. 2. Hence it is given in Appendix. 6.A.

In this section, we describe the bounded semantics of FOTL_1 in order to arrive at the SMT encoding, which is necessary for BMC. We use $\models^k$ as a restriction on $\models$, where k is the bound in the BMC strategy. To tackle the unbounded number of clients as well as unboundedness in the unfolding of the model, it is necessary to introduce two bounds: κ is a bound on the number of clients and λ is a bound on the execution steps of the net. The two parameters κ and λ are independent of each other. Since the runs are bounded, there are atmost κ clients in the system, namely $\text{CN} = \{0, \dots, \kappa - 1\}$.

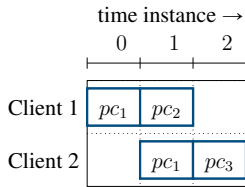


Figure 6.1: Snapshot at bound = 2

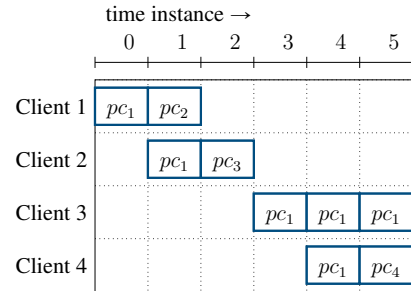


Figure 6.2: Snapshot at bound = 5



Figure 3.6: Recall the bounded loop-free path of length λ

We describe the bounded semantics of FOTL_1 in two subsections, based on the existence of a loop in the behaviour of the net (in Sec. 6.3.2) and without a loop (in Sec. 6.3.1).

6.3.1 Bounded Semantics Without Loop

First, we describe the bounded semantics without loop as shown in Fig. 3.6 where $0 \leq i \leq \lambda$, where i is the current instance on the bounded path. We introduce $\models_x^k$ as an extension of the satisfaction relation $\models^k$.

1. $M, i \models^k q$ iff $q \in \nu_i$.
2. $M, i \models^k \neg q$ iff $q \notin \nu_i$.
3. $M, i \models^k (\exists x)\alpha$ iff $\exists a \in CN, a \in V_i$ and $M, [x \mapsto a], i \models_x^k \alpha$.

The above formula is satisfied when there is at least one live client such that α is satisfied in the model at instance i for the particular live client a . Given $\alpha = pc_1 \vee pc_2$ and the snapshot of the system with the bound 2 is in Fig. 6.1. Here, $CN = \{1, 2\}$. At instance $i = 0$, model M satisfies formula α . Hence, this formula holds true at instance 0.

4. $M, i \models^k (\forall x)\alpha$ iff $\forall a \in CN$, if $a \in V_i$ then $M, [x \mapsto a], i \models_x^k \alpha$.

The above formula is satisfied when the α is satisfied in the model at instance i for all live clients in the set CN . Given $\alpha = pc_4$ and Fig. 6.1. The model M does not satisfy the formula for all clients, since client 2 does not satisfy α .

5. $M, i \models^k \psi_1 \vee \psi_2$ iff $M, i \models^k \psi_1$ or $M, i \models^k \psi_2$.
6. $M, i \models^k \psi_1 \wedge \psi_2$ iff $M, i \models^k \psi_1$ and $M, i \models^k \psi_2$.
7. $M, i \models^k \mathbf{X}_s \psi$ iff
$$\begin{cases} M, i+1 \models^k \psi & \text{if } (i < \lambda) \\ M, i \models^k \mathbf{X}_s \psi & \text{otherwise} \end{cases}$$

When instance i is less than the bound λ , the formula ψ is evaluated at instance $i+1$ else, the formula is unsatisfiable.

8. $M, i \models^k \mathbf{F}_s \psi$ iff $\exists j : i \leq j \leq \lambda, M, j \models^k \psi$.

This formula is satisfiable if there is some instance j such that $i \leq j \leq \lambda$ at which the property ψ holds. Given $\psi = (\exists x)pc_4$. In Fig. 6.1, the formula is unsatisfiable. However, in Fig. 6.2, it is satisfiable at instance 5, where client 4 is in local state pc_4 .

9. $M, i \models^k \mathbf{G}_s \psi$

In the absence of a loop in the bounded run, the above formula is always unsatisfiable, since global server behaviour cannot be evaluated without a loop.

$$10. M, i \models^k \psi_1 \mathbf{U}_s \psi_2 \text{ iff } \exists j : i \leq j \leq \lambda, M, j \models^k \psi_2 \text{ and } \forall j' : i \leq j' < j : M, j' \models^k \psi_1.$$

This formula is satisfied when formula ψ_2 is satisfied at some instance j and for all instances less than j , formula ψ_1 is satisfied.

$$11. M, [x \mapsto a], i \models_x^k p(x) \text{ iff } \xi_i(a, p) = \top.$$

This formula is satisfied if the property p is satisfied by live agent a at instance i . Here, the free variable x is substituted by the client name a .

$$12. M, [x \mapsto a], i \models_x^k \neg \alpha \text{ iff } M, [x \mapsto a], i \not\models_x^k \alpha.$$

$$13. M, [x \mapsto a], i \models_x^k \alpha \vee \beta \text{ iff } M, [x \mapsto a], i \models_x^k \alpha \text{ or } M, [x \mapsto a], i \models_x^k \beta.$$

$$14. M, [x \mapsto a], i \models_x^k \alpha \wedge \beta \text{ iff } M, [x \mapsto a], i \models_x^k \alpha \text{ and } M, [x \mapsto a], i \models_x^k \beta.$$

$$15. M, [x \mapsto a], i \models_x^k \mathbf{X}_c \alpha \text{ iff } \begin{cases} M, [x \mapsto a], i + 1 \models_x^k \alpha & \text{if } (i < \lambda \text{ and } a \in V_{i+1}) \\ M, [x \mapsto a], i \models_x^k \mathbf{X}_c \alpha & \text{otherwise} \end{cases}$$

If the instance i is less than the bound λ and the client is a live client at instance $i + 1$, then the formula is evaluated at the instance $i + 1$ for the live client a ; else the original formula is not satisfied.

$$16. M, [x \mapsto a], i \models_x^k \mathbf{F}_c \alpha \text{ iff } \exists j : i \leq j \leq \lambda, a \in V_j \text{ and } M, [x \mapsto a], j \models_x^k \alpha.$$

This formula is satisfiable if there is some instance j such that $i \leq j \leq \lambda$ at which the property α is satisfied for the client a and the client a is a live client at instance j . Given $\alpha = pc_2$. In Fig. 6.1 and Fig. 6.2, the formula is satisfiable at instance 1 due to client 1.

$$17. M, [x \mapsto a], i \models_x^k \mathbf{G}_c \alpha \text{ iff } \begin{cases} M, [x \mapsto a], j \models_x^k \alpha & \text{if } (\forall j > i \text{ and } a \in V_j) \\ \forall j' : i \leq j' < j, M, [x \mapsto a], j' \models_x^k \alpha & \text{otherwise} \\ \text{where } j > i \text{ be the least instance where } a \notin V_j \end{cases}$$

For all instances j such that $j > i$ if client a is live at instance j , the formula is not satisfiable at j . Let j be the least instance where client a is not live. For all instances j' such that $i \leq j' < j$, formula α is evaluated at j' .

18. $M, [x \mapsto a], i \models_x^k \alpha \mathbf{U}_c \beta$ iff $\exists j \geq i, a \in V_j, M, [x \mapsto a], j \models_x^k \beta$, and $\forall j' : i \leq j' < j, M, [x \mapsto a], j' \models^k \alpha$.

This is satisfied when β is satisfied at some instance j and client a is a live client at j ; and for all instances less than j , client formula α is satisfied.

Notice that in the bounded semantics equations 15-18, the semantics depend crucially on whether a is a live agent. Similarly, we can describe the bounded semantics with (λ, l) - loop similar to the (k, l) - loop in [23]. As shown in Fig. 3.7, i is the current instance on the bounded path, where the formula is evaluated, λ is the bound and l is the start position of the loop, where $0 \leq i \leq \lambda$ and $0 \leq l \leq \lambda$. Consider the following lemma that relates the bounded and unbounded semantics of logic FOTL₁.

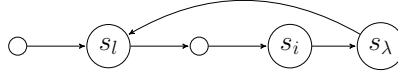


Figure 3.7: Recall the bounded path with (λ, l) - loop

6.3.2 Bounded Semantics with Loop

We now give the bounded semantics for FOTL₁ with loop as seen in Fig. 3.7.

1. $M, i \models^k q$ iff $q \in \nu_i$.
2. $M, i \models^k \neg q$ iff $q \notin \nu_i$.
3. $M, i \models^k (\exists x)\alpha$ iff $\exists a \in CN, a \in V_i$ and $M, [x \mapsto a], i \models_x^k \alpha$.
4. $M, i \models^k (\forall x)\alpha$ iff $\forall a \in CN$, if $a \in V_i$ then $M, [x \mapsto a], i \models_x^k \alpha$.
5. $M, i \models^k \psi_1 \vee \psi_2$ iff $M, i \models^k \psi_1$ or $M, i \models^k \psi_2$.
6. $M, i \models^k \psi_1 \wedge \psi_2$ iff $M, i \models^k \psi_1$ and $M, i \models^k \psi_2$.
7. $M, i \models^k \mathbf{X}_s \psi$ iff $\begin{cases} M, i+1 \models^k \psi & \text{if } (i < \lambda) \\ M, l \models^k \psi & \text{otherwise} \end{cases}$

When instance i is less than the bound λ , the formula ψ is evaluated at instance $i+1$ else, ψ is evaluated at instance l , which is the next instance of λ .

8. $M, i \models^k \mathbf{F}_s \psi$ iff $\exists j : \min(l, i) \leq j \leq \lambda, M, j \models^k \psi$.

This formula is satisfiable if there is some instance j such that $\min(l, i) \leq j \leq \lambda$, where the formula ψ is satisfied.

9. $M, i \models^k \mathbf{G}_s \psi$ iff $\forall j : \min(l, i) \leq j \leq \lambda, M, j \models^k \psi$.

This formula is satisfiable if for all instances j such that $\min(l, i) \leq j \leq \lambda$, the formula ψ is satisfied in each instance.

$$10. M, i \models^k \psi_1 \mathbf{U}_s \psi_2 \text{ iff } \left\{ \begin{array}{ll} \exists j : i \leq j \leq \lambda, M, j \models^k \psi_2 \text{ and} & \text{if } (i \leq l) \\ \forall j' : i \leq j' < j : M, j' \models^k \psi_1 & \\ \exists j : i \leq j \leq \lambda, M, j \models^k \psi_2 \text{ and} & \text{if } (i > l) \\ \forall j' : i \leq j' < j : M, j' \models^k \psi_1 & \\ \text{or} & \\ \exists j : l \leq j < i, M, j \models^k \psi_2 \text{ and} & \\ \forall j' : l \leq j' < j : M, j' \models^k \psi_1 & \end{array} \right.$$

Consider the two cases:

- If $(i \leq l)$, the current instance i is less than or equal to the loop instance l , this formula is satisfied when formula ψ_2 is satisfied at some instance j such that $i \leq j \leq \lambda$ and for all instances between i and j , formula ψ_1 is satisfied.
- If $(i > l)$, the formula may be satisfied in either of the two intervals between i to λ or between l to i . Hence, the formula is satisfied if either of the following is satisfied: formula ψ_2 is satisfied at some instance j such that $i \leq j \leq \lambda$ and for all instances less than j , formula ψ_1 is satisfied or, formula ψ_2 is satisfied at some instance j such that $l \leq j < i$ and for all instances between l and j , formula ψ_1 is satisfied.

11. $M, [x \mapsto a], i \models_x^k p(x)$ iff $\xi_i(a, p) = \top$.

12. $M, [x \mapsto a], i \models_x^k \neg \alpha$ iff $M, [x \mapsto a], i \not\models_x^k \alpha$.

13. $M, [x \mapsto a], i \models_x^k \alpha \vee \beta$ iff $M, [x \mapsto a], i \models_x^k \alpha$ or $M, [x \mapsto a], i \models_x^k \beta$.

14. $M, [x \mapsto a], i \models_x^k \alpha \wedge \beta$ iff $M, [x \mapsto a], i \models_x^k \alpha$ and $M, [x \mapsto a], i \models_x^k \beta$.

$$15. M, [x \mapsto a], i \models_x^k \mathbf{X}_c \alpha \text{ iff } \begin{cases} M, [x \mapsto a], i + 1 \models_x^k \alpha & \text{if } i < \lambda \text{ and } a \in V_{i+1} \\ M, [x \mapsto a], l \models_x^k \alpha & \text{if } i = \lambda \text{ and } a \in V_l \\ M, [x \mapsto a], i \models_x^k \mathbf{X}_c \alpha & \text{otherwise} \end{cases}$$

When current instance i is less than the bound λ , and the client a is a live agent at the next instance $i + 1$, the formula is evaluated at instance $i + 1$ else, in the current instance if the bound λ and the client is live at instance l , then the formula is evaluated at instance l , which is the next instance of λ . Otherwise, the original formula does not hold.

$$16. M, [x \mapsto a], i \models_x^k \mathbf{F}_c \alpha \text{ iff } \exists j : \min(l, i) \leq j \leq \lambda, a \in V_j, M, [x \mapsto a], j \models_x^k \alpha.$$

This formula is satisfiable if there is some instance j such that $\min(l, i) \leq j \leq \lambda$, and the client a is live at instance j , where the formula ψ is satisfied.

$$17. M, [x \mapsto a], i \models_x^k \mathbf{G}_c \alpha \text{ iff } \forall j : \min(l, i) \leq j \leq \lambda, a \in V_j, M, [x \mapsto a], j \models_x^k \alpha.$$

$$18. M, [x \mapsto a], i \models_x^k \alpha \mathbf{U}_c \beta \text{ iff } \begin{cases} \left\{ \begin{array}{l} \exists j : i \leq j \leq \lambda, a \in V_j, M, [x \mapsto a], j \models_x^k \beta \text{ and} \\ \forall j' : i \leq j' < j : M, [x \mapsto a], j' \models_x^k \alpha \end{array} \right. & \text{if } (i \leq l) \\ \left\{ \begin{array}{l} \exists j : i \leq j \leq \lambda, a \in V_j, M, [x \mapsto a], j \models_x^k \beta \text{ and} \\ \forall j' : i \leq j' < j : M, [x \mapsto a], j' \models_x^k \alpha \end{array} \right. & \text{if } (i > l) \end{cases}$$

or

$$\begin{cases} \left\{ \begin{array}{l} \exists j : l \leq j < i, a \in V_j, M, [x \mapsto a], j \models_x^k \beta \text{ and} \\ \forall j' : l \leq j' < j, M, [x \mapsto a], j' \models_x^k \alpha. \end{array} \right. \end{cases}$$

Consider the two cases:

- If $(i \leq l)$, the current instance i is less than or equal to the loop instance l , and the client a is live at instance j then this formula is satisfied when formula ψ_2 is satisfied at some instance j such that $i \leq j \leq \lambda$ and for all instances between i and j , formula ψ_1 is satisfied.
- If $(i > l)$, the formula may be satisfied in either of the two intervals between i to λ or between l to i . Hence, the formula is satisfied if either of the following is satisfied: formula ψ_2 is satisfied at some instance j such that $i \leq j \leq \lambda$ and the client a is live

at instance j and for all instances less than j , formula ψ_1 is satisfied or, formula ψ_2 is satisfied at some instance j such that $l \leq j < i$ and the client a is live at instance j and for all instances between l and j , formula ψ_1 is satisfied.

The Lemma 1, states that if a LTL formula is satisfied in a bounded run, then it is also satisfied in an unbounded run.

Lemma 1 ([23]). *Let f be a LTL formula and Π be a path then $\Pi \models^k f \implies \Pi \models f$.*

Following this, we state a similar lemma for FOTL_1 :

Lemma 2. *Let f be a FOTL_1 formula and Π be a path then $\Pi \models_x^k f \implies \Pi \models_x f$.*

Theorem 6.3.1. *FOTL_1 is decidable.*

Proof. MFOTL is decidable [74]. FOTL_1 is a syntactic fragment of MFOTL which is monodic and restricted to quantifier rank 1. □

6.4 Related Work

An *MFOTL* formula φ is **monodic** if every subformula has at most one free variable, in the scope of ψ 's. Decidability is shown by encoding models of a monodic sentence φ in structures called **quasimodels** and then expressing the statement “there exists a quasimodel satisfying a given monodic sentence” as a monadic second order sentence. In [73], they show that the monodic fragment is also *EXSPACE*-complete. In this work, we propose a one-variable fragment of *MFOTL*, called FOTL_1 for unbounded client-server systems. Additionally, we have provided the bounded semantics which led to the SMT encoding of this logic. The SMT encoding is desirable for implementing a bounded model checking tool where the properties are specified in this logic for unbounded client-server systems.

Parametric verification: In contrast to the BMC approach, there is existing literature on parametric verification such as [90], where authors consider a fixed number of identifiable processes on which safety and deadlock-freedom is verified. In [100], they work with a decidable fragment of First Order Logic, with a quantifier prefix $\exists^* \forall^*$, popularly known as the Bernays-Schönfinkel-Ramsey Class, where the temporal modalities are not considered in the same manner as in MFOTL. Their goal is to check the inductive invariants for safety and liveness properties, with expert inputs, whereas, ours focuses on automatic, push button verification of properties including safety, liveness and deadlock over unbounded ν -nets without additional user

intervention. Another key difference here, is how the fragment that we consider has only one sort, whereas, in [100], there are many sorts, some of which are bounded due to domain knowledge. In comparison with [83], where parametric verification of threshold automata where there are identifiers, our approach allows for an unbounded number of identified processes, without imposing an explicit bound on the number of processes. In [28, 83, 90, 118] the bounds are predetermined and in [100], a part of the parameters are predetermined and bounded.

Interactive Theorem Proving: While employing Interactive Theorem Provers or proof systems such as TLA^+ [36, 38, 82], user expertise and guidance during the verification process, in contrast to BMC approach which is automatic. Recently, in [70, 131], the interactive theorem prover COQ has been used for verifying the foundational Karp-Miller algorithm for computing the coverability tree. This technique has not been extended yet, to verify Petri net properties. As part of future work, we would like to build a BMC tool that takes in $FOTL_1$ specifications for unbounded client-server systems based on the SMT encoding given in this work. Building verification tools for unbounded client-server systems is a non-trivial engineering pursuit. The BMC implementation using $FOTL_1$ specifications will join the arsenal of tools such as KREACH [54], Petrinizer [57], QCOVER [26], ICOVER [63] for nets. We can also have multiple types of clients in the unbounded client-server systems, which may model a richer set of systems. Alongside this, the complexity and decidability of model checking unbounded client-server systems using $FOTL_1$ specifications is yet to be explored.

Conclusion

In this chapter, we discussed a verification tool for Petri nets with identifiers and a variant of First Order Logic. In the upcoming chapter, we consider Petri nets with nesting and look at suitable temporal properties on them and build a tool to verify them as well.

6.A Appendix: Unbounded semantics of FOTL₁

In this section, we introduce some satisfiability relations with respect to FOTL₁. Given a valid model $M = (\nu, V, \xi)$, the satisfiability relations $\models$ and $\models_x$ can be defined, via induction over the structure of the server formulae $\psi \in \Psi$, and client formulae $\alpha \in \Delta$, respectively.

1. $M, i \models q$ iff $q \in \nu_i$.

The atomic server formula $q \in P_s$ is satisfied in the model M at instant i if and only if q is a member of the local state of the server ν_i at instant i , where $i \in \mathbb{N}_0$.

2. $M, i \models \neg q$ iff $q \notin \nu_i$.
3. $M, i \models (\exists x)\alpha$ iff $\exists a \in CN, a \in V_i$ and $M, [x \mapsto a], i \models_x \alpha$.

The client formula α is satisfied at some instant i if and only if α is satisfied in any of the *live* clients in V_i i.e., for some client $a \in CN$ and the client name $a \in V_i$, where $i \in \mathbb{N}_0$.

4. $M, i \models (\forall x)\alpha$ iff $\forall a \in CN$, if $a \in V_i$ then $M, [x \mapsto a], i \models_x \alpha$.
5. $M, i \models \psi_1 \vee \psi_2$ iff $M, i \models \psi_1$ or $M, i \models \psi_2$.
6. $M, i \models \psi_1 \wedge \psi_2$ iff $M, i \models \psi_1$ and $M, i \models \psi_2$.
7. $M, i \models \mathbf{X}_s \psi$ iff $M, i + 1 \models \psi$.
8. $M, i \models \mathbf{F}_s \psi$ iff $\exists j \geq i, M, j \models \psi$.

The server formula $\mathbf{F}_s \psi$ is satisfied at some instant i if and only if there is some instant $j \geq i$, where the formula ψ holds.

9. $M, i \models \mathbf{G}_s \psi$ iff $\forall j \geq i, M, j \models \psi$.
10. $M, i \models \psi_1 \mathbf{U}_s \psi_2$ iff $\exists j \geq i, M, j \models \psi_2$ and $\forall j' : i \leq j' < j : M, j' \models \psi_1$.
11. $M, [x \mapsto a], i \models_x p(x)$ iff $\xi_i(a, p) = \top$.
12. $M, [x \mapsto a], i \models_x \neg \alpha$ iff $M, [x \mapsto a], i \not\models_x \alpha$.
13. $M, [x \mapsto a], i \models_x \alpha \vee \beta$ iff $M, [x \mapsto a], i \models_x \alpha$ or $M, [x \mapsto a], i \models_x \beta$.

14. $M, [x \mapsto a], i \models_x \alpha \wedge \beta$ iff $M, [x \mapsto a], i \models_x \alpha$ and $M, [x \mapsto a], i \models_x \beta$.

15. $M, [x \mapsto a], i \models_x \mathbf{X}_c \alpha$ iff $M, [x \mapsto a], i + 1 \models_x \alpha$ and $a \in V_{i+1}$.

16. $M, [x \mapsto a], i \models_x \mathbf{F}_c \alpha$ iff $\exists j \geq i, a \in V_j, M, [x \mapsto a], j \models_x \alpha$.

The client formula $\mathbf{F}_c \alpha$ is satisfied at some instant i if and only if there is some instant $j \geq i$, where the formula α holds in any of the *live* clients in V_j .

17. $M, [x \mapsto a], i \models_x \mathbf{G}_c \alpha$ iff $\forall j \geq i, a \in V_j, M, [x \mapsto a], j \models_x \alpha$.

18. $M, [x \mapsto a], i \models_x \alpha \mathbf{U}_c \beta$ iff $\exists j \geq i, M, [x \mapsto a], j \models_x \beta$ and $\forall j' : i \leq j' < j : M, j' \models \alpha$ and $a \in V_j$.

Chapter 7

Verification of Elementary Object Systems

7.1 Introduction

Imperfections are natural in the context of message passing systems: imperfect communication channels may spontaneously lose, duplicate, or shuffle carried messages or even deliver new unwanted ones. When compared to their perfect counterpart, verification of imperfect systems is usually easier. For example, reachability in Communicating Finite Automata (CFA) over *perfect* (FIFO) channels is undecidable [29], but it is decidable when the channels are *lossy* [35] (messages may be non-deterministically lost and never delivered) or *unordered* [7] (the message sending and reception order may be different). The same principle generally holds even outside CFA. Specifically, two counter machines can encode Turing Machines [52, 97] and suffer from undecidable reachability. Instead, Mayr [92, 93] showed that if lossiness is applied to the counters (the natural numbers stored in the counters may non-deterministically decrease), then reachability becomes decidable; nevertheless, several other problems remain undecidable. Similar results are also available for Petri Nets (PNs): Bouajjani and Mayr [27] studied the impact of lossiness on the model checking problem of Vector Addition Systems with States (VASS, equivalent to Petri Nets) against fragments of the Modal μ -Calculus. In this case, while the EF (and EG) fragment of the UB language (which includes negation and labeled variants of the CTL next operator) is undecidable for VASS [56], it is decidable for lossy VASS.

Imperfections may be naturally interpreted as perturbations of the system configurations and, thus, verification of imperfect systems can be studied under the lens of robustness [16, 81], i.e., checking whether a property holds if at most k perturbations occur, for some given $k \leq |\mathbb{N}|$. Recently, Köhler-Bussmeier and Capra [81] put forward the nets-within-nets paradigm [128] to

naturally specify robustness properties in a Multi Agent System (MAS) context. Specifically, in the PN setting, *black tokens* that mark a fixed set of places are moved around by a fixed set of transitions. In contrast, in nets-within-nets, the tokens could additionally be PN objects themselves. Thus, even without further expert information on the net design, these *object tokens* naturally model agents, which might be affected by perturbations. In [81], perturbations follow a drastic *let it crash* approach, causing agent break-downs. Technically, this is achieved by enforcing a deadlock in the object token. However, nets-within-nets with less disruptive perturbations may still be suitable to model perturbed MAS, where the agents may suffer imperfections even without completely breaking down.

We focus on Elementary Object Systems (EOSs) [80], which are a simple nets-within-nets model, yet featuring most of the important ingredients. Moreover, they can be generalized to more sophisticated models, such as full-fledged Object Systems [128]. This makes EOSs an excellent candidate for our study. Our key contributions are:

1. We formally define three forms of lossiness in the EOS setting, corresponding to the nesting levels of the tokens.
2. We provide examples illustrating the relevance of these lossiness relations and formalize lossy reachability/coverability problems on them.
3. We completely chart the decidability status of these problems (see Tab. 7.1).

Standard reachability/coverability problems have been studied in [80], but only in a perfect setting, i.e., without lossiness. Preliminary results on EOS robustness were put forward in [81]. However, they do not address the problem of perturbations on reachability/coverability in a systematic way. To the best of our knowledge, ours is the first work that attempts a full formal classification of EOS reachability/coverability with lossy perturbations. As discussed above, the concept of lossiness is well established in the PN literature; the most relevant works are [27, 93, 119]. However, their approach uniformly interleaves each standard step with lossiness and, thus, does not properly address robustness, where also the number of lossy events is of interest. Moreover, when compared to PNs, EOSs offer more room for lossiness because of the nesting of tokens. This may significantly complicate the picture. Instead, we consider the full spectrum of lossiness, from one occurrence to infinitely many, at all nesting levels.

7.2 Preliminaries

7.2.1 Binary Relations

Let us fix some notation for binary relations. Given a set X , the *identity relation* id_X on X is the relation $\{(x, x) \in X^2 \mid x \in X\}$. Given a binary relation $<$ on X , we denote its *reflexive closure* $< \cup id_X$ by $\leq$ and its *anti-reflexive part* $< \setminus id_X$ by $\prec$. We use the symbol $>$ to denote the relation such that, $x > y$ iff $y < x$. For example, if $<$ is transitive, then $\leq$ and $\geq$ are transitive and reflexive (i.e., quasi orders). The same applies to the symbols $\prec$ and $\succ$ and their closures. From now on, we use $<$ to denote arbitrary transitive relations, and $<$ (possibly with a subscript) to represent fixed transitive relations, e.g., the standard order of $\mathbb{N}$.

7.2.2 Multisets

A *multiset* $\mathbf{m}$ on a set D is a mapping $\mathbf{m} : D \rightarrow \mathbb{N}$. The *support* of $\mathbf{m}$ is the set $\text{Support}(\mathbf{m}) = \{i \mid \mathbf{m}(i) > 0\}$. The multiset $\mathbf{m}$ is finite if its $\text{Support}(\mathbf{m})$ is finite. The family of all multisets over D is denoted by $D^\oplus$. We denote a finite multiset $\mathbf{m}$ by enumerating the elements $d \in \text{Support}(\mathbf{m})$ exactly $\mathbf{m}(d)$ times in between $\{\{$ and $\}\}$, where the ordering is irrelevant. For example, the finite multiset $\mathbf{m} : \{p, q\} \rightarrow \mathbb{N}$ such that $\mathbf{m}(p) = 1$ and $\mathbf{m}(q) = 2$ is denoted by $\{\{p, q, q\}\}$. The empty multiset $\{\{\}\}$ (with empty support) is also denoted by $\emptyset$. On the empty domain $D = \emptyset$ the only defined multiset is $\emptyset$; to stress this out we denote this special case, i.e., the empty multiset over the empty domain, by ε . Given two multisets $\mathbf{m}_1$ and $\mathbf{m}_2$ on D , we define $\mathbf{m}_1 + \mathbf{m}_2$ and $\mathbf{m}_1 - \mathbf{m}_2$ on D as follows: $(\mathbf{m}_1 + \mathbf{m}_2)(d) = \mathbf{m}_1(d) + \mathbf{m}_2(d)$ and $(\mathbf{m}_1 - \mathbf{m}_2)(d) = \max(\mathbf{m}_1(d) - \mathbf{m}_2(d), 0)$. Similarly, for a finite set I of indices, $\sum_{i \in I} \{\{d_i\}\}$ denotes the multiset $\mathbf{m}$ over $\bigcup_{i \in I} \{d_i\}$ such that $\mathbf{m}(d) = |\{i \in I \mid d_i = d\}|$ for each $d \in D$. With a slight abuse of notation, we omit the double brackets, i.e., $\sum_{i \in I} \{\{d_i\}\} = \sum_{i \in I} d_i$. If $I = \{1, \dots, n\}$, then $\sum_{i \in I} d_i = \sum_{i=1}^n d_i$. Finally, we write $\mathbf{m}_1 \sqsubseteq \mathbf{m}_2$ if, for each $d \in D$, we have $\mathbf{m}_1(d) \leq \mathbf{m}_2(d)$.

7.2.3 Petri Nets

We recall the standard definition for PN from Definition. 2.1.1 and pre_N and post_N Definition. 2.1.2. In this chapter, we use the marking according to the multiset notation. For example, the marking μ that places one token on place p and two on place q is denoted by $\{\{p, q, q\}\}$. The

empty marking is denoted by $\emptyset$. We also work with the special *empty PN* $\blacksquare = (\emptyset, \emptyset, \emptyset)$, whose only marking is ε . A transition $t \in T$ is *enabled* on a marking μ (finite multiset of places) if, for each place $p \in P$, we have $\text{pre}_N(t)(p) \leq \mu(p)$. Its firing results in the marking μ' such that $\mu'(p) = \mu(p) - \text{pre}_N(t)(p) + \text{post}_N(t)(p)$, for each $p \in P$.

7.2.4 Elementary Object Systems

An EOS [80] is, intuitively, a PN (called system net) whose tokens carry an internal PN (called object net), taken from a finite set $\mathcal{N}$. Each place can host only one fixed type of internal PN. The EOS fires events, which synchronize a transition τ in the system net and multisets $\theta(N)$ of transitions in the object nets N consumed by τ . EOSs are formally defined in Definition. 2.2.6.

Since the nets in $\mathcal{N}$ are disjoint, we can denote each event $\langle \hat{\tau}, \theta \rangle$, as a pair $\langle \hat{\tau}, M \rangle$ for a multiset M over $\bigcup_{(P,T,F) \in \mathcal{N}} T$ such that $M(t) = \theta(N)(t)$ where $N = (P, T, F) \in \mathcal{N}$ and $t \in T$. EOS tokens are nested, i.e., each token at a system place p carries a PN marking μ for the object net $d(p)$. EOS markings, also called nested markings, are multisets of nested tokens. With a slight abuse of notation, we denote markings omitting double curly brackets from multiset notation.

Definition 1 (Nested Markings). Let $\mathfrak{E} = \langle \hat{N}, \mathcal{N}, d, \Theta \rangle$ be an EOS. The set of *nested tokens* $\mathcal{T}(\mathfrak{E})$ of $\mathfrak{E}$ is the set $\bigcup_{(P,T,F) \in \mathcal{N}} (d^{-1}(P, T, F) \times P^\oplus)$. The set of *nested markings* $\mathcal{M}(\mathfrak{E})$ of $\mathfrak{E}$ is $\mathcal{T}(\mathfrak{E})^\oplus$. Given $\lambda, \rho \in \mathcal{M}(\mathfrak{E})$, we say that λ is a *sub-marking* of μ if $\lambda \sqsubseteq \mu$.

Note that λ is a sub-marking of μ iff there is some nested marking μ' such that $\mu = \lambda + \mu'$. EOSs inherit the graphical representation of PNs with the provision that we represent nested tokens via a dashed line from the system net place to an instance of the object net where the internal marking is represented in the standard PN way. However, if the nested token is $\langle p, \varepsilon \rangle$ for a system net place p of type $\blacksquare$, we represent it with a black-token $\blacksquare$ on p . If a place p hosts $n > 2$ black-tokens, then we represent them by writing n on p . Each event $\langle \hat{\tau}, \theta \rangle$ is depicted by labeling $\hat{\tau}$ by $\langle \theta \rangle$ (possibly omitting double curly brackets). If there are several events involving $\hat{\tau}$, then $\hat{\tau}$ has several labels. Fig. 2.11 shows the client server system with unbounded number of clients represented as an EOS.

Example 5. Fig. 7.1 depicts the system net $\hat{N}$ (the idle transitions are omitted) and object net drone of an EOS $\mathfrak{E} = \langle \hat{N}, \mathcal{N}, d, \Theta \rangle$ modeling a drone that (1) moves between a base and a field, (2) has two batteries, (3) consumes one charge-unit per battery per movement, and (4) charges

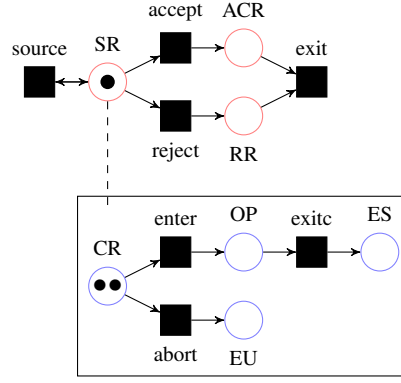


Figure 2.11: EOS modeling the single server (unbounded) multiple client system

its batteries by multiples of two charge-units when at base. Technically, $\mathcal{N} = \{\text{drone}, \blacksquare\}$ (even if $\blacksquare$ is unused), $d(\text{base}) = d(\text{field}) = \text{drone}$, and Θ synchronizes `takeOff` and `land` (respectively `charge`) in $\hat{N}$ with `move` (`charge1` and `charge2`) in `drone`. Formally, $\Theta = \{\langle \text{takeOff}, \{\{\text{move}\}\} \rangle, \langle \text{land}, \{\{\text{move}\}\} \rangle, \langle \text{charge}, \{\{\text{charge1}\}\} \rangle, \langle \text{charge}, \{\{\text{charge2}\}\} \rangle\}$. The marking $\mu = \langle \text{drone}, \{\{\text{batt1}, \text{batt1}\}\} \rangle$ represents a single partially charged drone at base, with two charge units in the first battery.

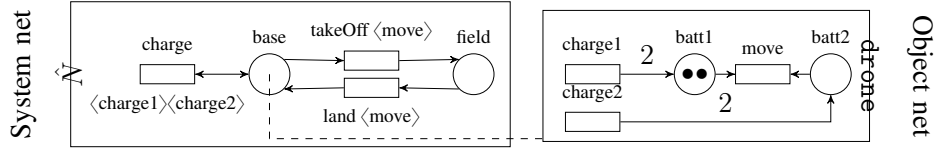


Figure 7.1: EOS in Example 5 with marking $\{\langle \text{drone}, \{\{\text{batt1}, \text{batt1}\}\} \rangle\}$. The idle transitions are omitted.

When firing an event $\langle \tau, \theta \rangle$, nested tokens in the system net are consumed according to the preconditions of τ in the standard PN way. At the same time, for each object net N , the inner tokens are merged so as to obtain a PN marking $\mu(N)$ for N (possibly empty). Then, transitions in $\theta(N)$ are fired in the standard PN way obtaining markings $\mu'(N)$. Next, nested markings with empty inner markings are produced in the system net according to the postconditions of τ . Finally, the markings $\mu'(N)$ are non-deterministically distributed among the empty nested tokens, according to the typing function. To be fired, the event must be enabled at both the system and at the object net level. This is captured by the enabledness condition, which makes use of projection operators at the system (Π^1) and at the object net level (Π_N^2 for each $N \in \mathcal{N}$).

Definition 2 (Projection Operators). Let $\mathfrak{E}$ be an EOS $\langle \hat{N}, \mathcal{N}, d, \Theta \rangle$. The *projection operators* Π^1 maps each nested marking $\mu = \sum_{i \in I} \langle \hat{p}_i, M_i \rangle$ for $\mathfrak{E}$ to the PN marking $\sum_{i \in I} \hat{p}_i$ for $\hat{N}$. Given

an object net $N \in \mathcal{N}$, the *projection operators* Π_N^2 maps each nested marking $\mu = \sum_{i \in I} \langle \hat{p}_i, M_i \rangle$ for $\mathbf{E}$ to the PN marking $\sum_{j \in J} M_j$ for N where $J = \{i \in I \mid d(\hat{p}_i) = N\}$.

To define the enabledness condition, we need the following notation. We set $\text{pre}_N(\theta(N)) = \sum_{i \in I} \text{pre}_N(t_i)$ where $(t_i)_{i \in I}$ is an enumeration of $\theta(N)$ counting multiplicities. We analogously set $\text{post}_N(\theta(N)) = \sum_{i \in I} \text{post}_N(t_i)$. The EOS enabledness condition is already defined in Definition. 2.2.9.

Example 6. In the setting of the EOS $\mathfrak{E}$ and marking μ in Ex. 5 (Fig. 7.1), the event $\langle \text{charge}, \{\{\text{charge1}\}\} \rangle$ is enabled on $\mu = \langle \text{base}, \{\{\text{batt1}, \text{batt1}\}\} \rangle$ with mode (λ, ρ) where $\lambda = \mu$ and $\rho = \langle \text{base}, \{\{\text{batt1}, \text{batt1}, \text{batt1}, \text{batt1}\}\} \rangle$. Since $\lambda = \mu$, its firing results in the step $\mu \xrightarrow{\langle e, \lambda, \rho \rangle} \rho$. Instead, the event $\langle \text{charge}, \{\{\text{charge2}\}\} \rangle$ is enabled on μ with mode (λ, ρ') where $\rho' = \langle \text{base}, \{\{\text{batt1}, \text{batt1}, \text{batt2}, \text{batt2}\}\} \rangle$. Its firing results in the step $\mu \xrightarrow{\langle e, \lambda, \rho' \rangle} \rho'$. These are the only enabling modes for $\langle \text{charge}, \{\{\text{charge1}\}\} \rangle$ and $\langle \text{charge}, \{\{\text{charge2}\}\} \rangle$ on μ . No other event is enabled on μ , irrespective of the mode.

The reachability problem for EOSs is defined in the usual way, i.e., whether there is a run (sequence of event firings) from an initial marking μ_0 to a target marking μ_f . Also coverability definition is standard, but with respect to the order $\leq_f$ (denoted by $\leq$ in [80]; see Def. 4 below) that allows one to add both (1) tokens in the inner markings of available nested tokens (2) or nested tokens with some internal marking on the system net places. It is known that both these problems are undecidable (Th. 4.3 in [80]). However, coverability is decidable on the fragment of *conservative EOSs* (cEOSs; Th. 5.2 in [80]). Nevertheless, reachability remains undecidable (Th. 5.5 in [80]).

In cEOSs, for each system net transition t , if t consumes a nested token on a place of type N , then it also produces at least one token on a place of the same type N . We give the formal definition of cEOS below:

Definition 3 (cEOS). A cEOS is an EOS $\mathfrak{E} = \langle \hat{N}, \mathcal{N}, d, \Theta \rangle$ with $\hat{N} = \langle \hat{P}, \hat{T}, \hat{F} \rangle$ where, for all $\hat{t} \in \hat{T}$ and $\hat{p} \in \text{Support}(\text{pre}_{\hat{N}}(\hat{t}))$, there exists $\hat{p}' \in \text{Support}(\text{post}_{\hat{N}}(\hat{t}))$ such that $d(\hat{p}) = d(\hat{p}')$.

We use conservativity to obtain the decidability results in this chapter.

7.3 Problem

We study reachability and coverability of EOSs (cEOSs) affected by several forms of lossiness. First, we define three lossiness relations and show their relevance. Second, we formally define the problem we study in the following sections.

7.3.1 Lossy EOSs

We study EOSs (cEOSs) affected by lossiness, where nested markings may non-deterministically lose their tokens according to a quasi order, called lossiness relation. Lossiness can occur (1) at the object level, if lossiness removes only tokens from the inner markings of nested tokens, (2) at the system level, if lossiness removes whole nested tokens only, and (3) at both levels (the full EOS), if both whole nested tokens and/or regular tokens from the remaining nested tokens are removed. These levels are captured, respectively, by the lossiness quasi orders $\leq_o$ (object-lossiness), $\leq_s$ (system-lossiness), and $\leq_f$ (full-lossiness) as defined next.

Definition 4. Given an EOS $\mathfrak{E}$ and two nested markings μ and μ' for $\mathfrak{E}$, we have (1) $\mu \leq_s \mu'$ if $\mu \sqsubseteq \mu'$ or, equivalently, there is some μ'' such that $\mu' = \mu + \mu''$, (2) $\mu \leq_o \mu'$ if we can write $\mu = \sum_{i \in I} \langle \hat{p}_i, M_i \rangle$ and $\mu' = \sum_{i \in I} \langle \hat{p}_i, M'_i \rangle$ and, for each $i \in I$, $M_i \sqsubseteq M'_i$, and (3) $\mu \leq_f \mu'$ if there is some nested marking μ'' such that $\mu \leq_o \mu'' \leq_s \mu'$.

Example 7. Consider the marking $\mu = \langle \text{base}, \{\{\text{batt1}, \text{batt1}\}\} \rangle$ in Ex. 5. By removing 1 or 2 charge units we obtain the markings $\mu_1 = \langle \text{base}, \{\{\text{batt1}\}\} \rangle$ and $\mu_2 = \langle \text{base}, \emptyset \rangle$. By adding to μ a discharged token at place field, we obtain the $\mu' = \langle \text{base}, \{\{\text{batt1}, \text{batt1}\}\} \rangle + \langle \text{field}, \emptyset \rangle$. By removing the drone from μ , we obtain $\mu'' = \emptyset$. We have, among the others, $\mu' \geq_s \mu$, $\mu \geq_o \mu_1 \geq_o \mu_2$, $\mu' \geq_f \mu$, $\mu' \geq_f \mu_2$, and $\mu' \geq_f \mu''$.

The relation $\leq_f$ coincides with $\leq$ in [80]. Moreover, the order of $\leq_o$ and $\leq_s$ is irrelevant in $\leq_f$ definition (Rem. 1 below). An EOS (cEOS) suffering from object-, system-, or full-lossiness is called, respectively, object-, system-, or full-lossy EOS (cEOS) or, simply lossy EOS (cEOS).

These lossiness relations are relevant to model non-deterministic phenomena not directly captured by the EOS. For example, in the context of Ex. 5, object-lossiness results in the non-deterministic loss of tokens at places batt1 and batt2, which models the partial/total discharge of drone batteries because of non-modeled drone movements within the base or the field, or because of other unexpected phenomena. In a slightly more complex EOS, with intermediate

places capturing the flight from base to field and vice-versa, object-lossiness captures also the non-deterministic usage of extra charge-units because of contingencies like strong winds. Instead, system lossiness results in the loss of nested tokens modeling drones. This captures the loss of drones because of, e.g., break-downs, wrong flight paths, or seizure from higher priority processes (assuming the EOS is a module in a more complex system). Full-lossiness capture both aspects. Similar interpretations can be given each time the (nested) tokens represent resources, like charge-units or drones in Ex. 5. These scenarios are common in the literature (see, e.g., water- and fire-units in [128] and raw-resources in production plants in [34]).

Lossy EOSs are relevant also to capture partial/total internal break-downs. This happens, e.g., when the tokens model resource containers instead of resources themselves. For example, after modifying the drone object net into the object net drone2 in Fig. 7.2,¹ each regular token represents a battery with bounded capacity. Its charge level is captured by its position in the object net. Consequently, object-lossiness represents the break-down of internal components, in this case the battery. The loss of all batteries results in drone deadlock (cf. [81]). This case is analogous to the application of system-lossiness discussed above, since drones can be seen as internal components of a higher level process (captured by the whole EOS): the loss/break-down of all drones results in system deadlock. More in general, this interpretation applies when the EOS uses conservative system and/or object nets. Also these scenario are common in the literature (see, e.g., the finite control of robots [80], the internal state of fire-fighters in [128], and customers and cars in [89]).

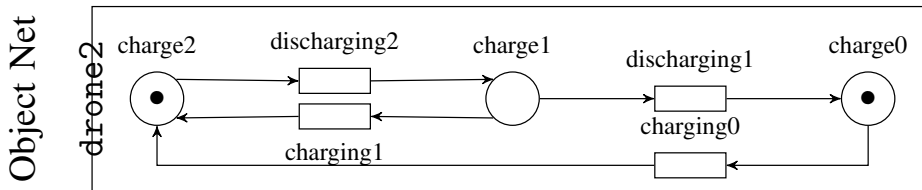


Figure 7.2: Object net drone2 with one fully charged 2-bounded battery and a fully discharged one.

¹Also the events have to be modified accordingly, i.e., for each $n \in \{0, 1, 2\}$, by synchronizing `takeOff` and `landing` with `dischargen` object net transitions, and `charge` with the `chargen` object net transitions.

7.3.2 Lossy-reachability/coverability

We study the problem of ℓ -reachability/coverability, i.e., whether a target nested marking can be reached/covered from an initial one via a run suffering at most ℓ lossy steps, where $\ell \in \mathbb{N} \cup \{\omega\}$.² These problems are relevant to study EOS robustness in front of losses/break-downs.

Definition 5. Given a transition system $TS = (V, \rightarrow)$, a transitive relation $<$ on V , and an $\ell \in \mathbb{N} \cup \{\omega\}$, a $(<, \ell)$ -run in TS is a run whose steps are labeled either by $\rightarrow$, called *standard steps*, or by $>$, called *<-lossy* or *lossy steps*, and at most ℓ steps are lossy. The set of $(<, \ell)$ -runs from μ_0 is denoted by $Runs_\ell(<, \mu_0)$. A $(<, \ell)$ -run is called ℓ' -strong if it contains exactly ℓ' lossy steps.

The definition also applies to reflexive or anti-reflexive transitive relations, i.e., we can also talk about $(\leq, \ell)$ -runs and $(\preceq, \ell)$ -runs. We denote a labeled step from μ to μ' by $\mu \rightsquigarrow \mu'$. To stress that the step is labeled by $\rightarrow$ or by $<$, we denote it by $\mu \rightarrow \mu'$ or by $\mu > \mu'$, respectively. Whenever we have a lossy run σ from μ to μ' , we write $\mu \rightsquigarrow^\sigma \mu'$. The $(<, \ell)$ -reachability/coverability problems ask whether a target can be reached/covered under $<$ from an initial configuration with at most ℓ $<$ -lossy steps.

Definition 6 ($(<, \ell)$ -reachability/coverability for EOSs (cEOSs)). Let $\ell \in \mathbb{N} \cup \{\omega\}$.

Input: An EOS (cEOS) E , an initial marking μ_0 and a target marking μ_1 for E .

Output of reachability: Is there a run $\sigma \in Runs_\ell(<, \mu_0)$ such that $\mu_0 \rightsquigarrow^\sigma \mu_1$?

Output of coverability: Is there a run $\sigma \in Runs_\ell(<, \mu_0)$ such that $\mu_0 \rightsquigarrow^\sigma \mu \geq \mu_1$ for some μ ?

We call these problems *lossy-problems*. A *<-lossy-problem* is a lossy-problem under $<$. The *degree* of a $(<, \ell)$ -reachability/coverability problem is ℓ . If $\ell = 0$ we obtain standard reachability/coverability, i.e., over perfect runs. Our objective is to fully chart the decidability status of the lossy-problems for $\leq_f$, $\leq_s$, and $\leq_o$. Previous results for EOSs are available only for $\ell = 0$ and the relation $\leq_f$. Consequently, they do not inform us on the status of the other (proper) lossy-problems, whose study still requires a careful and in-depth analysis.

7.4 Coincident Problems

The well known notion of compatibility from WSTS has a strong impact on ℓ -reachability problems. In fact, we now show that all these problems, for $\ell \geq 1$ (including ω) and any quasi

²Recall that ω is the first limit ordinal, whose cardinality is $|\mathbb{N}|$, i.e., the same as $\mathbb{N}$.

order $\leq$, collapse to $(\leq, 0)$ -coverability if and only if the lossiness relation is compatible.

Lemma 3. *Each yes-instance of $(\leq, \ell)$ -reachability is also a yes instance of $(\leq, \ell)$ -coverability. [45]*

Proof. Immediate consequence of reflexivity of the quasi order $\leq$. $\square$

Lemma 4. *Each yes-instance of $(\leq, \ell)$ -coverability is also a yes instance of $(\leq, \ell + 1)$ -reachability, if ℓ is finite, and a yes-instance of $(\leq, \omega)$ -reachability, if $\ell = \omega$. [45]*

Proof. If μ_1 is coverable from μ_0 , then there is a $(\leq, \ell)$ -run σ from μ_0 to μ and $\mu \geq \mu_1$. Take the run σ' as the run σ followed by the lossy step $\mu \geq \mu_1$. Thus, σ' reaches μ_1 from μ_0 . Moreover, if ℓ is finite, then σ' is a $(\leq, \ell + 1)$ -run and, otherwise, σ' is a $(\leq, \omega)$ -run. $\square$

Corollary 1. *$(\leq, \omega)$ -reachability and $(\leq, \omega)$ -coverability coincide. [45]*

Note that Cor. 1 is consistent with other lossy PN models (see, e.g., [27]).

Lemma 5. *Each yes-instance of $(\leq, \ell)$ -reachability or $(\leq, \ell)$ -coverability is also a yes instance of $(\leq, \omega)$ -reachability or $(\leq, \omega)$ -coverability, respectively. [45]*

Proof. Immediate consequence of the fact that each $(\leq, \ell)$ -run is also a $(\leq, \omega)$ -run. $\square$

Summarising, for each quasi order $\leq$, the $\leq$ -lossy-problems form a hierarchy ordered according to inclusion of the yes-instance sets. For each $i \in \mathbb{N}$, the i -th hierarchy level for $\leq$ is the $(\leq, i/2)$ -reachability problem, if i is even, and the $(\leq, (i - 1)/2)$ -coverability problem, if i is odd. We say that the hierarchy *collapses* if all the $(\leq, \ell)$ -reachability and $(\leq, \ell)$ -coverability problems with $\ell \geq 1$ coincide with (standard) $(\leq, 0)$ -coverability or, equivalently, if the yes-instances of $(\leq, \omega)$ -reachability are also yes-instances of $(\leq, 0)$ -coverability. The next lemma states that this latter property is equivalent to compatibility, that is, if $\mu_1 \geq \mu_2 \rightarrow \mu_3$, then there is a μ_4 such that $\mu_1 \rightarrow^* \mu_4 \geq \mu_3$.

Lemma 6. *$\leq$ is compatible iff each yes-instance of $(\leq, \omega)$ -reachability is also a yes-instance of $(\leq, 0)$ -coverability. [45]*

Proof. Assume that $\leq$ is compatible. If v_1 is reachable from v_0 via an ω -run σ , then, without loss of generality, we can assume that σ is finite and, thus, it is a ℓ -run for some finite ℓ . By compatibility, we can push, one by one, the finitely many lossy steps in σ at the end of the run, obtaining an ℓ -run σ' (possibly with different length) where all lossy steps occur at the end, i.e.,

σ' is of the form $v_0 \rightarrow^* w_1 \geq w_2 \cdots \geq w_\ell \geq v_1$. By transitivity of $\geq$, there is also a run σ'' of the form $v_0 \rightarrow^* w_1 \geq v_1$, which witness that v_1 is $\leq$ -coverable from v_0 . Vice-versa, assume that each yes-instance of $(\leq, \omega)$ -reachability is also a yes-instance of $(\leq, 0)$ -coverability. If $v_0 \geq v_1 \rightarrow v_2$, then v_2 is $(\leq, 1)$ -reachable from v_0 , as well as $(\leq, \omega)$ -reachable. Thus, v_2 is also coverable from v_0 , i.e., $v_0 \rightarrow^* v_1 \geq v_2$. $\square$

Corollary 2. *The hierarchy of lossy-problems induced by $\leq$ collapses iff $\leq$ is compatible. [45]*

Note that Cor. 2 can be generalized to other lossy models, since its proof does not take advantage of the technical details of lossy EOSs, but relies only on a compatible quasi order. This fact has some immediate consequence on the lossy-problems we are studying. In fact, it is known that $\leq_f$ is strong compatible on cEOSs, that is, if $\mu_1 \geq_f \mu_2 \xrightarrow{\langle e, \lambda, \rho \rangle} \mu_3$, then there is some μ_4 such that $\mu_1 \xrightarrow{\langle e, \lambda, \rho \rangle} \mu_4$ (Lemma 5.1 in [80]). Thus, the hierarchy for full-lossy cEOSs collapses. Note that $\leq_f$ is not compatible over EOSs. This helps to prove the undecidability of reachability and coverability over them (cf. Th.4.3 in [80]). Since $(\leq_f, 0)$ -coverability for cEOSs is decidable (Th. 5.2 in [80]), we obtain the following theorem.

Theorem 1. *For $\ell \geq 1$, $(\leq_f, \ell)$ -reachability and $(\leq_f, \ell)$ -coverability for cEOSs are decidable. [45]*

Instead $\leq_s$ is compatible for both EOSs and cEOSs, as shown next.

Lemma 7. *If (λ, ρ) enables the event e on μ and $\mu' \geq_s \mu$, then (λ, ρ) enables the event e also on μ' . [45]*

Proof. If (λ, ρ) enables the event e on μ , then the enabledness formula $\Phi(e, \lambda, \rho)$ holds and $\lambda \leq_s \mu$. Since $\mu \leq_s \mu'$, by transitivity of $\leq_s$, we have that $\lambda \leq_s \mu'$. Thus, (λ, ρ) enables e on μ' . $\square$

Lemma 8. *$\leq_s$ is strong compatible on EOSs. [45]*

Proof. If $\mu_1 \geq_s \mu_2 \xrightarrow{\langle e, \lambda, \rho \rangle} \mu_3$, then $\lambda \leq_s \mu_2$, $\mu_3 = \mu_2 - \lambda + \rho$, and there is some $\Delta(\mu_2)$ such that $\mu_1 = \mu_2 + \Delta(\mu_2)$. Moreover, since (λ, ρ) enables e on μ_2 , then, by Lemma 7, (λ, ρ) enables e on μ_1 . Thus, there is a μ_4 such that $\mu_1 \xrightarrow{\langle e, \lambda, \rho \rangle} \mu_4$. Moreover, by EOS semantics and the fact that $\lambda \leq_s \mu_2$, we have that $\mu_4 = \mu_1 - \lambda + \rho = \mu_2 + \Delta(\mu_2) - \lambda + \rho \geq_s \mu_2 - \lambda + \rho = \mu_3$. $\square$

Theorem 2. *The hierarchies for system-lossy EOSs and system-lossy cEOSs collapse. [45]*

Thus, the study of system-lossiness on EOSs and cEOSs boils down to $(\leq_s, 0)$ -coverability for EOSs and cEOSs (we study them in Th. 6 below). Finally, we show that $\leq_o$ is compatible on cEOSs. The following preliminary remarks can be easily proved.

Remark 1. $\leq_f = \leq_o \circ \leq_s = \leq_s \circ \leq_o$.

Remark 2. $\Pi^1(\mu_1) + \Pi^1(\mu_2) = \Pi^1(\mu_1 + \mu_2)$. If $\mu_1 \leq_o \mu_2$, then $\Pi^1(\mu_1) = \Pi^1(\mu_2)$.

Lemma 9. $\leq_o$ is strong compatible on cEOSs. [45]

Proof. The proof is depicted in Fig. 7.3. If $\mu_1 \geq_o \mu_2 \xrightarrow{\langle e, \lambda, \rho \rangle} \mu_3$, then, since $\leq_o \subseteq \leq_f$ and $\leq_f$ is strong compatible on cEOSs (Th. 5.1 in [80]), there is some μ_4 such that $\mu_1 \xrightarrow{\langle e, \lambda, \rho \rangle} \mu_4 \geq_f \mu_3$. Since $\leq_f = \leq_s \circ \leq_o$ (Rem. 1), there is also some μ'_4 such that $\mu_3 \leq_s \mu'_4 \leq_o \mu_4$. Thus, there is some $\Delta(\mu_3)$ such that $\mu'_4 = \mu_3 + \Delta(\mu_3)$ and, by Rem. 2, we have $\Pi^1(\mu'_4) = \Pi^1(\mu_4)$. Note that, by EOS semantics, μ_1 and $\mu_2 + \Delta(\mu_3)$ have respectively the predecessors $\mu_4 = \mu_1 - \lambda + \rho$ and $\mu'_4 = \mu_2 + \Delta(\mu_3) - \lambda + \rho$. By some simple algebra,³ $\Pi^1(\mu_1) = \Pi^1(\mu_2 + \Delta(\mu_3))$. However, again by Rem. 2, since $\mu_2 \leq_o \mu_1$, we have $\Pi^1(\mu_1) = \Pi^1(\mu_2)$ and, summarising, $\Pi^1(\mu_2) + \Pi^1(\Delta(\mu_3)) = \Pi^1(\mu_2 + \Delta(\mu_3)) = \Pi^1(\mu_1) = \Pi^1(\mu_2)$. Thus, $\Pi^1(\Delta(\mu_3)) = \emptyset$. Consequently, $\Delta(\mu_3) = \emptyset$ and $\mu_4 \geq_o \mu'_4 = \mu_3 + \Delta(\mu_3) = \mu_3$.

□

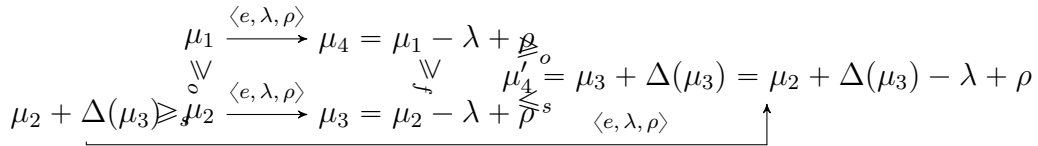


Figure 7.3: Depiction of proof of Th. 9.

Theorem 3. The hierarchy for object-lossy cEOSs collapses. [45]

Thus, the study of object-lossiness on cEOSs boils down to $(\leq_o, 0)$ -coverability (we study this problem in Th. 5). Summarising, compatibility considerably simplifies the landscape of lossy problems for lossy cEOSs and for system-lossy EOSs. Specifically, for cEOSs, the only relevant problems are only the status of $(\leq_o, 0)$ -coverability and of $(\leq_s, 0)$ -coverability. Similarly, for system-lossy EOSs, the only relevant question is the status of $(\leq_s, 0)$ -coverability.

³ $\Pi^1(\mu_1) + \Pi^1(\lambda) + \Pi^1(\rho) = \Pi^1(\mu_1 - \lambda + \rho) = \Pi^1(\mu_4) = \Pi^1(\mu'_4) = \Pi^1(\mu_2 + \Delta(\mu_3) - \lambda + \rho) = \Pi^1(\mu_2 + \Delta(\mu_3)) - \Pi^1(\lambda) + \Pi^1(\rho)$.

7.5 Distinct Problems

Unfortunately, compatibility does not hold for $\leq_o$ and $\leq_f$ on EOSs. This is the main fact allowing the simulation of inhibitory nets via EOSs in [80]. Thus, the hierarchies induced by $\leq_o$ and $\leq_f$ do not collapse. In fact, we now show that all the problems in the hierarchy are distinct. We make use of a gadget with a dedicated place that counts the lossy steps.

Definition 7. The *lossiness-counter gadget* $\mathcal{G}$ is the EOS $(\hat{N}, \mathcal{N}, d, \Theta)$ depicted in Fig. 7.4 where

1. $\hat{N} = (\hat{P}, \hat{T}, \hat{F})$ where $\hat{P} = \{p_1, p_2, count\}$, $\hat{T} = \{\tau_1, \tau_2\}$, $\hat{F}(x) = 1$ if $x \in \{(p_1, \tau_1), (\tau_1, count), (\tau_1, p_2), (p_2, \tau_2), (\tau_2, count), (\tau_2, p_1)\}$ and $\hat{F}(x) = 0$ otherwise,
2. $\mathcal{N} = \{N_1, N_2\}$ where $N_1 = (\{p\}, \{inc_1\}, F_1)$, $F_1(\{(inc_1, p)\}) = 1$, $F_1(\{(p, inc_1)\})$, $N_2 = (\{q\}, \{inc_2\}, F_2)$, $F_2(\{(inc_2, q)\}) = 1$, and $F_2(\{(q, inc_2)\})$,
3. $d(p_1) = N_1$, $d(p_2) = N_2$, and $d(count) = \blacksquare$, and
4. $\Theta = \{\langle \tau_1, inc_2 \rangle, \langle \tau_2, inc_1 \rangle\}$.

In what follows, we work with a fixed initial nested marking $\mu_0 = \langle p_1, \{\{p\}\} \rangle$ of $\mathcal{G}$.

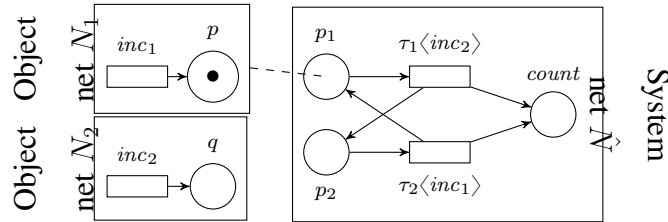


Figure 7.4: The lossiness-counter gadget $\mathcal{G}$ in Def. 7 (where $d(p_1) = N_1$, $d(p_2) = N_2$, $d(count) = \blacksquare$) with initial marking $\mu_0 = \{\{\langle p_1, \{\{p\}\}\rangle\}\}$.

7.5.1 Distinct Problems for Object-lossy EOSs

We first study the case of object-lossiness.

Lemma 10. *Let $<$ be a transitive relation. Given $\ell \in \mathbb{N} \cup \{\omega\}$, we have that μ is $(\leq, \ell)$ -reachable from μ_0 in $\mathcal{G}$ iff μ is $(\preceq, \ell)$ -reachable from μ_0 . [45]*

Proof. Each $\preceq$ -lossy step $\mu_1 \succcurlyeq \mu_2$ can be interpreted as a $\leq$ -lossy step $\mu_1 \geq \mu_2$. Thus, each $(\preceq, \ell)$ -run can be interpreted as a $(\leq, \ell)$ -run. Similarly, each $(\leq, \ell)$ -run that does not contain

reflexive lossy steps of the form $\mu \geq \mu$ can be interpreted as a $(\preceq, \ell)$ -run. Moreover, each maximal finite or infinite sub-run $\mu \geq \mu \geq \dots \mu \geq \dots$ of an arbitrary $(\preceq, \ell)$ -run σ can be substituted by a single occurrence of μ , obtaining a $(\preceq, \ell)$ -run without reflexive lossy steps. $\square$

Lemma 11. *For each $\ell \geq 0$, the lossiness-counter gadget $\mathcal{G}$ exhibits a single maximal ℓ -strong $(\preceq_o, \ell)$ -run from its initial nested-marking μ_0 . This run has the form*

$$\mu_0 \succeq_o \mu'_0 \rightarrow \mu_1 \succeq_o \mu'_1 \rightarrow \mu_2 \succeq_o \dots \mu_{\ell-1} \succeq_o \mu'_{\ell-1} \rightarrow \mu_\ell$$

where, for each $i \leq \ell$, we have $\mu_i = \langle x_i, \{\{y_i\}\} \rangle + i \langle \text{count}, \varepsilon \rangle$ and $\mu'_i = \langle x_{i+1}, \emptyset \rangle + i \langle \text{count}, \varepsilon \rangle$ where, for each $j \in \mathbb{N}$, $x_j = p_1$ and $y_j = p$ if j is even and $x_j = p_2$ and $y_j = q$ if j is odd. [45]

Proof. By induction on ℓ . If $\ell = 0$, then, since μ_0 is a deadlock for $\rightarrow$, the run μ_0 of length 0 is the only $(\preceq_o, \ell)$ -run. Moreover, 0 is even and $\mu_0 = \langle p_1, \{\{p\}\} \rangle + 0 \langle \text{count}, \varepsilon \rangle$.

If the inductive hypothesis holds for an arbitrary even ℓ , then $\mu_\ell = \langle p_1, \{\{p\}\} \rangle + \ell \langle \text{count}, \varepsilon \rangle$ is a deadlock for $\rightarrow$. The only way to continue the run is by one $\succeq_o$ step. However, the only token that can be lost under $\leq_o$ is the token inside $\langle p_1, \{\{p\}\} \rangle$, thus, the only $\leq_o$ -successor of μ_ℓ is $\mu'_\ell = \langle p_1, \emptyset \rangle + \ell \langle \text{count}, \varepsilon \rangle$. On μ'_ℓ there is no available $\succeq_o$ step and the only enabled event is (τ_1, inc_2) with mode (λ, ρ) where $\lambda = \langle p_1, \emptyset \rangle$ and $\rho = \langle p_2, \{\{q\}\} \rangle + \langle \text{count}, \varepsilon \rangle$. Its firing reaches $\mu_{\ell+1} = \langle p_2, \{\{q\}\} \rangle + (\ell + 1) \langle \text{count}, \varepsilon \rangle$. This configuration is a deadlock for $\rightarrow$ and the so obtained $(\preceq_o, \ell + 1)$ -run from μ_0 to $\mu_{\ell+1}$ already contains $\ell + 1$ $\preceq_o$ -steps. Thus, this run is maximal among the $(\preceq_o, \ell + 1)$ -runs.

If the inductive hypothesis holds for an arbitrary odd ℓ , the same argument applies with the provision that p_1 and p_2 , p and q , as well as (τ_1, inc_2) and (τ_2, inc_1) , have to be swapped. $\square$

Corollary 3. *For each finite ℓ , the set of nested markings which are $(\leq_o, \ell)$ -reachable from μ_0 is $\{\mu_i \mid i \in \{0, \dots, \ell\}\} \cup \{\mu'_i \mid i \in \{0, \dots, \ell-1\}\}$ where μ_i and μ'_i are defined as in Lemma 11. [45]*

Consequently, using the same notation as in Lem. 11, for each $\ell \in \mathbb{N}$, we have that μ'_ℓ is $(\leq_o, \ell)$ -coverable but not $(\leq_o, \ell)$ -reachable and $\mu_{\ell+1}$ is $(\leq_o, \ell + 1)$ -reachable but not $(\leq_o, \ell)$ -coverable. Thus, the sequence of yes-instance sets of $(\leq_o, \ell)$ -reachability/coverability problems (recall Lem. 3 and Lem. 4) is a sequence of proper subsets. Consequently, for each finite ℓ , all $(\leq_o, \ell)$ -reachability/coverability problems are pairwise distinct. Moreover, while $(\leq_o, \omega)$ -reachability coincides with $(\leq_o, \omega)$ -coverability by Lem. 6, also $(\leq_o, \omega)$ -reachability is distinct from $(\leq_o, \ell)$ -reachability/coverability for each ℓ .⁴

⁴Otherwise, it would coincide also with $(\leq_o, \ell + 1)$ -reachability/coverability. Thus, $(\leq_o, \ell)$ -reachability/coverability and $(\leq_o, \ell + 1)$ -reachability/coverability would coincide, which is a contradiction.

Corollary 4. *For each $\ell_1 < \ell_2 \leq \omega$, we have that $(\leq_o, \ell_1)$ -reachability, $(\leq_o, \ell_1)$ -coverability, $(\leq_o, \ell_2)$ -reachability, and $(\leq_o, \ell_2)$ -coverability are pair-wise distinct problems. [45]*

7.5.2 Distinct Problems for Full-lossy EOSs

A fact analogous to Cor. 4 applies also to $\leq_f$. In fact, even if $\mathcal{G}$ exhibits more complex maximal $(\leq_f, \ell)$ -runs, we still need ℓ lossy steps to put ℓ tokens on *count*. We first show that if a marking is reachable/coverable using $\leq_f$ -lossy steps, then it is also covered under $\leq_s$ by some marking reachable via a run as in Lem. 11.

Lemma 12. *If $\sigma \in \text{Runs}_\ell(\leq_f, \mu_0)$ and $\mu_0 \rightsquigarrow^\sigma \mu$, then there is some $\lambda \geq_s \mu$ and there is an ℓ' -strong run $\sigma' \in \text{Runs}_{\ell'}(\leq_o, \mu_0)$ such that $\mu_0 \rightsquigarrow^{\sigma'} \lambda$, for some $\ell' \leq \ell$. [45]*

Proof. Since $\leq_f = \leq_o \circ \leq_s$, we can expand each $\leq_f$ -lossy step in two subsequent $\leq_o$ - and $\leq_s$ -lossy steps. By compatibility of $\leq_s$ on EOSs (Lem. 9) and the fact that $\leq_s \circ \leq_o = \leq_o \circ \leq_s$ (Rem. 1), we can push all the $\leq_s$ -lossy steps at the end of the run, obtaining a run $\sigma' \in \text{Runs}_\ell(\leq_o, \mu_0)$ such that $\mu_0 \rightsquigarrow^{\sigma'} \lambda \geq_s \dots \geq_s \mu$, for some marking λ . By transitivity of $\leq_s$, we have $\lambda \geq_s \mu$. Moreover, we can drop the $\leq_o$ -lossy steps in σ' of the form $\pi \geq_o \pi$ for some π , obtaining an ℓ' -strong run $\sigma'' \in \text{Runs}_{\ell'}(\leq_o, \mu_0)$ for some $\ell' \leq \ell$. $\square$

We now show that also the sequence of yes-instance sets of $\leq_f$ -lossy reachability/coverability problems (recall Lem. 3 and Lem. 4) is a sequence of proper subsets. We do that by using the same markings μ_ℓ and μ'_ℓ defined in Lem. 11.

Lemma 13. *Using the notation in Lem. 11, for each finite $\ell \in \mathbb{N}$, we have that $\mu_{\ell+1}$ is $(\leq_f, \ell+1)$ -reachable, but not $(\leq_f, \ell)$ -coverable. [45]*

Proof. By Lem. 11 we know that $\mu_{\ell+1}$ is $(\leq_o, \ell+1)$ -reachable and, thus, also $(\leq_f, \ell+1)$ -reachable. We now show that $\mu_{\ell+1}$ is not $(\leq_f, \ell)$ -coverable. Assume by contradiction that $\mu_{\ell+1}$ is $(\leq_f, \ell)$ -coverable. Then, there is some run $\sigma \in \text{Runs}_\ell(\leq_f, \mu_0)$ and a nested marking λ such that $\mu_0 \rightsquigarrow^\sigma \lambda \geq_f \mu_{\ell+1}$. Since $\mu_{\ell+1} \geq_s (\ell+1)\langle \text{count}, \varepsilon \rangle$, also $\lambda \geq_f (\ell+1)\langle \text{count}, \varepsilon \rangle$. Thus, $\lambda \geq_s (\ell+1)\langle \text{count}, \varepsilon \rangle$.⁵ By applying Lem. 12 on the run σ , there is some nested marking $\lambda' \geq_s \lambda$ and, for some $\ell' \leq \ell$, a ℓ' -strong run $\sigma' \in \text{Runs}_{\ell'}(\leq_o, \mu_0)$ such that $\mu_0 \rightsquigarrow^{\sigma'} \lambda'$. Thus, by

⁵In fact, $\lambda \geq_f (\ell+1)\langle \text{count}, \varepsilon \rangle$ implies that there is some π such that $\lambda \geq_s \pi \geq_o (\ell+1)\langle \text{count}, \varepsilon \rangle$; since the latter marking only places black tokens at the system net level, π is obtained by adding zero tokens at the object net level, i.e., $\pi = (\ell+1)\langle \text{count}, \varepsilon \rangle$. Hence $\lambda \geq_s (\ell+1)\langle \text{count}, \varepsilon \rangle$.

Lem. 11, we have that either $\lambda' = \mu_{\ell'}$ or $\lambda' = \mu'_{\ell'-1}$ and, hence, λ' places at most ℓ' black tokens on *count*. However, $\lambda' \geq_s \lambda \geq_s \ell + 1 \langle \text{count}, \varepsilon \rangle$, i.e., λ' puts at least $\ell + 1$ tokens on *count* even if $\ell' \leq \ell < \ell + 1$, which is a contradiction. $\square$

Lemma 14. *Using the notation in Lem. 11, for each finite $\ell \in \mathbb{N}$, we have that $\mu'_{\ell+1}$ is $(\leq_f, \ell)$ -coverable, but not $(\leq_f, \ell)$ -reachable. [45]*

Proof. By Lem. 11 we know that $\mu'_{\ell+1}$ is $(\leq_o, \ell)$ -coverable and, thus, also $(\leq_f, \ell)$ -coverable. We now show that $\mu'_{\ell+1}$ is not $(\leq_f, \ell)$ -reachable. Assume by contradiction that $\mu'_{\ell+1}$ is $(\leq_f, \ell)$ -reachable. Then, there is some run $\sigma \in \text{Runs}_\ell(\leq_f, \mu_0)$ such that $\mu_0 \rightsquigarrow^\sigma \mu'_{\ell+1}$. By applying Lem. 12 on the run σ , there is some nested marking $\lambda \geq_s \mu'_{\ell+1}$ and, for some $\ell' \leq \ell$, a ℓ' -strong run $\sigma' \in \text{Runs}_{\ell'}(\leq_o, \mu_0)$ such that $\mu_0 \rightsquigarrow^{\sigma'} \lambda$. Since $\lambda \geq_s \mu'_{\ell+1} \geq_s (\ell + 1) \langle \text{count}, \varepsilon \rangle$, we have that λ puts at least $\ell + 1$ tokens on *count*. However, by Lem. 11, we have that either $\lambda = \mu_{\ell'}$ or $\lambda = \mu'_{\ell'-1}$. Hence, λ puts at most $\ell' \leq \ell < \ell + 1$ tokens on *count*, which is a contradiction. $\square$

Corollary 5. *For each $\ell_1 < \ell_2 \leq \omega$, we have that $(\leq_f, \ell_1)$ -reachability, $(\leq_f, \ell_1)$ -coverability, $(\leq_f, \ell_2)$ -reachability, and $(\leq_f, \ell_2)$ -coverability are pair-wise distinct problems. [45]*

7.6 Undecidability for object- and system-lossy cEOSs

We now study the decidability status of $(\leq_o, 0)$ -coverability and $(\leq_s, 0)$ -coverability for cEOSs. In [79], reachability for cEOSs is proved undecidable via a reduction from reachability of 2CMs. We provide a variant of that reduction that reduces reachability of 2CMs to $\leq_o$ -coverability and to $\leq_s$ -coverability of cEOSs. This proves that both coverability problems are undecidable. By Th. 3 and Th. 2, all ℓ -reachability problems for object-lossy and system-lossy cEOSs are, thus, undecidable. Since cEOSs are a special case of EOSs, undecidability applies also to object-lossy and system-lossy EOSs.

7.6.1 Reduction to Reachability

We show a reduction from reachability of a target configuration (q_f, n_1, n_2) of any 2CM $\mathcal{K} = (Q, \delta, q_0)$ with increment, decrement, and zero-check instructions to reachability of a cEOS $\mathbf{E}_{\mathcal{K}}$.

Definition 8. Given a 2CM $\mathcal{K} = (Q, \delta, q_0)$, we define the EOS $\mathbf{E}_{\mathcal{K}} = (\hat{N}, \mathcal{N}, d, \Theta)$ where

1. $\hat{N} = (\hat{P}, \hat{T}, \hat{F})$ is such that:

- $\hat{P}$ contains Q , a place g , a place $next_i$ for each instruction $i \in \delta$, and two places c_j and c'_j for each counter cnt_j ;
 - $\hat{T}$ contains two transitions t_i^1 and t_i^2 for each instruction $i \in \delta$;
 - For each increment (resp., decrement, zero-check) instruction $i \in \delta$, $\hat{F}$ captures the pre- and post-conditions depicted in Fig. 7.5a (Fig. 7.5b, Fig. 7.5c).
2. $\mathcal{N}$ contains the net $\blacksquare$ and a single net $N = (P, T, F)$ where $P = \{p\}$, $T = \{inc_N, dec_N\}$, $F(inc_N, p) = F(p, dec_N) = 1$, and $F(p, inc_N) = F(dec_N, p) = 0$.
 3. $d(c_1) = d(c_2) = d(g) = N$ and $d(x) = \blacksquare$ for each other place $x \in \hat{P} \setminus \{c_1, c_2, g\}$.
 4. The synchronization structure Θ contains the events
 - $e_i^1 = (t_i^1, \{\{inc_N\}\})$ and $e_i^2 = (t_i^2, \emptyset)$ for each increment instruction i .
 - $e_i^1 = (t_i^1, \{\{dec_N\}\})$ and $e_i^2 = (t_i^2, \emptyset)$ for each decrement instruction i .
 - $e_i^1 = (t_i^1, \emptyset)$ and $e_i^2 = (t_i^2, \emptyset)$ for each zero-check instruction i .

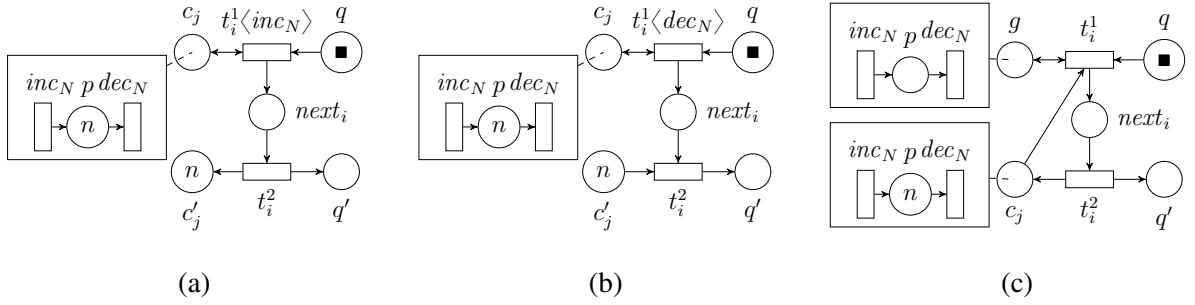


Figure 7.5: Part of $\mathbf{E}_{\mathcal{K}}$ capturing an (a) increment, (b) decrement, or (c) zero-check instruction $i \in \delta$.

Note that $\mathbf{E}_{\mathcal{K}}$ is a cEOS. It weakly simulates the increment and decrement instructions of $\mathcal{K}$ and performs zero-guesses in place of zero-check instructions. These guesses may be wrong but leave behind them two irreversible evidences: tokens in the internal marking of the object net at g and non-matching numbers of tokens on the place c_j and in the object net on c'_j , for some $j \in \{0, 1\}$. The former can be detected by $(\leq_s, 0)$ -coverability; the latter can be detected by $(\leq_o, 0)$ -coverability. We make these notions precise.

Definition 9. We say that a nested marking μ for $\mathbf{E}_{\mathcal{K}}$ is *legal* if it places exactly one object-token N at c_0 , c_1 , and g , exactly one $\blacksquare$ on exactly one place $q \in Q$, and no token on any place in $\hat{P} \setminus \{c'_0, c'_1, q\}$. If μ is legal, we denote

1. by $[\mu]'_j$ the number of black tokens placed at c'_j ,

2. by $[\mu]_j$ the number of black-tokens in the object-token at c_j ,
3. by $[\mu]_g$ the number of black-tokens in the object-token at g , and
4. by $[\mu]_Q$ the place $q \in Q$ marked by μ with a black-token.

Moreover, we say that μ :

1. is *broken at (counter) j* if $[\mu]_j \neq [\mu]'_j$, *sub-broken at j* if $[\mu]_j < [\mu]'_j$, *broken at g* if $[\mu]_g \neq 0$,
2. is *broken* if it is broken at 0, at 1, or at g ,
3. *encodes* the 2CM configuration $c = (q, n_0, n_1)$, denoted by $\mu = \langle c \rangle$, if μ is non-broken and $c = ([\mu]_Q, [\mu]_0, [\mu]_1)$.

The following lemma is a direct consequence of the pre- and post-conditions of the transitions in E_K , the shape of its synchronization structure Θ , the EOS semantics, and Def. 8. Its proof consists in simple, yet space-consuming algebraic checks and, thus, it is omitted.

Lemma 15. *Let $i \in \{(q, +, j, q'), (q, -, j, q'), (q, =, j, q')\} \cap \delta$ and μ a legal nested marking for E_K . If e_i^1 is enabled on μ , there is some μ' and μ'' such that $\mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$ and*

1. $[\mu]_Q = q$, μ'' is legal, $[\mu'']_Q = q'$, $[\mu'']_{1-j} = [\mu]_{1-j}$, and $[\mu'']'_{1-j} = [\mu]'_{1-j}$.
2. if $i = (q, +, j, q')$, then $[\mu'']_j = [\mu]_j + 1$, $[\mu'']'_j = [\mu]'_j + 1$ and $[\mu'']_g = [\mu]_g$.
3. if $i = (q, -, j, q')$, then $[\mu'']_j = [\mu]_j - 1$, $[\mu'']'_j = [\mu]'_j - 1$ and $[\mu'']_g = [\mu]_g$.
4. if $i = (q, =, j, q')$, then $[\mu'']_j = 0$, $[\mu'']'_j = [\mu]'_j$, and $[\mu'']_g = [\mu]_g + [\mu]_j$. [45]

Corollary 6. *Let $\mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$ and μ legal. If μ is sub-broken at 0 or at 1, then μ'' is sub-broken at 0 or at 1, respectively. If μ is broken at g , then μ'' is broken at g . [45]*

Corollary 7. *Let $\mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$, and μ legal and non-broken at 0 or at 1. If μ'' is broken at 0 or at 1, respectively, then μ'' is sub-broken at 0 or at 1. [45]*

Corollary 8. *Let $\mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$, μ legal, and $i \in \{(q, +, j, q'), (q, -, j, q')\} \cap \delta$. If μ is non-broken, then μ'' is non-broken. [45]*

Cor. 8 indicates that the simulation of increment or decrement instructions cannot lead, on their own, to broken markings. We now show that this is not the case for some simulation of zero-checks, called *wrong zero-guesses*.

Definition 10. A *wrong zero-guess* on counter j is a run $\mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$ where i is a zero-check instruction on counter j , μ is legal, and $[\mu]_j > 0$.

Lemma 16. *If $\sigma : \mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$ and μ is legal and non-broken, then μ'' is broken at j for some $j \in \{0, 1\}$ iff σ is a wrong zero-guess on counter j . [45]*

Proof. By Cor. 8, since μ is not broken at j but μ'' is, i is a zero-check instruction on counter j . If $[\mu]_j = [\mu']_j = 0$, then $[\mu'']_j = 0 = [\mu']_j = [\mu'']_j$ and μ'' is not broken at j , contradiction. Thus, $[\mu]_j = [\mu']_j > 0$ and, thus, σ is a wrong zero-guess. Vice-versa, if σ is a wrong-zero guess on counter j , then, by Lemma 15, μ'' is broken at j . $\square$

The next lemma is proved analogously.

Lemma 17. *If $\sigma : \mu \xrightarrow{e_i^1} \mu' \xrightarrow{e_i^2} \mu''$ and μ is legal and non-broken, then μ is broken at g iff σ is a wrong zero-guess. [45]*

Corollary 9. *If $\sigma : \langle (q_0, 0, 0) \rangle \rightarrow^* \mu$ is a run of even length, then the following are equivalent: (1) μ is sub-broken at 0 or 1, (2) μ is broken at g , (3) μ is broken, and (4) σ has a wrong zero-guess. [45]*

Clearly, for each run $c_0 \xrightarrow{i_0} c_1 \rightarrow i_1 \dots$ in $\mathcal{K}$ there is a run $\langle c_0 \rangle \xrightarrow{e_{i_0}^1} \mu_0 \xrightarrow{e_{i_0}^2} \langle c_1 \rangle \xrightarrow{e_{i_1}^1} \mu_1 \xrightarrow{e_{i_1}^2} \dots$. If the target configuration (q_f, n_0, n_1) is reachable from $(q_0, 0, 0)$ in $\mathcal{K}$, then $\mu_f = \langle (q_f, n_0, n_1) \rangle$ is reachable from $\mu_0 = \langle (q_0, 0, 0) \rangle$ in $\mathbf{E}_{\mathcal{K}}$. Vice-versa, if μ_f is reachable from μ_0 in $\mathbf{E}_{\mathcal{K}}$ via a run σ , then, μ_f is legal, non-broken, $[\mu_f]_Q = q_f$, and σ has even length. Thus, σ does not have any wrong zero-guess and can be simulated by a corresponding run in $\mathcal{K}$ from $(q_0, 0, 0)$.

Theorem 4. *(q_f, n_0, n_1) is reachable from $(q_0, 0, 0)$ in $\mathcal{K}$ iff $\mu_f = \langle (q_f, n_0, n_1) \rangle$ is reachable from $\mu_0 = \langle (q_0, 0, 0) \rangle$ in $\mathbf{E}_{\mathcal{K}}$. [45]*

Note that the EOS in Def. 8 is a cEOS. Thus, since 2CM reachability is undecidable, our result confirms that reachability for cEOSs is undecidable. However, we can conclude more.

7.6.2 From Reachability to $(\leq_o, 0)$ -coverability

We now show that our construction yields undecidability also for $(\leq_o, 0)$ -coverability. This is based on the fact that the nested marking μ_f in Th. 4 is reachable if and only if it is $(\leq_o, 0)$ -coverable. In fact, if $\mu_0 \rightarrow^* \mu \geq_o \mu_f$, then, for each $j \in \{0, 1\}$,

1. μ and μ_f mark in the same way all system net places of type $\blacksquare$.

2. $[\mu]_Q$ is well-defined, $[\mu]_Q = [\mu_f]_Q = q_f$, μ is reachable only via runs of even length, and μ is legal.
3. for each $j \in \{0, 1\}$, $[\mu]'_j = [\mu_f]'_j = [\mu_f]_j \leq [\mu]_j$;
4. if μ is broken at j , then by Cor. 6 and Cor. 7 it is sub-broken at j and, thus, $[\mu]_j < [\mu]'_j \leq [\mu]_j$, contradiction; thus, μ is not broken at j and is not broken at 0 and, consequently, not broken at g .
5. $[\mu]_j = [\mu]'_j = [\mu_f]'_j = [\mu_f]_j$ and $[\mu]_g = 0 = [\mu_f]_g$.

Summarising μ and μ_f coincide. Thus, μ_f is $(\leq_o, 0)$ -coverable from μ_0 iff it is reachable. By Th. 4, we obtain the following result.

Theorem 5. $(\leq_o, 0)$ -coverability for cEOSs is undecidable. [45]

By Th. 3, by undecidability of reachability for cEOSs (Th.5.5 in [80]), and by the fact that each cEOS is also an EOS, we obtain the following result.

Corollary 10. For each $\ell \in \mathbb{N} \cup \{\omega\}$, we have that $(\leq_o, \ell)$ -reachability and $(\leq_o, \ell)$ -coverability for cEOSs and for EOSs are undecidable. [45]

7.6.3 From Reachability to $(\leq_s, 0)$ -coverability

We now show that similar statements apply also for $(\leq_s, 0)$ -coverability. In fact, if $\mu_0 \rightarrow^* \mu \geq_s \mu_f$, then, for each $j \in \{0, 1\}$,

1. μ places at least one black-token on $[\mu_f]_Q = q_f$, thus μ is reachable only via runs of even length, and μ is legal.
2. possibly with the exception of c'_0 and c'_1 , μ and μ_f coincide on all system net places, including all object-tokens on them.
3. $[\mu]_j = [\mu_f]_j = [\mu_f]'_j \leq [\mu]'_j$ and $[\mu]_g = [\mu_f]_g$, thus μ is not broken and $[\mu]'_j = [\mu]_j = [\mu_f]_j$.

Summarising μ and μ_f coincide. Thus, μ_f is $(\leq_s, 0)$ -coverable from μ_0 iff it is reachable. By Th. 4, we obtain the following result.

Theorem 6. $(\leq_s, 0)$ -coverability for cEOSs is undecidable. [45]

By Th. 2, by undecidability of reachability for cEOSs (Th.5.5 in [80]), and the fact that each cEOS is also an EOS, we obtain the following corollaries.

Corollary 11. For each $\ell \in \mathbb{N} \cup \{\omega\}$, we have that $(\leq_s, \ell)$ -reachability and $(\leq_s, \ell)$ -coverability for cEOSs and for EOSs are undecidable. [45]

7.7 Undecidability for full-lossy EOSs

We now show that all $(\leq_f, \ell)$ -reachability problems for EOSs with finite $\ell \geq 1$ are undecidable. This is achieved via a reduction of reachability of 2CMs to $(\leq_f, \ell)$ -reachability with any given ℓ .

Fix the value of $\ell \geq 1$. Given an arbitrary 2CM $\mathcal{K} = (Q, \delta, q_0)$, a target configuration (q_f, n_1, n_2) of $\mathcal{K}$, and its simulating EOS $\mathbf{E}_{\mathcal{K}} = (\hat{N}_{\mathcal{K}}, \mathcal{N}_{\mathcal{K}}, d_{\mathcal{K}}, \Theta_{\mathcal{K}})$ as in Sec. 7.6, we merge $\mathbf{E}_{\mathcal{K}}$ with the lossiness-counter gadget $\mathcal{G} = (\hat{N}_{\mathcal{G}}, \mathcal{N}_{\mathcal{G}}, d_{\mathcal{G}}, \Theta_{\mathcal{G}})$ from Sec. 7.4. Specifically, for $\hat{N}_{\mathcal{K}} = (\hat{P}_{\mathcal{K}}, \hat{T}_{\mathcal{K}}, \hat{F}_{\mathcal{K}})$ and $\hat{N}_{\mathcal{G}} = (\hat{P}_{\mathcal{G}}, \hat{T}_{\mathcal{G}}, \hat{F}_{\mathcal{G}})$, we start with the EOS $\mathcal{M} = (\hat{M}, \mathcal{N}_{\mathcal{K}} \cup \mathcal{N}_{\mathcal{G}}, d_{\mathcal{K}} \cup d_{\mathcal{G}}, \Theta_{\mathcal{K}} \cup \Theta_{\mathcal{G}})$ where $\hat{M} = (\hat{P}_{\mathcal{K}} \cup \hat{P}_{\mathcal{G}}, \hat{T}_{\mathcal{K}} \cup \hat{T}_{\mathcal{G}}, \hat{F}_{\mathcal{K}} \cup \hat{F}_{\mathcal{G}})$.

We now add to $\mathcal{M}$ a system net transition *enabling* together with a dedicated event $e = \langle \text{enabling}, \emptyset \rangle$ (see Fig. 7.6). This transition consumes (1) ℓ tokens from *count*, (2) one from p_1 if ℓ is even, (3) one from p_2 if ℓ is odd. The transition *enabling* produces (1) one in q_0 , and (2) one object-token with empty internal marking in each of g , c_0 , and c_1 . We call the so obtained EOS $\mathcal{F}$. The initial marking μ_0 of $\mathcal{F}$ is $\langle p_1, \{\{p\}\} \rangle$, as for $\mathcal{G}$.

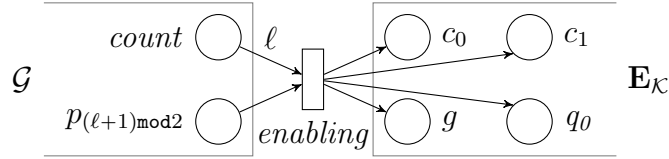


Figure 7.6: The places, transitions, and conditions we add on top of $\mathcal{M}$.

Note that, along the runs of $\mathcal{F}$, the event $e = \langle \text{enabling}, \emptyset \rangle$ fires at most once. Before firing e , there is no token on $\mathbf{E}_{\mathcal{K}}$ while, after firing e , there is no token on $\mathcal{G}$. Let σ be a $(\leq_f, \ell)$ -run of $\mathcal{F}$. If e is not fired along σ , then σ does not reach the target marking $\mu_f = \langle (q_f, n_1, n_2) \rangle$ (which puts some token on $\mathbf{E}_{\mathcal{K}}$). Otherwise, σ can be split into two runs σ_1 and σ_2 , such that $\sigma = \sigma_1 \xrightarrow{e} \sigma_2$. Since e consumes ℓ tokens from *count* and one from either p_1 or p_2 in $\mathcal{G}$, the last marking μ_1 in σ_1 has to put at least ℓ tokens on *count* and at least one token on either p_1 or p_2 , respectively.

If σ_1 is not ℓ -strong, then it is ℓ' -strong for some $\ell' < \ell$. By Lem. 12, there is some ℓ'' -strong run $\sigma \in \text{Runs}_{\ell''}(\leq_o, \mu_0)$ for some $\ell'' \leq \ell'$ such that $\mu_0 \rightsquigarrow^{\sigma_1} \lambda \geq_s \mu_1$ for some marking λ . Thus, by Lem. 11, λ puts on *count* at most $\ell'' < \ell$ tokens. Since $\lambda \geq_s \mu_1$, so does μ_1 , which is a contradiction. Thus, σ_1 is ℓ -strong.

Consequently, since σ is a $(\leq_f, \ell)$ -run, σ_2 must be a (perfect) $(\leq_f, 0)$ -run of $\mathbf{E}_{\mathcal{K}}$. Also,

because of the post-conditions of *enabling*, the first marking in σ_2 is $\langle(q_0, 0, 0)\rangle$. Thus, the 2CM $\mathcal{K}$ reaches the target (q_f, n_1, n_2) from $(q_0, 0, 0)$ iff $\mathbf{E}_{\mathcal{K}}$ has a (perfect) $(\leq_f, 0)$ -run from $\langle(q_0, 0, 0)\rangle$ to $\mu_f = \langle(q_f, n_1, n_2)\rangle$ iff $\mathcal{F}$ exhibits an ℓ -strong $(\leq_f, \ell)$ -run from μ_0 to μ_f iff $\mathcal{F}$ exhibits $(\leq_f, \ell)$ -run from μ_0 to μ_f . Since reachability of 2CM is undecidable, we get the next theorem.

Theorem 7. *For each finite $\ell \in \mathbb{N}$, $(\leq_f, \ell)$ -reachability for EOSs is undecidable. [45]*

One can adapt this construction so as to concatenate the lossy-counter gadget $\mathcal{G}$ with any EOS $\mathbf{E}$ (in place of $\mathbf{E}_{\mathcal{K}}$ for any 2CM $\mathcal{K}$). If the initial marking of $\mathbf{E}$ contains several nested tokens, a chain of enabling events like e may be necessary to initialize it. As above, in order to fire them, the $(\leq_f, \ell)$ -runs of the concatenated EOS $\mathcal{F}$ have to preliminary fire all their lossy steps. Moreover, after firing e , the intended initial marking of $\mathbf{E}$ is initialized and the $(\leq_f, \ell)$ -runs of $\mathcal{F}$ can continue only by simulating $\mathbf{E}$ without any further lossy step. Thus, $\mathbf{E}$ reaches/covers a target marking μ_f iff $\mathcal{F}$ $(\leq_f, \ell)$ -reaches/covers the same target μ_f from $\mu_0 = \langle p_1, \{\{p\}\}\rangle$. Since (perfect) $(\leq_f, 0)$ -reachability is known to be undecidable for EOSs (Th. 4.3 in [80]), one get again Th. 7. Moreover, also (perfect) $(\leq_f, 0)$ -coverability is known to be undecidable (again, Th. 4.3 in [80]). Thus, we get undecidability also for $(\leq_f, \ell)$ -coverability for each finite ℓ .

Theorem 8. *For each finite $\ell \in \mathbb{N}$, $(\leq_f, \ell)$ -coverability for EOSs is undecidable. [45]*

7.8 Decidability of $(\leq_f, \omega)$ -reachability for EOSs

We now show that $(\leq_f, \omega)$ -reachability is decidable for EOSs. We use a modified semantics for full-lossy EOSs which merges standard and lossy steps.

Definition 11. A *merged-EOS* (mEOS) is an EOS interpreted under the semantics induced by the step relation $\leadsto$ where $\leadsto = \geq_f \cup \rightarrow$.

Since EOSs and mEOSs are syntactically the same, given an EOS $\mathbf{E}$, we denote by $\mathbf{E}_S$ the EOS $\mathbf{E}$ interpreted under the standard $\rightarrow$ step relation (S stands for standard), and by $\mathbf{E}_M$ the EOS $\mathbf{E}$ interpreted under the $\leadsto$ step relation (M stands for merged). Similarly, we denote the set of predecessors of μ in $\mathbf{E}_S$ by $Pred_S(\mu)$ and in $\mathbf{E}_M$ by $Pred_M(\mu)$. The benefit of mEOSs is that $\leq_f$ becomes trivially compatible. This solves the major source of undecidability behind the undecidability result of $\leq_f$ -reachability for EOSs.

Lemma 18. $\leq_f$ is compatible for mEOSs. [45]

Proof. By definition of $\leadsto$, if $\mu_1 \geq_f \mu_2 \leadsto \mu_3$, then either (1) $\mu_1 \geq_f \mu_2 \geq_f \mu_3$ and, by transitivity of $\geq_f$, also $\mu_1 \leadsto^* \mu_1 \geq_f \mu_3$, or (2) $\mu_1 \geq_f \mu_2 \rightarrow \mu_3$ and, by definition of $\leadsto$ and reflexivity of $\leq_f$, also $\mu_1 \leadsto \mu_3 \geq_f \mu_3$. $\square$

Consequently, since $\leq_f$ is a well-quasi order (see Th.5.2 in [80], which states that boundedness and coverability are decidable for cEOSs), mEOSs with $\leq_f$ are well-structured transition systems (WSTS; see [59]). Clearly, $\leq_f$ is decidable. Moreover, mEOSs have the effective pred-basis property. This is because, $\uparrow \text{Pred}_M(\uparrow \mu) = \uparrow \{\mu\} \cup \uparrow \text{Pred}_S(\uparrow \mu)$ and $\mathbf{E}_S$ has the effective pred-basis property [80], where $\uparrow X$ denotes the upward-closure of X .⁶

Lemma 19. For an EOS $\mathbf{E}$ and a nested marking μ , we have $\uparrow \text{Pred}_M(\uparrow \mu) = \uparrow \mu \cup \uparrow \text{Pred}_S(\uparrow \mu)$. [45]

Proof. If $\mu_1 \in \uparrow \text{Pred}_M(\uparrow \mu)$, then there are μ' and μ'' such that $\mu_1 \geq_f \mu' \leadsto \mu'' \geq_f \mu$. If $\mu' \geq_f \mu''$, then, by transitivity of $\leq_f$, we have that $\mu_1 \in \uparrow \mu$. If $\mu' \rightarrow \mu''$, then $\mu' \in \text{Pred}_S(\uparrow \mu)$ and, hence, $\mu_1 \in \uparrow \text{Pred}_S(\uparrow \mu)$. Vice-versa, since $\rightarrow \subseteq \leadsto$, we have $\text{Pred}_S(\mu) \subseteq \text{Pred}_M(\mu)$ and, thus, $\uparrow \text{Pred}_S(\uparrow \mu) \subseteq \uparrow \text{Pred}_M(\uparrow \mu)$. Moreover, since $\mu' \geq_f \mu$ implies $\mu' \in \text{Pred}_M(\mu)$, we have $\uparrow \{\mu'\} \subseteq \uparrow \text{Pred}_M(\uparrow \mu)$. $\square$

We can then apply the theory of WSTS and obtain decidability of coverability for mEOSs.

Lemma 20. $(\leq_f, 0)$ -coverability for mEOSs is decidable. [45]

Since each $(\leq_f, \omega)$ -run in $\mathbf{E}_S$ is a $(\leq_f, 0)$ -run in $\mathbf{E}_M$ and vice-versa, we have that $(\leq_f, 0)$ -coverability for $\mathbf{E}_M$ coincides with $(\leq_f, \omega)$ -coverability for $\mathbf{E}_S$, which, in turn, coincides with $(\leq_f, \omega)$ -reachability for $\mathbf{E}_S$ (Cor. 6). We thus obtain the following theorem.

Theorem 9. $(\leq_f, \omega)$ -reachability is decidable for EOSs. [45]

Concerning the complexity of $(\leq_f, \omega)$ -reachability for EOSs, this problem extends $(\leq_f, \omega)$ -reachability for cEOSs, which is equivalent to $(\leq_f, 0)$ -coverability for cEOSs. By noting that these can encode PN coverability, we obtain a lower bound for $(\leq_f, \omega)$ -reachability for EOSs.

⁶I.e., if X is a set of markings, $\uparrow X$ is the set of markings μ such that $\mu \geq_f x$ for some $x \in X$; also, $\uparrow \mu$ denotes $\uparrow \{\mu\}$.

	Problem	$\leq_f$	$\leq_o$	$\leq_s$
cEOS	0-reach.	undec. (Th 5.5 [80])	undec. (Th 5.5 [80])	undec. (Th 5.5 [80])
	cover.	dec. (Th 5.2 [80])	undec. [2CM]	undec. [2CM]
	ℓ -reach./cover. for $\ell \in \mathbb{N}_0$	dec. [comp.]	undec. [comp.]	undec. [comp.]
	ω -reach./cover	dec. [comp.]	undec. [comp.]	undec. [comp.]
EOS	0-reach	undec. (Th 4.3 [80])	undec. (Th 4.3 [80])	undec. (Th 4.3 [80])
	cover.	undec. (Th 4.3 [80])	undec. [cEOS]	undec. [cEOS]
	ℓ -reach./cover. for $\ell \in \mathbb{N}_0$	undec. $[\mathcal{G}]$	undec. [cEOS]	undec. [comp.]
	ω -reach./cover	dec. [WSTS]	undec. [cEOS]	undec. [comp.]

Table 7.1: Decidability status of lossy problems for full-, object-, and system-lossy EOS and cEOS. $\mathbb{N}_0$ denotes $\mathbb{N} \setminus \{0\}$. References are put next to already known results. The labels next to our results indicate the techniques used to obtain them: *[comp.]* - compatibility; *[2CM]* - 2CM reachability; *[cEOS]* - generalization of cEOS results; *[G]* - lossiness-counter gadget $\mathcal{G}$ merging; *[WSTS]* - WSTS theory.

Theorem 10. $(\leq_f, \omega)$ -reachability is EXPSPACE-hard for EOSs. [45]

We have completely charted the decidability status of all lossy-reachability problems for three lossiness relations: full-lossiness $\leq_f$, object-lossiness $\leq_o$, and system-lossiness $\leq_s$. The decidability landscape is summarized in Tab. 7.1. For cEOSs, proper lossy-reachability coincides with standard coverability under the respective lossiness quasi order. All problems for object- and system-lossy EOSs and cEOSs are undecidable. This is enabled by the fact that the orders $\leq_o$ and $\leq_s$ are not well-quasi orders (cf. [88]). For full-lossy EOSs, all $(\leq_f, \ell)$ -reachability problems are undecidable even if they do not coincide with standard coverability under $\leq_f$. The most interesting result is the decidability of $(\leq_f, \omega)$ -reachability for EOSs. This result follows from the fact that each quasi order $\leq$ induces a WSTS when interpreted over $(\leq, \omega)$ -runs (cf. [93]). This problem is at least as hard as $\leq_f$ -coverability for cEOSs, which, in turn, extends PN coverability. This yields an EXPSPACE lower-bound. The precise complexity of $(\leq_f, \omega)$ -reachability for EOSs and, to the best of our knowledge, of $\leq_f$ -coverability for cEOSs, remains uncharted. We aim to fill this gap in future works.

Decidability of $(\leq_f, \omega)$ -reachability enables, in principle, the analysis of EOS models, e.g., of business processes where resources may be lost both at the system and object level.

However, where undecidability applies, we may still perform verification by employing partial procedures, e.g., by resorting to bounded model checking approaches. Recently, in [34], a Maude encoding of EOSs was proposed and reachability searches on a bounded EOS were performed. However, to the best of our knowledge, there is no tool that natively addresses (bounded) model checking of lossy EOSs. Such a tool should also be able to express formulas about the number and distribution of the lossy steps in the runs. This feature is reminiscent of program definitions in the temporal operators of Dynamic Propositional Logic (PDL) [15, 60]. Interestingly, recent Answer Set Programming tools [62], such as Telingo [32], support PDL constraints, which may be used to perform bounded model checking. A recent prototype [112] is discussed in [46]. The development of such a tool would enable us to practically verify the robustness of EOS models.

7.9 Tool to verify lossy Petri Nets and lossy EOSs

We provided an SMT encoding for PNs and built BMC tools. Here, we explore an alternative encoding of PNs and EOSs, to be able to compare the two approaches. In this section, we discuss the prototype for the simulation and analysis of lossy-PNs and lossy-EOSs based on Answer Set Programming (ASP) technologies. Specifically, we aim at simulating and analyzing bounded lossy-EOS runs using Telingo [32], a specialization of the ASP system Clingo [61] to temporal domains. Implementations of PN variants in Clingo are [10] and [53]. However, none of them takes lossy PNs and EOSs in consideration and does not experiment with the temporal features of Telingo. In fact, the most appealing feature of Telingo is its support in constraints of Temporal Equilibrium Logic over finite traces (TEL_f) formulas, which we expect to be an excellent tool to flexibly control the amount of lossiness in the simulated runs.

7.9.1 Telingo

Telingo specializes the ASP solver Clingo to temporal domains and uses a logic program to specify finite runs. The program can use the scopes: *initial*, *dynamic*, *always*, *final*, which are evaluated at the first, each except first, each, and last step, respectively. Rule bodies can refer to the extension of the previous configuration by prefixing the literals with a prime. Finite-, linear-time formulas can appear in the dedicated atom `&tel` in constraints and behind default negation. For example, the constraint `:- &tel{>? (a>a)}` filters out all runs that eventually

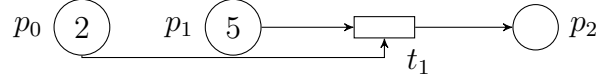


Figure 7.7: The PN with initial marking in Ex. 8.

reach ($>?$ stands for eventually reach) a configuration C with successor C' ($>$ stands for next) where a is true on both. Telingo can be called setting the option `--imax` to a number of maximal step to be simulated. In the standard configuration, Telingo stops as soon as it finds a finite run satisfying the program or exceeds `--imax`.

7.10 PNs in Telingo

7.10.1 PNs in Logic Programs

Example 8. Fig. 7.7 depicts the PN $(\{p_0, p_1, p_2\}, \{t_1\}, F)$ where $F(p_0, t_1) = F(p_1, t_1) = F(t_1, p_2) = 1$ and $F(t_1, p_0) = F(t_1, p_1) = F(p_2, t_1)$, with initial marking $\mu_0 = \{\{2p_0, 5p_1\}\}$. The net reaches $\mu_1 = \{\{3p_0, 2p_2\}\}$, covers $\mu_2 = \{\{\}\}$, reaches a deadlock after two steps, and is not 1-safe.

The standard syntax for PNs is the PNML language [71]. For example, the input PNs provided by the MCC [85] are provided in PNML syntax. The first task we faced was the translation of PNML files into Logic Program (LP) files for Telingo. This required to fix a syntax to specify PNs in LP. Inspired by previous works (see, e.g., [10, 53]), we used the following solutions:

- The number n of places and m of transitions are specified by two constants via the directives `#const numPlaces=n` and `#const numPlaces=m`. The names of the places and transition is abstracted away, i.e., they range in $\{0, \dots, n-1\}$ and $\{0, \dots, m-1\}$ respectively.
- Each pre-condition $F(p, t) = n$ and post-condition $F(t, p) = n$ is explicitly specified only if $n > 0$ using the fact `pre(p, t, n)` and `post(p, t, n)`, respectively. These facts must be available during the whole computation and, thus, are put in the scope of the `#program always` directive.
- The file contains also the specification of the initial marking, which is represented as a function from places to numbers. Specifically, for each place p hosting $n \in \mathbb{N}$ tokens (possibly $n = 0$), we add the fact `mark(p, n)` to the scope of the `#program initial`

directive. Finally, we clean the Telingo output using the directive `#show mark/2.`

Example 9. The LP specification of the PN in Ex. 8 is:

<pre>1 #program always. 2 #const numPlaces=3. 3 #const numTransitions=1.</pre>	<pre>4 pre(0,0,1). 5 pre(1,0,1). 6 post(2,0,1). 7 #program initial.</pre>	<pre>8 mark(0,2). 9 mark(1,5). 10 mark(2,0). 11 #show mark/2.</pre>
--	---	---

Using ANTLR, we built an open source tool [112], implemented in C++, to produce LP specifications out of MCC benchmarks.

7.10.2 PN dynamics

The PN dynamics can be easily implemented in Telingo using the standard guess-and-check methodology of ASP: at each step, we 1) sample a transition using a choice rule, 2) check whether it is enabled on the previous marking using a constraint, and 3) deduce the facts encoding the new marking using a couple of simple rules that take in consideration the sampled transition, its conditions, and the previous marking. This last step is done using the rules

<pre>1 mark(P,K-N+M) :- pre(P,T,N), post(P,T,M), 'mark(P,K), fire(T). 2 mark(P,K+M) :- not pre(P,T,_), post(P,T,M), 'mark(P,K), fire(T).</pre>	<pre>3 mark(P,K-N) :- pre(P,T,N), not post(P,T,_), 'mark(P,K), fire(T). 4 mark(P,K) :- not pre(P,T,_), not post(P,T,_), 'mark(P,K), fire(T).</pre>
--	--

The lossy dynamics is supported by allowing Telingo to possibly sample, next to the transitions at phase 1, also the `lossy` flag and, consequently, a sub-marking by firing the choice rule

```
1 {mark(P,1..N)}=1 :- 'mark(P,N), lossy.
```

Assuming that the dynamics is encoded in `dynamic.lp`, the simulation of the runs of maximum length n of a PN encoded in `pn.lp` can be executed as follows:

```
1 Telingo dynamic.lp pn.lp 0 --imax=n
```

7.10.3 PN Verification Problems

We considered several problems from the MCC. Since Telingo does not natively support branching time formulas, we focused on linear paths namely reachability/coverability, deadlock detection, and 1-safeness. Since Telingo simulates runs incrementally, it is sufficient to check these

properties just at the last step of the finite run. In fact, reachability/coverability and deadlock detection are all eventuality properties. Moreover, also the opposite of 1-safeness, i.e., whether a non-1-safe marking can be reached, is of the same type.

Example 10. To check the reachability in Ex. 8 of the marking $(1, 0, 2)$, we need the rules

<pre>1 #program final. 2 :- not mark(0,1).</pre>	<pre>3 :- not mark(1,0). 4 :- not mark(2,2).</pre>
--	--

Coverability can be similarly be specified using the rules

<pre>1 #program final. 2 :- mark(1,N), N<1.</pre>	<pre>3 :- mark(1,N), N<0. 4 :- mark(2,N), N<2.</pre>
--	--

Deadlock reachability requires to check the enabledness of all transitions and, so, requires their enumeration in a dedicated predicate.

<pre>1 #program final. 2 transition(0..numTransitions-1). 3 disabled(T) :- transition(T), pre(P,T,N), mark(P,M), M<N.</pre>	<pre>4 enabled(T) :- not disabled(T), transition(T). 5 nonDeadlock :- enabled(T). 6 deadlock :- not nonDeadlock. 7 :- not deadlock.</pre>
--	---

Finally, (the opposite of) 1-safeness is specified by

<pre>1 #program final.</pre>	<pre>2 unsafe :- mark(P,N), N>1. 3 :- not unsafe.</pre>
------------------------------	--

By adding these rules, if no maximum number m of steps is signaled, Telingo will return a finite run witnessing the property, if it exists, or will never terminate, otherwise. If m is provided, Telingo will always terminate, but will return a witnessing run of at most m steps.

For all these properties, we can seamlessly restrict the analysis to runs with at most $\ell \in \mathbb{N} \cup \{|\mathbb{N}|\}$ many lossy-steps for any $\ell \in \mathbb{N}$. For example, the rules

<pre>1 #program initial.</pre>	<pre>2 :- &tel{>?(lossy >(>? lossy))}.</pre>
--------------------------------	---

filter out all runs with at least two distinct lossy steps, so as to output only solutions witnessing the existence of suitable runs with at most $\ell = 1$ lossy step. The parameter ℓ can be controlled by nesting the string `>(>? lossy)` appropriately.

problem	lossiness	–imax =5 (s)	–imax =10 (s)	–imax =20 (s)	TAPAAL (s)
deadlock	none	UNSAT in 0.052	SAT in 0.622	SAT in 82.754	SAT in $5e-6$
deadlock	any	SAT in 0.009	SAT in 0.010	SAT in 0.009	NA
1-safeness	none	UNSAT in 0.010	UNSAT in 0.014	UNSAT in 0.027	UNSAT in 0
1-safeness	any	UNSAT in 0.013	UNSAT in 0.018	UNSAT in 0.032	NA

Table 7.2: Comparative Results with TAPAAL for the Eratosthenes-PT-010 PN from the MCC benchmarks [85].

PN	problem	lossiness	–imax =5	–imax =10	–imax =20
Eratosthenes-PT-010	deadlock	none	UNSAT in 0.012 s	SAT in 0.013	SAT in 0.013
Eratosthenes-PT-010	deadlock	any	SAT in 0.009s s	SAT in 0.009	SAT in 0.009
Eratosthenes-PT-010	1-safeness	none	UNSAT in 0.011s	SAT in 0.014	SAT in 0.023
Eratosthenes-PT-010	1-safeness	any	SAT in 0.010s	SAT in 0.010	SAT in 0.009

Table 7.3: Experimental Results with the Eratosthenes-PT-010 PN from the MCC benchmarks.

7.10.4 PN experiments

In the running example above, we used a simple PN to illustrate the concepts. However, for our experimentation, we considered several standard benchmarks taken from MCC [85]

which range across various industrial case studies and have different sizes of the PNs. We experimented with the analysis of deadlock reachability and 1-safeness, for various maximal numbers of steps (5, 10, and 20): these properties can be expressed without any expert knowledge on the PN structure. We compared the output of our prototype for all benchmarks with the output provided, on the same instances, by the state-of-the-art tool TAPAAL 3.9.3 [75]. The outputs match exactly; an indication of the correctness of the results, thereby giving our prototype a tool confidence of 100% (despite noncompetitive times). We analyzed the benchmarks for both runs without lossy steps and with arbitrarily many lossy steps. A subset of our results and comparisons (for no loss) on a single PN is reported in Tab. 7.2.

7.11 EOSs in Telingo

The implementation of EOS in Telingo was harder than the implementation of PNs. This is because the nesting of tokens and the complex EOS semantics required a more careful approach.

7.11.1 EOSs in Logic Programs

As for PNs, we started by fixing a syntax for EOS in logic programming. The major difference with PNs is the presence of events and of nested tokens. Events $\langle \hat{\tau}, \theta \rangle$ are specified by a name in a predicate `event/1`, a mapping of the name to τ in a predicate `eventSys/2`, and a mapping of the name to each transition in θ with multiplicity in a predicate `eventObj/3`

Example 11. The event `takingOff` = $\langle \text{takeOff}, \{\{\text{move}\}\} \rangle$ in Ex. 5 is specified by the facts (1) `event(takingOff)`, (2) `eventSys(takingOff, takeOff)`, and (3) `eventObj(takingOff1, move, 1)`.

We came up with two encoding of nested markings. The first aims at representing nested tokens directly by linearizing the carried marking in a tuple. This requires to provide an order among the places and the usage of functional symbols as well as external Python functions.

Example 12. 2 nested tokens $\langle \text{base}, \{\{2\text{batt1}\}\} \rangle$ for the EOS in Ex. 5 are specified by the fact `nM(2, base, (2, 0))` assuming the order `batt1 ≤ batt2` for the object net `drone`.

The second representation employs a purely relational representation of each token in a relation `tok/3`. It keeps track of a token id, the token place, and the parent token (i.e., the token at the system level carrying it), if it exists (otherwise, a dummy constant `_sys` is put in its place). The choice of identifiers is irrelevant as long as they form a primary key for `tok`.

Example 13. One nested token from Ex. 11 is equivalently captured by the facts (1) `tok(0, base, _sys)`, (2) `tok(1, batt1, 0)`, and (3) `tok(2, batt1, 1)`.

7.11.2 EOS dynamics

When compared to PNs, EOS dynamics has to take care of two main aspects: the choice of enabling modes (λ, ρ) next to events e , and the manipulation of nested markings. While modes range, in principle, over an infinite domain, we can restrict the choice over a finite set, by recalling that λ should be a sub-marking of the current marking. After choosing a λ and checking its compatibility with the enabling predicate for e , the PN markings $\Pi^1(\rho)$ and $\Pi_N^2(\rho)$ are univocally determined, for each object net $N \in \mathcal{N}$. To materialize these projection in a nested marking ρ , we make use of the relational representation of tokens. This allows us to easily assign children tokens to their parents non-deterministically. Afterwards, we convert ρ in the nested token representation and update the marking. This solution requires the call of external

Python functions handle the tuples for internal markings. The preliminary simulations on the EOS in Ex. 5 were not optimal. We are currently working on a better implementation.

7.11.3 EOS verification

We specify EOS properties analogously to Sec. 7.10.3, but taking into account nested tokens. For example, the reachability of the target $\langle \text{base}, \{\{\text{batt1}, \text{batt1}\}\} \rangle$ is checked by the rules

<pre>1 #program final. 2 :- not nM(1,base,(2,0)).</pre>	<pre>3 :- nM(N,base,Tup), Tup != (2,0). 4 :- nM(N,field,Tup).</pre>
---	---

Its coverability at the system net level is specified by rules 1 and 2 above. Coverability at the object level is specified by the rules

<pre>1 #program final. 2 covered :- nM(1,base,Tup), Tup <=</pre>	<pre>(2,0). 3 :- not covered.</pre>
---	-------------------------------------

Full coverability is checked by

<pre>1 #program final. 2 covered :- nM(N,base,Tup), Tup <=</pre>	<pre>(2,0). 3 :- not covered.</pre>
---	-------------------------------------

Even in this case, the specification of the amount of lossiness is orthogonal to that of the verification property and is performed using the same constraints as shown in Sec. 7.10.3.

We explored the applicability of Telingo to the simulation and analysis of lossy PNs and EOSs. Our approach is similar to [10, 53], but we additionally encoded lossiness, EOSs, and provided a translator from PNML to LP syntax. We also conducted preliminary tests on the verification of deadlock reachability and 1-safeness of lossy PNs on runs with zero or arbitrarily many lossy steps. When compared with state-of-the-art tools like TAPAAL, we obtained sound results, yet with optimizable performances. On the one hand, the specification of lossy runs was especially elegant and flexible in Telingo, e.g., when compared to SMT-based PN verifiers [111]. Our approach addresses PNs in general and is applicable to PNs with reset, transfer, and inhibitory arcs. On the other hand, TAPAAL does not natively support the analysis of properties under lossiness, unless somehow encoded in the PN itself. As a byproduct of our experiments, we provide a standalone utility [112] compatible with Linux to translate standard PNML files to the PN LP syntax. We hope that this translator will provide ASP practitioners with a convenient tool to approach PN benchmarks. The prototype for EOS needs further devel-

opment before tests can be meaningfully conducted. This task is challenging, since PN reachability is non-elementary and several reachability problems for lossy EOSs are undecidable in general [45]. However, to the best of our knowledge, there are no available tools dedicated to lossy EOSs apart from ours.

Conclusion

In this chapter, we charted the decidability boundaries for reachability, coverability in EOSs in the case of lossiness at the system, object levels and both. We also implemented a Telingo based open source tool to verify lossy Petri nets and lossy EOSs.

Chapter 8

Conclusions

We summarize the work done in this dissertation:

- In Chapter. 2 we discussed the various models for unbounded concurrency. First, we discussed Petri nets, the most widely studied model, where the tokens are undistinguishable. Second, we studied ν -nets, where the tokens can be distinguished using identifiers and there can be labeled arcs to enable the movement of particular tokens, which allows for a richer representation than Petri nets. We saw how ν -nets can be used to represent identifiable clients in a system with single server and unbounded number of clients. Third, we explored a particular class of higher order nets, where Petri nets can be nested. When the nesting depth is restricted to two, we obtain Elementary Object Systems, which are natural for representing clients as a nested Petri net within the server Petri net. We explored the suitability of each of these models to represent the unbounded client server systems case study.
- In Chapter. 3 we discussed the various temporal logics that can be used to specify properties of the models discussed in Chapter. 2. We explored the expressibility of Linear Temporal Logic, Linear Temporal Logic with integer arithmetic $LT L_{LIA}$ and First Order Logic with Monodic restriction $FOTL_1$. In each of these logics, we looked at their applicability in expressing properties of the above concurrent models in a natural manner.
- In Chapter. 4 we explored the motivation behind verification of unbounded client server systems and the research questions that we aim to answer in this thesis. We discussed the algorithm of bounded model checking and its extension, two dimensional bounded model checking (2D-BMC). 2D-BMC is particularly useful to verify models such as Petri nets

where there is unboundedness in the concurrency as well as with respect to the temporal modality.

- In Chapter. 5, we implemented a tool to perform 2D-BMC on unbounded Petri nets with specifications written in LTL_{LIA} , a variant of Linear Temporal Logic. We described the design of the tool, the SMT encoding and the experiments where it competes with the state of the art Petri net verification tools on competition benchmarks. This is the first of its kind to support true concurrent semantics of Petri nets.
- In Chapter. 6, we implemented another tool to verify properties of ν -nets, which are Petri nets where the tokens are distinguished. We employed a variant of First Order Logic $FOTL_1$, to represent the specifications and we verified the specifications using 2D-BMC algorithm and made use of SMT solvers to verify them.
- In Chapter. 7, we charted the decidability status of reachability, coverability problems in Elementary Object Systems, which are a type of nested Petri nets, in the context of lossiness. We additionally built a tool using Answer Set Programming, to verify the reachability, safety and deadlock properties on lossy Petri nets and lossy EOSs.

Future Work

In this work, we took the approach of representing single server multiple client systems as a single component and represented them as a single ν -net. As part of future work, we shall explore verification of a system of ν -nets, with several underlying components.

We represented the single server multiple client setting using EOSs. The following are natural extensions - the single client multiple server, multiple client and multiple server settings. In a single client multiple server setting, we may represent client process as a token net whose behaviour changes dynamically in various places, corresponding to the server behaviour. It would be interesting to implement tools to verify programs following these paradigms, as they are widely in use in industry.

We worked with interleaving and true concurrent semantics of Petri nets. In practice, while analysing the time spent by the tools, we see that the SMT invocation takes relatively lesser time in compared to the formulation of the constraints. In case of the true concurrent semantics, we have several more constraints that are added, to ensure that the firing of some

transitions concurrently does not disable the others and so on. As part of future work, we would like to have an optimal encoding of the PN with true concurrent semantics, such that the two can be compared more clearly.

We introduced 2D-BMC, which is an extension of the standard bounded model checking algorithm. It would be interesting to explore the complexity and completeness criteria of this algorithm.

The model checking problem for Linear Temporal Logic with integer arithmetic is undecidable, it would be interesting to see if decidability can be achieved in variants of this logic.

References

- [1] Model Checking Contest 2022 Rules. <https://mcc.lip6.fr/pdf/rules.pdf>, September 2022.
- [2] Results of Model Checking Contest. <https://mcc.lip6.fr/results.php>, September 2022.
- [3] Parosh Aziz Abdulla, S. Purushothaman Iyer, and Aletta Nylén. Unfoldings of unbounded Petri nets. In *Computer Aided Verification, 12th International Conference, CAV 2000, Chicago, IL, USA, July 15-19, 2000, Proceedings*, volume 1855 of *Lecture Notes in Computer Science*, pages 495–507. Springer, 2000.
- [4] Parosh Aziz Abdulla, S. Purushothaman Iyer, and Aletta Nylén. SAT-solving the coverability problem for Petri nets. *Form. Methods Syst. Des.*, 24(1):25–43, January 2004.
- [5] Parosh Aziz Abdulla and Bengt Jonsson. Ensuring completeness of symbolic verification methods for infinite-state systems. *Theor. Comput. Sci.*, 256(1-2):145–167, 2001.
- [6] Parosh Aziz Abdulla, Bengt Jonsson, Ahmed Rezine, and Mayank Saksena. Proving liveness by backwards reachability. In *CONCUR 2006 - Concurrency Theory, 17th International Conference, CONCUR 2006, Bonn, Germany, August 27-30, 2006, Proceedings*, volume 4137 of *Lecture Notes in Computer Science*, pages 95–109. Springer, 2006.
- [7] C. Aiswarya. On network topologies and the decidability of reachability problem. In *International Conference on Networked Systems*, volume 12129 of *LNCS*, pages 3–10, 2020.
- [8] Nicolas Amat, Bernard Berthomieu, and Silvano Dal-Zilio. On the combination of polyhedral abstraction and smt-based model checking for Petri nets. In *PETRI NETS*, LNCS, pages 164–185. Springer, 2021.
- [9] Nicolas Amat, Silvano Dal-Zilio, and Thomas Hujsa. Property directed reachability for generalized Petri nets. In *Tools and Algorithms for the Construction and Analysis of Sys-*

- tems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I, volume 13243 of *Lecture Notes in Computer Science*, pages 505–523. Springer, 2022.
- [10] Saadat Anwar et al. Encoding higher level extensions of Petri nets in answer set programming. In *LPNMR*, volume 8148 of *LNCS*, pages 116–121, 2013.
 - [11] Krzysztof R. Apt and Dexter Kozen. Limits for automatic verification of finite-state concurrent systems. *Inf. Process. Lett.*, 22(6):307–309, 1986.
 - [12] Gilles Audemard, Alessandro Cimatti, Artur Kornilowicz, and Roberto Sebastiani. Bounded model checking for timed systems. In *Formal Techniques for Networked and Distributed Systems - FORTE 2002, 22nd IFIP WG 6.1 International Conference Houston, Texas, USA, November 11-14, 2002, Proceedings*, volume 2529 of *Lecture Notes in Computer Science*, pages 243–259. Springer, 2002.
 - [13] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT Press, 2008.
 - [14] A. R. Balasubramanian, Javier Esparza, and Marijana Lazic. Complexity of verification and synthesis of threshold automata. In *Automated Technology for Verification and Analysis - 18th International Symposium, ATVA 2020, Hanoi, Vietnam, October 19-23, 2020, Proceedings*, volume 12302 of *Lecture Notes in Computer Science*, pages 144–160. Springer, 2020.
 - [15] Philippe Balbiani and Emiliano Lorini. Ockhamist propositional dynamic logic: A natural link between PDL and CTL. In *Logic, Language, Information, and Computation Proceedings*, volume 8071 of *LNCS*, pages 251–265, 2013.
 - [16] Matteo Baldoni, Cristina Baroglio, and Roberto Micalizio. Fragility and robustness in multiagent systems. In *International Workshop on Engineering Multi-Agent Systems*, volume 12589 of *LNCS*, pages 61–77, 2020.
 - [17] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength SMT solver. In *Tools and Algorithms for*

- the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I*, volume 13243 of *Lecture Notes in Computer Science*, pages 415–442. Springer, 2022.
- [18] Clark W. Barrett, Christopher L. Conway, Morgan Deters, Liana Hadarean, Dejan Jovanovic, Tim King, Andrew Reynolds, and Cesare Tinelli. CVC4. In *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*, volume 6806 of *Lecture Notes in Computer Science*, pages 171–177. Springer, 2011.
- [19] Clark W. Barrett and Cesare Tinelli. Satisfiability modulo theories. In Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors, *Handbook of Model Checking*, pages 305–343. Springer, 2018.
- [20] E. Bartocci and Y. Falcone. *Lectures on Runtime Verification: Introductory and Advanced Topics*. Lecture Notes in Computer Science. Springer International Publishing, 2018.
- [21] Francesco Belardinelli, Panagiotis Kouvaros, and Alessio Lomuscio. Parameterised verification of data-aware multi-agent systems. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 98–104. ijcai.org, 2017.
- [22] Bernard Berthomieu and François Vernadat. Time Petri nets analysis with TINA. In *Third International Conference on the Quantitative Evaluation of Systems (QEST 2006), 11-14 September 2006, Riverside, California, USA*, pages 123–124. IEEE Computer Society, 2006.
- [23] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, Ofer Strichman, and Yunshan Zhu. Bounded model checking. *Adv. Comput.*, pages 117–148, 2003.
- [24] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. Symbolic model checking without bdds. In *TACAS, LNCS*, pages 193–207. Springer, 1999.

- [25] Alexander Bilgram, Peter Gjørl Jensen, Thomas Pedersen, Jiri Srba, and Peter Haahr Taankvist. Methods for efficient unfolding of colored Petri nets. *Fundam. Informaticae*, 189(3-4):297–320, 2022.
- [26] Michael Blondin, Alain Finkel, Christoph Haase, and Serge Haddad. Approaching the coverability problem continuously. In *TACAS, LNCS*, pages 480–496. Springer, 2016.
- [27] Ahmed Bouajjani and Richard Mayr. Model checking lossy vector addition systems. In *Annual Symposium on Theoretical Aspects of Computer Science*, volume 1563 of *LNCS*, pages 323–333, 1999.
- [28] Ioana Boureanu, Panagiotis Kouvaros, and Alessio Lomuscio. Verifying security properties in unbounded multiagent systems. In Catholijn M. Jonker, Stacy Marsella, John Thangarajah, and Karl Tuyls, editors, *Proceedings of the 2016 International Conference on Autonomous Agents & Multiagent Systems, Singapore, May 9-13, 2016*, pages 1209–1217. ACM, 2016.
- [29] Daniel Brand and Pitro Zafiropulo. On communicating finite-state machines. *J. ACM*, 30(2):323–342, 1983.
- [30] Michael C. Browne, Edmund M. Clarke, and Orna Grumberg. Reasoning about networks with many identical finite state processes. *Inf. Comput.*, 81(1):13–31, 1989.
- [31] J. Richard Büchi. On a decision method in restricted second order arithmetic. In Ernest Nagel, Patrick Suppes, and Alfred Tarski, editors, *Logic, Methodology and Philosophy of Science (Proc. 1960 Internat. Congr.)*, pages 1–11, Stanford, Calif., 1962. Stanford University Press.
- [32] Pedro Cabalar. Temporal ASP: from logical foundations to practical use with telingo. In *Reasoning Web. Declarative Artificial Intelligence*, volume 13100 of *LNCS*, pages 94–114, 2021.
- [33] Zhanghua Cai, Yantao Zhou, Yihong Qi, Weihua Zhuang, and Lei Deng. A millimeter wave dual-lens antenna for iot-based smart parking radar system. *IEEE Internet Things J.*, pages 418–427, 2021.

- [34] Lorenzo Capra and Michael Köhler-Bussmeier. Modelling adaptive systems with nets-within-nets in maude. In *International Conference on Evaluation of Novel Approaches to Software Engineering*, pages 487–496, 2023.
- [35] Pierre Chambart and Philippe Schnoebelen. Mixing lossy and perfect fifo channels. In *International Conference on Concurrency Theory*, volume 5201 of *LNCS*, pages 340–355, 2008.
- [36] Saksham Chand, Yanhong A. Liu, and Scott D. Stoller. Formal verification of multi-paxos for distributed consensus. In John Fitzgerald, Constance Heitmeyer, Stefania Gnesi, and Anna Philippou, editors, *FM 2016: Formal Methods*, pages 119–136, Cham, 2016. Springer International Publishing.
- [37] Wachiraphan Charoenwet, Patanamon Thongtanunam, Van-Thuan Pham, and Christoph Treude. An empirical study of static analysis tools for secure code review. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, page 691–703. Association for Computing Machinery, 2024.
- [38] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nikolai Zeldovich. Using crash hoare logic for certifying the fscq file system. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP '15*, page 18–37, New York, NY, USA, 2015. Association for Computing Machinery.
- [39] Allan Cheng, Javier Esparza, and Jens Palsberg. Complexity results for 1-safe nets. *Theor. Comput. Sci.*, 147(1&2):117–136, 1995.
- [40] Vassilis Christophides, Richard Hull, Gregory Karvounarakis, Akhil Kumar, Geliang Tong, and Ming Xiong. Beyond discrete e-services: Composing session-oriented services in telecommunications. In *Technologies for E-Services, Second International Workshop, TES 2001, Rome, Italy, September 14-15, 2001, Proceedings*, volume 2193 of *Lecture Notes in Computer Science*, pages 58–73. Springer, 2001.
- [41] E. M. Clarke and O. Grumberg. Avoiding the state explosion problem in temporal logic model checking. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing, PODC '87*, page 294–303, New York, NY, USA, 1987. Association for Computing Machinery.

- [42] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement for symbolic model checking. *J. ACM*, 50(5):752–794, September 2003.
- [43] Edmund M. Clarke, William Klieber, Milos Nováček, and Paolo Zuliani. Model checking and the state explosion problem. In *Tools for Practical Software Verification, LASER, International Summer School 2011, Elba Island, Italy, Revised Tutorial Lectures*, volume 7682 of *Lecture Notes in Computer Science*, pages 1–30. Springer, 2011.
- [44] Edmund M. Clarke, William Klieber, Miloš Nováček, and Paolo Zuliani. *Model Checking and the State Explosion Problem*, pages 1–30. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012.
- [45] Francesco Di Cosmo, Soumodev Mal, and Tephilla Prince. Deciding reachability and coverability in lossy EOS. In *Proceedings of the International Workshop on Petri Nets and Software Engineering 2024 co-located with the 45th International Conference on Application and Theory of Petri Nets and Concurrency (PETRI NETS 2024), June 24 - 25, 2024, Geneva, Switzerland*, volume 3730 of *CEUR Workshop Proceedings*, pages 74–95. CEUR-WS.org, 2024.
- [46] Francesco Di Cosmo and Tephilla Prince. Bounded verification of petri nets and eoss using telingo: An experience report. In *Proceedings of the 39th Italian Conference on Computational Logic, Rome, Italy, June 26-28, 2024*, volume 3733 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2024.
- [47] Wojciech Czerwinski, Slawomir Lasota, Ranko Lazic, Jérôme Leroux, and Filip Mazowiecki. The reachability problem for Petri nets is not elementary. *J. ACM*, 68(1):7:1–7:28, 2021.
- [48] Silvano Dal-Zilio. MCC: A tool for unfolding colored Petri nets in PNML format. In *Application and Theory of Petri Nets and Concurrency*, volume 12152 of *Lecture Notes in Computer Science*, pages 426–435. Springer, 2020.
- [49] Alexandre David, Lasse Jacobsen, Morten Jacobsen, Kenneth Yrke Jørgensen, Mikael H. Møller, and Jirí Srba. TAPAAL 2.0: Integrated development environment for timed-arc Petri nets. In *TACAS*, volume 7214 of *LNCS*, pages 492–497. Springer, 2012.

- [50] Maurice Dawson, Darrell Burrell, Emad Rahim, and Stephen Brewster. Integrating software assurance into the software development life cycle (sdlc). *Journal of Information Systems Technology and Planning*, 3:49–53, 01 2010.
- [51] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. Z3: an efficient SMT solver. In *TACAS, LNCS*, pages 337–340. Springer, 2008.
- [52] Nachum Dershowitz. Let’s be honest. *Commun. ACM*, 64(5):37–41, 2021.
- [53] Yannis Dimopoulos et al. Encoding reversing Petri nets in answer set programming. In *RC*, volume 12227 of *LNCS*, pages 264–271, 2020.
- [54] Alex Dixon and Ranko Lazic. Kreach: A tool for reachability in Petri nets. In *TACAS, LNCS*, pages 405–412. Springer, 2020.
- [55] Javier Esparza. Model checking using net unfoldings. *Science of Computer Programming*, 23(2):151–195, 1994.
- [56] Javier Esparza. Decidability of model checking for infinite-state concurrent systems. *Acta Informatica*, 34(2):85–107, 1997.
- [57] Javier Esparza, Ruslán Ledesma-Garza, Rupak Majumdar, Philipp J. Meyer, and Filip Niksic. An smt-based approach to coverability analysis. In *CAV, LNCS*, pages 603–619. Springer, 2014.
- [58] Paolo Felli, Alessandro Gianola, and Marco Montali. Smt-based safety checking of parameterized multi-agent systems. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pages 6321–6330. AAAI Press, 2021.
- [59] A. Finkel and Ph. Schnoebelen. Well-structured transition systems everywhere! *Theoretical Computer Science*, 256(1):63–92, 2001. ISS.
- [60] Michael J. Fischer and Richard E. Ladner. Propositional dynamic logic of regular programs. *J. Comput. Syst. Sci.*, 18(2):194–211, 1979.

- [61] Martin Gebser et al. Multi-shot ASP solving with clingo. *Theory Pract. Log. Program.*, 19(1):27–82, 2019.
- [62] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Multi-shot ASP solving with clingo. *Theory Pract. Log. Program.*, 19(1):27–82, 2019.
- [63] Thomas Geffroy, Jérôme Leroux, and Grégoire Sutre. Occam’s razor applied to the Petri net coverability problem. *Theor. Comput. Sci.*, 2018.
- [64] Patrice Godefroid, J. van Leeuwen, J. Hartmanis, G. Goos, and Pierre Wolper. *Partial-Order Methods for the Verification of Concurrent Systems: An Approach to the State-Explosion Problem*. Springer-Verlag, Berlin, Heidelberg, 1996.
- [65] Michel Hack. The recursive equivalence of the reachability problem and the liveness problem for Petri nets and vector addition systems. In *15th Annual Symposium on Switching and Automata Theory*, pages 156–164. IEEE Computer Society, 1974.
- [66] Rachid Hadjidj and Hanifa Boucheneb. On-the-fly tctl model checking for time petri nets. *Theoretical Computer Science*, 410(42):4241–4261, 2009.
- [67] Bill Haskins, Jonette Stecklein, Brandon Dick, Gregory Moroney, Randy Lovell, and James Dabney. Error cost escalation through the project life cycle. *INCOSE International Symposium*, 14:1723–1737, 06 2004.
- [68] Cong He and Zhijun Ding. More efficient on-the-fly verification methods of colored petri nets. *Computing and Informatics*, 40(1):195–215, Aug. 2021.
- [69] Keijo Heljanko. Bounded reachability checking with process semantics. In *CONCUR 2001 - Concurrency Theory, 12th International Conference, Aalborg, Denmark, August 20-25, 2001, Proceedings*, volume 2154 of *Lecture Notes in Computer Science*, pages 218–232. Springer, 2001.
- [70] Thibault Hilaire, David Ilcinkas, and Jérôme Leroux. A state-of-the-art karp-miller algorithm certified in coq. In *Tools and Algorithms for the Construction and Analysis of Systems: 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6–11, 2024, Proceedings, Part I*, page 370–389, Berlin, Heidelberg, 2024. Springer-Verlag.

- [71] Lom-Messan Hillah et al. Extending pnml scope: A framework to combine Petri nets types. *TOPNOC*, 6:46–70, 2012.
- [72] Lom-Messan Hillah, Fabrice Kordon, Laure Petrucci, and Nicolas Trèves. PNML framework: An extendable reference implementation of the Petri net markup language. In *PETRI NETS*, LNCS, pages 318–327. Springer, 2010.
- [73] Ian M. Hodkinson, Roman Kontchakov, Agi Kurucz, Frank Wolter, and Michael Zakharyashev. On the computational complexity of decidable fragments of first-order linear temporal logics. In *10th International Symposium on Temporal Representation and Reasoning / 4th International Conference on Temporal Logic (TIME-ICTL 2003), 8-10 July 2003, Cairns, Queensland, Australia*, pages 91–98. IEEE Computer Society, 2003.
- [74] Ian M. Hodkinson, Frank Wolter, and Michael Zakharyashev. Monodic fragments of first-order temporal logics: 2000-2001 a.d. In *LPAR*, pages 1–23, 2001.
- [75] Jonas Finnemann Jensen et al. TAPAAL and reachability analysis of P/T nets. *TOPNOC*, 11:307–318, 2016.
- [76] Kurt Jensen. *Coloured Petri Nets: Basic Concepts, Analysis Methods and Practical Use. Volume 1*. Springer Publishing Company, Incorporated, 2010.
- [77] Kurt Jensen, Lars Michael Kristensen, and Lisa Wells. Coloured Petri nets and cpn tools for modelling and validation of concurrent systems. *Int. J. Softw. Tools Technol. Transf.*, 9(3–4):213–254, June 2007.
- [78] Eric Y. T. Juan, Jeffrey J. P. Tsai, and Tadao Murata. Compositional verification of concurrent systems using petri-net-based condensation rules. *ACM Trans. Program. Lang. Syst.*, 20(5):917–979, September 1998.
- [79] Michael Köhler. Reachable markings of object Petri nets. *Fundamenta Informaticae*, 79(3-4):401–413, 2007.
- [80] Michael Köhler-Bußmeier. A survey of decidability results for elementary object systems. *Fundamenta Informaticae*, 130(1):99–123, 2014.

- [81] Michael Köhler-Bussmeier and Lorenzo Capra. Robustness: A natural definition based on nets-within-nets. In *International Workshop on Petri Nets and Software Engineering*, volume 3430 of *CEUR Workshop Proceedings*, pages 70–87, 2023.
- [82] Igor Konnov, Marijana Lazic, Ilina Stoilkovska, and Josef Widder. Survey on parameterized verification with threshold automata and the byzantine model checker. *Log. Methods Comput. Sci.*, 19(1), 2023.
- [83] Igor V. Konnov, Helmut Veith, and Josef Widder. On the completeness of bounded model checking for threshold-based distributed algorithms: Reachability. *Inf. Comput.*, 252:95–109, 2017.
- [84] F. Kordon, P. Bouvier, H. Garavel, L. M. Hillah, F. Hulin-Hubard, N. Amat., E. Amparore, B. Berthomieu, S. Biswal, D. Donatelli, F. Galla, , S. Dal Zilio, P. G. Jensen, C. He, D. Le Botlan, S. Li, , J. Srba, . Thierry-Mieg, A. Walner, and K. Wolf. Complete Results for the 2020 Edition of the Model Checking Contest. <http://mcc.lip6.fr/2021/results.php>, June 2021.
- [85] F. Kordon et al. Complete Results for the Model Checking Contest. <https://mcc.lip6.fr/2023/results.php>, 2023.
- [86] Panagiotis Kouvaros and Alessio Lomuscio. Parameterised verification for multi-agent systems. *Artif. Intell.*, 234:152–189, 2016.
- [87] Orna Kupferman. Automata theory and model checking. In *Handbook of Model Checking*, pages 107–151. Springer, 2018.
- [88] Slawomir Lasota. Decidability border for Petri nets with data: WQO dichotomy conjecture. In *Application and Theory of Petri Nets and Concurrency*, volume 9698 of *LNCS*, pages 20–36, 2016.
- [89] Irina A. Lomazova. Nested Petri nets - a formalism for specification and verification of multi-agent distributed systems. *Fundamenta Informaticae*, 43(1-4):195–214, 2000.
- [90] Gavin Lowe. Parameterized verification of systems with component identities, using view abstraction. *Int. J. Softw. Tools Technol. Transf.*, 24(2):287–324, 2022.

- [91] Riccardo De Masellis and Valentin Goranko. Logic-based specification and verification of homogeneous dynamic multi-agent systems. In Frank Dignum, Alessio Lomuscio, Ulle Endriss, and Ann Nowé, editors, *AAMAS '21: 20th International Conference on Autonomous Agents and Multiagent Systems, Virtual Event, United Kingdom, May 3-7, 2021*, pages 1727–1729. ACM, 2021.
- [92] Richard Mayr. Lossy counter machines. Technical report, Institute of Computer Science, Technical Institute of Munich, 1998.
- [93] Richard Mayr. Undecidable problems in unreliable computations. *Theoretical Computer Science*, 297(1-3):337–354, 2003.
- [94] Kenneth L. McMillan. Using unfoldings to avoid the state explosion problem in the verification of asynchronous circuits. In *Proceedings of the Fourth International Workshop on Computer Aided Verification, CAV '92*, page 164–177, Berlin, Heidelberg, 1992. Springer-Verlag.
- [95] Kenneth L. McMillan. *Symbolic Model Checking*. Kluwer Academic Publishers, Norwell, MA, USA, 1993.
- [96] Artur Meski, Agata Pólrola, Wojciech Penczek, Bożena Wozna-Szczesniak, and Andrzej Zbrzezny. Bounded model checking approaches for verification of distributed time Petri nets. In *Proceedings of the International Workshop on Petri Nets and Software Engineering, Newcastle upon Tyne, UK, June 20-21, 2011*, volume 723 of *CEUR Workshop Proceedings*, pages 72–91. CEUR-WS.org, 2011.
- [97] Marvin L. Minsky. *Computation: finite and infinite machines*. Prentice-Hall, Inc., USA, 1967.
- [98] T. Murata. Petri nets: Properties, analysis and applications. *IEEE*, 77(4):541–580, 1989.
- [99] Amat Nicolas. Benchmarks of Unbounded Petri Nets. <https://github.com/nicolasAmat/SMPT>, September 2022.
- [100] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. Paxos made EPR: decidable reasoning about distributed protocols. *Proc. ACM Program. Lang.*, 1(OOPSLA):108:1–108:31, 2017.

- [101] Terence Parr and Kathleen Fisher. $Ll(*)$: the foundation of the ANTLR parser generator. In *PLDI*, pages 425–436, 2011.
- [102] Doron Peled. All from one, one for all: on model checking using representatives. In Costas Courcoubetis, editor, *Computer Aided Verification*, pages 409–423, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
- [103] Carl Adam Petri. Kommunikation mit Automaten. Dissertation, Schriften des IIM 2, Rheinisch-Westfälisches Institut für Instrumentelle Mathematik an der Universität Bonn, Bonn, 1962.
- [104] Ramchandra Phawade, Tephilla Prince, and Sheerazuddin S. Bounded model checker for unbounded client-server systems, July 2025.
- [105] Ramchandra Phawade, Tephilla Prince, and S. Sheerazuddin. Bounded model checking for unbounded client server systems, 2022.
- [106] Ramchandra Phawade, Tephilla Prince, and S. Sheerazuddin. Dcmodelchecker 2.0: A bmc tool, 2022.
- [107] Ramchandra Phawade, Tephilla Prince, and S. Sheerazuddin. Dcmodelchecker tool for verification of unbounded Petri nets, 2022.
- [108] Amir Pnueli. The temporal logic of programs. In *FOCS*, pages 46–57, 1977.
- [109] Amir Pnueli, Jessie Xu, and Lenore D. Zuck. Liveness with $(0, 1, \text{infty})$ -counter abstraction. In *CAV*, pages 107–122, 2002.
- [110] Mukul R. Prasad, Armin Biere, and Aarti Gupta. A survey of recent advances in sat-based formal verification. *Int. J. Softw. Tools Technol. Transf.*, pages 156–173, 2005.
- [111] Tephilla Prince. DCMoDelChecker 2.0: A BMC tool for Unbounded PN, November 2022.
- [112] Tephilla Prince and Francesco Di Cosmo. Nets within nets telingo analyser, May 2024.
- [113] Charles Rackoff. The covering and boundedness problems for vector addition systems. *Theoretical Computer Science*, 6(2):223–231, 1978.

- [114] Fernando Rosa-Velardo and David de Frutos-Escrig. Name creation vs. replication in Petri net systems. In *Petri Nets and Other Models of Concurrency - ICATPN 2007, 28th International Conference on Applications and Theory of Petri Nets and Other Models of Concurrency, ICATPN 2007, Siedlce, Poland, June 25-29, 2007, Proceedings*, volume 4546 of *Lecture Notes in Computer Science*, pages 402–422. Springer, 2007.
- [115] Fernando Rosa-Velardo and David de Frutos-Escrig. Name creation vs. replication in Petri net systems. *Fundam. Informaticae*, 88(3):329–356, 2008.
- [116] James Rumbaugh, Ivar Jacobson, and Grady Booch. *Unified Modeling Language Reference Manual, The (2nd Edition)*. Pearson Higher Education, 2004.
- [117] Yunus Emre Sahin, Petter Nilsson, and Necmiye Ozay. Multirobot coordination with counting temporal logics. *IEEE Trans. Robotics*, 36(4):1189–1206, 2020.
- [118] Ocan Sankur and Jean-Pierre Talpin. An abstraction technique for parameterized model checking of leader election protocols: Application to FTSP. In Axel Legay and Tiziana Margaria, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 23rd International Conference, TACAS 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings, Part I*, volume 10205 of *Lecture Notes in Computer Science*, pages 23–40, 2017.
- [119] Philippe Schnoebelen. Lossy counter machines decidability cheat sheet. In *International Workshop on Reachability Problems*, volume 6227 of *LNCS*, pages 51–75, 2010.
- [120] Divjyot Sethi, Muralidhar Talupur, and Sharad Malik. Model checking unbounded concurrent lists. *Int. J. Softw. Tools Technol. Transf.*, 18(4):375–391, 2016.
- [121] Mary Sheeran, Satnam Singh, and Gunnar Stålmarck. Checking safety properties using induction and a sat-solver. In *Formal Methods in Computer-Aided Design, Third International Conference, FMCAD 2000, Austin, Texas, USA, November 1-3, 2000, Proceedings*, volume 1954 of *Lecture Notes in Computer Science*, pages 108–125. Springer, 2000.
- [122] Yann Thierry-Mieg. Symbolic model-checking using its-tools. In *TACAS, LNCS*, pages 231–237. Springer, 2015.

- [123] Yann Thierry-Mieg. Structural reductions revisited. In *PETRI NETS*, LNCS, pages 303–323. Springer, 2020.
- [124] Wolfgang Thomas. Automata on infinite objects. In *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*, pages 133–191. Elsevier and MIT Press, 1990.
- [125] Louis-Marie Traonouez, Didier Lime, and Olivier H. Roux. Parametric model-checking of time Petri nets with stopwatches using the state-class graph. In *Formal Modeling and Analysis of Timed Systems, 6th International Conference, FORMATS 2008, Saint Malo, France, September 15-17, 2008. Proceedings*, volume 5215 of *Lecture Notes in Computer Science*, pages 280–294. Springer, 2008.
- [126] Rüdiger Valk. Nets in computer organization. In *Petri Nets: Applications and Relationships to Other Models of Concurrency*, pages 218–233, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.
- [127] Rüdiger Valk. Object petri nets: Using the nets-within-nets paradigm. In Jörg Desel, Wolfgang Reisig, and Grzegorz Rozenberg, editors, *Lectures on Concurrency and Petri Nets, Advances in Petri Nets*, volume 3098 of *Lecture Notes in Computer Science*, pages 819–848. Springer, 2003.
- [128] Rüdiger Valk. Object Petri nets: Using the nets-within-nets paradigm. In *Advances in Petri Nets*, volume 3098 of *LNCS*, pages 819–848, 2003.
- [129] Moshe Y. Vardi and Pierre Wolper. An automata-theoretic approach to automatic program verification (preliminary report). In *Proceedings of the Symposium on Logic in Computer Science (LICS '86), Cambridge, Massachusetts, USA, June 16-18, 1986*, pages 332–344. IEEE Computer Society, 1986.
- [130] Pierre Wolper, Moshe Y. Vardi, and A. Prasad Sistla. Reasoning about infinite computation paths (extended abstract). In *24th Annual Symposium on Foundations of Computer Science, Tucson, Arizona, USA, 7-9 November 1983*, pages 185–194. IEEE Computer Society, 1983.
- [131] Mitsuharu Yamamoto, Shogo Sekine, and Saki Matsumoto. Formalization of karp-miller tree construction on petri nets. In *Proceedings of the 6th ACM SIGPLAN Conference*

on Certified Programs and Proofs, CPP 2017, page 66–78, New York, NY, USA, 2017. Association for Computing Machinery.

- [132] Gongjun Yan, Weiming Yang, Danda B. Rawat, and Stephan Olariu. Smartparking: A secure and intelligent parking system. *IEEE Intell. Transp. Syst. Mag.*, pages 18–30, 2011.
- [133] Richard M. Zahoransky, Julius Holderer, Adrian Lange, and Christian Brenig. Process analysis as first step towards automated business security. In *ECIS*, page Research Paper 46, 2016.

List of Publications

- [1] Ramchandra Phawade, S. Sheerazuddin, and Tephilla Prince, “Two dimensional bounded model checking for unbounded Petri nets”, 2025, *Under Submission*
- [2] Ramchandra Phawade, S. Sheerazuddin, and Tephilla Prince, “Verification of Unbounded Client-Server Systems with Distinguishable Clients”, 2025, *Under Submission*
- [3] Tephilla Prince, “On Verifying Unbounded Client-Server Systems”, *Multi-Agent Systems - 20th European Conference, EUMAS 2023, Naples, Italy, September 14-15, 2023, Proceedings, Springer, Lecture Notes in Computer Science, volume 14282, pages 465–471, https://doi.org/10.1007/978-3-031-43264-4_33*
- [4] Francesco Di Cosmo, Soumodev Mal and Tephilla Prince, “Deciding Reachability and Coverability in Lossy EOS”, *Proceedings of the International Workshop on Petri Nets and Software Engineering 2024 co-located with the 45th International Conference on Application and Theory of Petri Nets and Concurrency (PETRI NETS 2024), June 24 - 25, 2024, Geneva, Switzerland, CEUR Workshop Proceedings, Volume 3730, pages 74–95, <https://ceur-ws.org/Vol-3730/paper04.pdf>*
- [5] Francesco Di Cosmo, Soumodev Mal and Tephilla Prince, “Decidability and Complexity of Reachability and Coverability in Lossy EOS”, 2024, *Under Submission*
- [6] Francesco Di Cosmo and Tephilla Prince, “Bounded Verification of Petri Nets and EOSs using Telingo: An Experience Report”, *Proceedings of the 39th Italian Conference on Computational Logic, Rome, Italy, June 26-28, 2024, CEUR Workshop Proceedings, Volume 3733, <https://ceur-ws.org/Vol-3733/short6.pdf>*